

Δημιουργία διαδικτυακής εφαρμογής ταξιδιών: κοινωνικό δίκτυο TravelLight App

ΤΖΙΟΥΒΑΚΑ Γ. ΙΩΑΝΝΑ

ΠΤΥΧΙΑΚΗ ΕΡΓΑΣΙΑ

Επιβλέπων καθηγητής: Θεόδωρος Κασκάλης
Εξεταστές:

Τμήμα Εφαρμοσμένης Πληροφορικής

Πανεπιστήμιο Μακεδονίας
Θεσσαλονίκη

Ιανουάριος, 2022

Copyright ©Τζιουβάκα Ιωάννα, 2022
Με επιφύλαξη παντός δικαιώματος. All rights reserved.

Η έγκριση της πτυχιακής εργασίας από το Τμήμα Εφαρμοσμένης Πληροφορικής του Πανεπιστημίου Μακεδονίας δεν υποδηλώνει απαραίτητως και αποδοχή των απόψεων του συγγραφέα εκ μέρους του Τμήματος.

ΠΕΡΙΛΗΨΗ

Στην παρούσα πτυχιακή εργασία, περιγράφεται η δημιουργία ενός κοινωνικού δικτύου με κύρια θεματική τα ταξίδια. Ο σκοπός ανάπτυξης αυτής της διαδικτυακής πλατφόρμας είναι να ενημερώνει ανθρώπους που θέλουν να ταξιδέψουν σε διάφορους προορισμούς δίνοντας τους την ευκαιρία να μοιραστούν τις εμπειρίες τους, να ψυχαγωγεί τους χρήστες και να προσφέρει συμβουλές. Το κοινωνικό δίκτυο αυτό, είναι σχεδιασμένο και υλοποιημένο έτσι ώστε να είναι όσο το δυνατόν πιο εύχρηστο για τον κάθε άνθρωπο που θα το χρησιμοποιήσει. Ο εκάστοτε χρήστης θα μπορεί να έχει τον δικό του λογαριασμό στη διαδικτυακή πλατφόρμα και θα έχει την δυνατότητα να λαμβάνει τις πληροφορίες που χρειάζεται για κάθε προορισμό, καθώς και να μοιράζεται τις εμπειρίες του δικού του ταξιδιού με πολυμεσικό υλικό. Η ανάπτυξη και η κατασκευή του κοινωνικού δικτύου πραγματοποιήθηκε με την MERN Stack. Συγκεκριμένα για το backend του κοινωνικού δικτύου χρησιμοποιήθηκε Node.js, για τη βάση δεδομένων MongoDB και για το frontend React.js.

ABSTRACT

The present thesis describes the development of a social network that reviews travels. The aim of this social platform, is to inform people who wish to travel to many destinations, giving them the opportunity to share their experiences, to entertain them and to offer them useful advice. This social network is designed and implemented to be user-friendly for everyone who uses the platform. Each user can login with his Google account or sign up in the application with his personal password in order to log in to the platform and collect all the information he needs, whereas he can share his own travel experience through multimedia. The development of this social network was implemented using the MERN Stack. Specifically, the backend of this platform was implemented with Node.js, the database with MongoDB and the frontend with React.js.

Περιεχόμενα

1 Εισαγωγή.....	9
1.1 Ορισμός κοινωνικών δικτύων.....	9
1.1.1 Ο σκοπός των κοινωνικών δικτύων.....	11
1.1.2 Ανάλυση κοινωνικών δικτύων.....	11
1.1.3 Χαρακτηριστικά κοινωνικών δικτύων.....	11
1.2 Ιστορική αναδρομή στα κοινωνικά δίκτυα.....	12
1.3 Κοινωνική δικτύωση.....	12
1.4 Αναπαράσταση και μοντελοποίηση των κοινωνικών δικτύων.....	13
2 Θεωρητικό και Τεχνολογικό Υπόβαθρο.....	15
2.1 Θεωρητικό Υπόβαθρο.....	15
2.2 Τεχνολογικό Υπόβαθρο.....	15
2.2.1 MERN Stack.....	16
2.2.2 MERN Stack - FRONTEND.....	17
2.2.2.1 ReactJS.....	17
2.2.2.2 Material-UI Library.....	17
2.2.3 MERN Stack - BACKEND.....	19
2.2.3.1 NodeJS.....	19
2.2.3.2 ExpressJS.....	19
2.2.3.3 MongoDB.....	20
2.2.3.4 Axios.....	20
2.2.3.5 Mongoose.....	20
2.3 Ανασκόπηση παρόμοιων εφαρμογών.....	21
3 Σχεδίαση και Υλοποίηση εφαρμογής Travel Light.....	30
3.1 Ανάπτυξη εφαρμογής Travel Light.....	30
3.1.1 Αρχική σελίδα εφαρμογής.....	30
3.1.2 Σελίδα σύνδεσης και εγγραφής χρήστη.....	32
3.1.3 Σελίδα δημοσιεύσεων.....	33
3.1.4 Σελίδα δημιουργίας δημοσίευσης(Add a Post).....	34
3.1.5 Σελίδα δημοσίευσης(Post).....	35
3.1.6 Σελίδα προφίλ χρήστη.....	36
3.2 Περιπτώσεις χρήστη.....	37
3.2.1 Πρώτο Σενάριο.....	37
3.2.1.1 Σύνδεση στην εφαρμογή.....	38
3.2.1.2 Περιήγηση στην αρχική σελίδα.....	41
3.2.1.3 Αναζήτηση τίτλων και ετικετών.....	47
3.2.1.4 Δημιουργία δημοσίευσης.....	50
3.2.1.5 Προφίλ χρήστη.....	51
3.2.2 Δεύτερο Σενάριο.....	55
3.2.2.1 Περιήγηση στην αρχική σελίδα.....	56
3.2.2.2 Αναζήτηση τίτλων και ετικετών.....	57
3.2.2.3 Δημιουργία δημοσίευσης.....	59
3.2.2.4 Προφίλ χρήστη.....	60

3.3 Υλοποίηση Εφαρμογής Travel Light.....	61
3.3.1 Βάση Δεδομένων.....	61
3.3.2 Backend Εφαρμογής.....	63
3.3.3 Frontend Εφαρμογής.....	75
4 Συμπεράσματα και Μελλοντικές βελτιώσεις.....	109
4.1 Συμπεράσματα.....	109
4.2 Μελλοντικές βελτιώσεις.....	110
Βιβλιογραφία.....	111

Κατάλογος εικόνων

Εικόνα 1.1 Αναπαράσταση ενός κοινωνικού δικτύου.....	10
Εικόνα 1.2 Πλήρης Γράφος.....	14
Εικόνα 2.1 Διάγραμμα αναπαράστασης λειτουργίας MERN Stack.....	16
Εικόνα 2.2 Διάγραμμα αναπαράστασης components της React.....	17
Εικόνα 2.3 Αρχική σελίδα Google Maps.....	22
Εικόνα 2.4 Οδηγίες μετάβασης σε ένα προορισμό με διάφορα μέσα.....	23
Εικόνα 2.5 Αξιολογήσεις χρηστών για ένα συγκεκριμένο μέρος.....	24
Εικόνα 2.6 Αρχική σελίδα Trip Advisor.....	25
Εικόνα 2.7 Εξερεύνηση προορισμού στο Trip Advisor.....	26
Εικόνα 2.8 Αρχική σελίδα της εφαρμογής Travello.....	27
Εικόνα 2.9 Μενού της εφαρμογής Travello.....	28
Εικόνα 2.10 Εξερεύνηση δραστηριοτήτων που προσφέρονται μέσω της εφαρμογής για συγκεκριμένο μέρος και ημερομηνία.....	29
Εικόνα 3.1 Αρχική σελίδα εφαρμογής.....	31
Εικόνα 3.2 Σελίδα σύνδεσης λογαριασμού.....	33
Εικόνα 3.3 Σελίδα δημοσιεύσεων.....	34
Εικόνα 3.4 Σελίδα δημιουργίας δημοσίευσης.....	35
Εικόνα 3.5 Σελίδα ενδεικτικής δημοσίευσης.....	36
Εικόνα 3.6 Προφίλ χρήστη.....	37
Εικόνα 3.7 Σύνδεση με Google Login.....	38
Εικόνα 3.8 Εγγραφή στην εφαρμογή.....	39
Εικόνα 3.9 Σύνδεση στην εφαρμογή με τα στοιχεία της εγγραφής.....	40
Εικόνα 3.10 Αρχική σελίδα δημοσιεύσεων συνδεδεμένου χρήστη.....	41
Εικόνα 3.11 Σελιδοποίηση δημοσιεύσεων.....	42
Εικόνα 3.12 Σελίδα δημοσίευσης συνδεδεμένου χρήστη.....	43
Εικόνα 3.13 Προτάσεις δημοσιεύσεων εφαρμογής για τον χρήστη.....	44
Εικόνα 3.14 Προφίλ άλλου χρήστη.....	45
Εικόνα 3.15 Λειτουργικότητα Like δημοσίευσης.....	46
Εικόνα 3.16 Αναζήτηση τίτλων συνδεδεμένου χρήστη.....	47
Εικόνα 3.17 Αναζήτηση ετικετών συνδεδεμένου χρήστη.....	48
Εικόνα 3.18 Αναζήτηση ετικετών συνδεδεμένου χρήστη χωρίς αποτελέσματα...49	
Εικόνα 3.19 Δημιουργία δημοσίευσης.....	50
Εικόνα 3.20 Δημιουργία δημοσίευσης.....	51
Εικόνα 3.21 Προφίλ συνδεδεμένου χρήστη.....	52
Εικόνα 3.22 Διαγραφή δημοσίευσης.....	53
Εικόνα 3.23 Τροποποίηση δημοσίευσης.....	54
Εικόνα 3.24 Διορθωμένη φόρμα.....	55
Εικόνα 3.25 Αρχική σελίδα επισκέπτη.....	56
Εικόνα 3.26 Αναζήτηση τίτλων επισκέπτη.....	57
Εικόνα 3.27 Αναζήτηση ετικετών επισκέπτη.....	58
Εικόνα 3.28 Σελίδα δημιουργία δημοσίευσης επισκέπτη.....	59
Εικόνα 3.29 Προφίλ επισκέπτη.....	60
Εικόνα 3.30 MongoDB ER Diagram.....	61
Εικόνα 3.31 Connection URL.....	61

Εικόνα 3.32 PostMessage Schema.....	62
Εικόνα 3.33 User Schema.....	62
Εικόνα 3.34 PostMessage Data.....	63
Εικόνα 3.35 User Data.....	63
Εικόνα 3.36 Backend depedencies.....	64
Εικόνα 3.37 Δομή φακέλων του backend.....	65
Εικόνα 3.38 Αρχείο middleware/auth.js.....	66
Εικόνα 3.39 Αρχείο routes/posts.js.....	67
Εικόνα 3.40 Αρχείο routes/user.js.....	68
Εικόνα 3.41 Αρχείο controllers/posts.js.....	69
Εικόνα 3.42 Αρχείο controllers/user.js.....	73
Εικόνα 3.43 Αρχείο server/index.js.....	74
Εικόνα 3.44 Frontend Depedencies.....	75
Εικόνα 3.45 Δομή φακέλων frontend.....	77
Εικόνα 3.46 Αρχείο src/index.js.....	80
Εικόνα 3.47 Αρχείο src/index.css.....	81
Εικόνα 3.48 Αρχείο src/App.js.....	82
Εικόνα 3.49 Αρχείο actions/posts.js.....	83
Εικόνα 3.50 Αρχείο actions/auth.js.....	86
Εικόνα 3.51 Αρχείο api/index.js.....	87
Εικόνα 3.52 Αρχείο constants/actionTypes.js.....	88
Εικόνα 3.53 Αρχείο reducers/index.js.....	89
Εικόνα 3.54 Αρχείο reducers/auth.js.....	89
Εικόνα 3.55 Αρχείο reducers/posts.js.....	90
Εικόνα 3.56 Διάγραμμα αναπαράστασης components της εφαρμογής.....	91
Εικόνα 3.57 Αρχείο components/auth/auth.js.....	92
Εικόνα 3.58 Αρχείο components/createpost/createpost.js.....	94
Εικόνα 3.59 Αρχείο components/footer/footer.js.....	95
Εικόνα 3.60 Αρχείο components/form/form.js.....	96
Εικόνα 3.61 Αρχείο components/home/home.js.....	98
Εικόνα 3.62 Αρχείο components/mainpage/mainpage.js.....	100
Εικόνα 3.63 Αρχείο components/myposts/myposts.jsx.....	101
Εικόνα 3.64 Αρχείο components/navbar/navbar.js.....	103
Εικόνα 3.65 Αρχείο components/postcreator/postcreator.js.....	104
Εικόνα 3.66 Αρχείο components/postdetails/postdetails.js.....	105
Εικόνα 3.67 Αρχείο components/posts/posts.js.....	106
Εικόνα 3.68 Αρχείο components/post/post.js.....	107
Εικόνα 3.69 Αρχείο components/pagination.jsx.....	108

ΚΕΦΑΛΑΙΟ 1

Εισαγωγή

Η τεχνολογία εξελίσσεται με γρήγορους ρυθμούς, γεγονός που καθιστά τα δίκτυα απαραίτητο και σημαντικό κομμάτι της καθημερινότητας. Ο όρος δίκτυο μπορεί να αναφέρεται σε οποιαδήποτε αλληλοσυνδεδεμένη ομάδα ή σύστημα. Το δημοφιλέστερο δίκτυο, τα τελευταία χρόνια, είναι το Διαδίκτυο που έχει αναπτυχθεί σε παγκόσμια κλίμακα. Στο Διαδίκτυο υπάρχουν πλατφόρμες που δημιουργούνται κοινωνικές σχέσεις μεταξύ χρηστών, οι οποίοι είναι ενεργά μέλη με κοινές πεποιθήσεις ή δραστηριότητες ενός κοινωνικού δικτύου. Τα κοινωνικά δίκτυα (social networks) έχουν προσελκύσει το ενδιαφέρον αλλά και την περιέργεια της επιστημονικής κοινότητας της τελευταίας δεκαετίας. Η ελκυστικότητα αυτή ανακύπτει τόσο από συσχετίσεις και τη συνοχή των οντοτήτων που μετέχουν σε αυτά όσα και από τη διάρθρωση και τη γενικότερη δομή που δημιουργούν στο σύνολό τους[3].

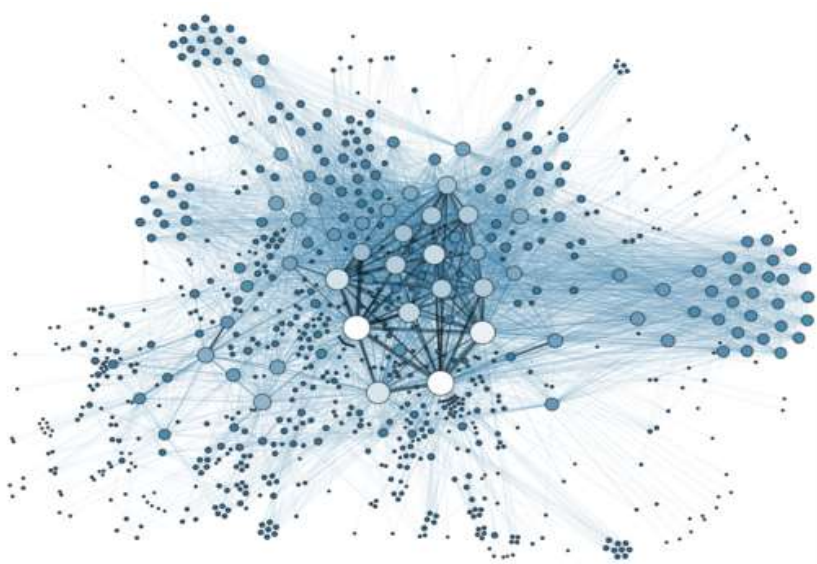
1.1 Ορισμός κοινωνικών δικτύων

Οι ομάδες ανθρώπων που δημιουργούνται και υφίστανται στον πραγματικό κόσμο, πραγματεύονται οργανωμένες δομές που σχηματίζουν ένα δίκτυο. Τα δίκτυα αυτά, τα τελευταία χρόνια έχουν μετοικήσει στο διαδίκτυο όπου σχηματίζουν κοινότητες και μέσω αυτών οι χρήστες μπορούν να συνδέονται μεταξύ τους, πλαισιώνοντας έτσι ένα κοινωνικό δίκτυο. Οι Walker, MacBride & Vachon (1977), όρισαν ως «κοινωνικό δίκτυο» το άθροισμα των προσωπικών επαφών μέσω των οποίων το άτομο διατηρεί την κοινωνική του ταυτότητα, λαμβάνει συναισθηματική υποστήριξη, υλική ενίσχυση και συμμετοχή στις υπηρεσίες, έχει πρόσβαση στις πληροφορίες και δημιουργεί νέες κοινωνικές επαφές. Τα κοινωνικά δίκτυα συνήθως αποτελούνται από μέλη της οικογένειας, τους φίλους και τους «σημαντικούς» άλλους της βιογραφίας του ατόμου (π.χ. εκπαιδευτικοί). Τα κοινωνικά δίκτυα αναφέρονται ουσιαστικά στις κοινωνικές σχέσεις του ατόμου, στον τρόπο με τον οποίο αυτά αντιλαμβάνονται και αξιολογούν τις εν λόγω σχέσεις[2].

Γενικότερα, ως κοινωνικό δίκτυο (social network) ορίζεται κάθε δίκτυο σχέσεων και αλληλεπιδράσεων. Οι κόμβοι απαρτίζονται από δρώντες (actors) ή μέλη και οι απαρτίζονται από τις σχέσεις ή τις αλληλεπιδράσεις μεταξύ των μελών[1]. Εναλλακτική σημασία των κοινωνικών δικτύων δίνεται μέσα από τον όρο «ιστοσελίδες κοινωνικής δικτύωσης», όπου ο όρος «δικτύωση» σημαίνει, κυρίως την έναρξη σχέσεων μεταξύ αγνώστων ή γνωστών ατόμων. Αφενός, η έναρξη σχέσεων με άγνωστα άτομα δεν θεωρείται αδύνατη, αφετέρου, δεν είναι η πρωτεύουσα τακτική για την πλειοψηφία των χρηστών[1]. Τα κοινωνικά δίκτυα είναι σχεδιασμένα έτσι, ώστε να παρέχουν εύκολη πρόσβαση σε όλους, ανεξαρτήτου ηλικίας, φύλου, εθνικότητας, και μορφωτικού επιπέδου. Επιπλέον, είναι διαθέσιμα ανά πάσα στιγμή

και από διαφορετικά μέσα (laptop, netbook, smartphone, iPad κτλ.)[5].

Εικόνα 1.1 Αναπαράσταση ενός κοινωνικού δικτύου



Πηγή: [9]

Η αντικειμενική θεώρηση των κοινωνικών δικτύων περιλαμβάνει θεωρίες, μοντέλα και εφαρμογές που αποκαλύπτουν τις συσχετίσεις και τις διεργασίες μεταξύ των συστατικών του (οντότητες, χρήστες). Οι σχέσεις μεταξύ των χρηστών καθορίζονται από τις μεταξύ τους συνδέσεις (links), συστατικά απαραίτητα για την μελέτη και απεικόνιση της δικτυακής δομής αυτών των δικτύων. Η γνώση αυτή αποκομίζεται από τις αλληλεπιδράσεις μεταξύ των χρηστών και συμβάλει στην κατανόηση του μηχανισμού επιλογής και εκτέλεσης των διεργασιών[3]. Παρά την ποικιλομορφία τους, τα κοινωνικά δίκτυα έχουν κάποια βασικά δομικά στοιχεία:

Προφίλ (Profile): Ο κάθε χρήστης έχει στη διάθεση του ένα προφίλ το οποίο τον κάνει μέλος του δικτύου. Το προφίλ περιέχει ατομικά και προσωπικά ενδιαφέροντα, ικανότητες και ασχολίες του κάθε ατόμου. Το κάθε προφίλ αποτελεί έναν κόμβο και το σύνολο των κόμβων αποτελεί το δίκτυο συνολικά.

Συνδέσεις (Connections or links): Οι συνδέσεις αντιπροσωπεύουν τις ακμές, και είναι οι πράξεις και οι ενέργειες που συνδέουν χρήστες μεταξύ τους, και ορίζονται από τους ίδιους.

Υπάρχουν τρεις μαθηματικοί τρόποι για να περιγραφούν τα κοινωνικά δίκτυα:

Θεωρία γραφικών παραστάσεων

Αλγεβρική Ανάλυση

Κοινωνιογράμματα

Ο κάθε τρόπος μπορεί να προσαρμοστεί έτσι ώστε να αντιπροσωπεύει διαφορετικά στοιχεία δικτύων, μολονότι για συγκεκριμένες μορφές δεδομένων και για κάποιους τύπους δικτύων προτιμάται συγκεκριμένος τρόπος βασιζόμενος στην σαφήνεια και την αποδοτικότητα του.

1.1.1 Ο σκοπός των κοινωνικών δικτύων

Τα κοινωνικά δίκτυα εξυπηρετούν αρκετούς σκοπούς αλλά είναι συγκεκριμένοι που χρησιμοποιούνται ως κοινοί στις περισσότερες ιστοσελίδες. Ο πρώτος σκοπός είναι η διατήρηση και η ενδυνάμωση των ήδη υπάρχοντων κοινωνικών δεσμών ή η δημιουργία νέων κοινωνικών δεσμών. Ο δεύτερος σκοπός είναι η δυνατότητα κοινοποίησης προσωπικού περιεχομένου κάθε χρήστη. Τρίτος και τελευταίος σκοπός είναι η εύρεση νέων περιεχομένων μέσω της ανακάλυψης της σύστασης και της οργάνωσης περιεχομένων που έχουν ήδη κοινοποιηθεί από τους χρήστες[6].

1.1.2 Ανάλυση κοινωνικών δικτύων

Η ανάλυση κοινωνικών δικτύων (SNA – social network analysis) χρησιμοποιείται σε πολλά σύγχρονα πεδία των ερευνητών όπως στην ψυχολογία, στην κοινωνιολογία, στην ανθρωπολογία και στην οικονομία. Οι ερευνητικές ομάδες χρησιμοποιούν την ανάλυση κοινωνικών δικτύων για να κατανοήσουν τον τρόπο που χρησιμοποιούνται μεταξύ των ανθρώπων για επικοινωνία και συνεργασία καθώς και για την αναγνώριση της ροής της γνώσης για κάθε οργανισμό. Στο σύγχρονο οικονομικό περιβάλλον με τα κοινωνικά δίκτυα εξετάζονται οι αλληλεπιδράσεις μεταξύ των εταιριών καθώς και μεταξύ των υπαλλήλων τους[6].

1.1.3 Χαρακτηριστικά κοινωνικών δικτύων

Τα κύρια χαρακτηριστικά των κοινωνικών δικτύων περιλαμβάνουν τις σχέσεις που υπάρχουν μέσα σε αυτά, καθώς και το σχήμα τους και μηχανισμούς τους[8].

Οι φορείς των κοινωνικών δικτύων βασίζονται στις τεχνικο-οργανωτικές διασυνδέσεις, στην οργάνωση προσωπικού και θεωρείται ότι απορρέουν από την αυτονομία τους, αμοιβαιότητα και σταθερότητα. Οι σχέσεις μεταξύ φορέων του κοινωνικού δικτύου που αναπτύσσονται, κατηγοριοποιούνται σε περιεχόμενα (προϊόντα ή υπηρεσίες), σε είδος (διάρκεια και εγγύτητα της σχέσης) και την ένταση (συχνότητα επικοινωνίας). Τα δύο τελευταία εδραιώνουν τη δικτυακή δομή των κοινωνικών δικτύων[8].

Η χρησιμότητα ενός κοινωνικού δικτύου καθορίζεται από το σχήμα του. Το κάθε κοινωνικό δίκτυο έχει πολλές συνδέσεις με άτομα εκτός του κύριου δικτύου αφού στα μικρότερα δίκτυα τα μέλη έχουν πρόσβαση σε μία ευρεία γκάμα πληροφοριών και ως αποτέλεσμα σε περισσότερες ιδέες και ευκαιρίες[8].

Τα κοινωνικά δίκτυα πετυχαίνουν αποτελέσματα είτε με την εσωτερίκευση είτε με την εξωτερίκευση. Μέσω της εσωτερίκευσης ανατίθεται περιορισμένη λειτουργική

ανάθεση σε τρίτους, που πραγματοποιείται με συντονιστικούς μηχανισμούς. Η εσωτερίκευση και η εξωτερίκευση μπορούν να χρησιμοποιηθούν ταυτόχρονα[8].

1.2 Ιστορική αναδρομή στα κοινωνικά δίκτυα

Η πρώτη ιστοσελίδα που δημιουργήθηκε το 1995 είχε την ονομασία Classmates.com και ο σκοπός της ήταν η επανένωση συμμαθητών από τα σχολεία. Η ιστοσελίδα έφτασε τους σαράντα εκατομμύρια εγγεγραμμένους χρήστες ωστόσο, δεν επέτρεπε στους χρήστες να συνδέονται γενικά μεταξύ τους αλλά μόνο με τα σχολεία που είχαν παρακολουθήσει.

Το πρώτο αναγνωρίσιμο κοινωνικό δίκτυο κυκλοφόρησε το 1997 και ονομάστηκε SixDegrees.com. Αυτό το κοινωνικό δίκτυο επέτρεπε στους χρήστες να δημιουργούν προφίλ και να συνδέονται διαδικτυακά με άλλους χρήστες δημιουργώντας μια λίστα διαδικτυακών φίλων, όπου αργότερα, το 1998, προστέθηκε η δυνατότητα επισκόπησης της λίστας αυτής. Η κάθε δυνατότητα που προσφέρει το SixDegrees είχε δημιουργηθεί με κάποια μορφή και στο παρελθόν. Τα προφίλ χρηστών υπήρχαν στα περισσότερα dating sites και σε ιστοσελίδες διαφόρων κοινοτήτων.

Από το 1997 έως το 2001 δημιουργήθηκαν αρκετά μικρά κοινωνικά δίκτυα αλλά χωρίς προοπτικές ανάπτυξης. Τα επόμενα χρόνια έκαναν την εμφάνιση τους κοινωνικά δίκτυα που αξιοποιήθηκαν για τα δίκτυα των επιχειρήσεων όπως για παράδειγμα το Ryze.com, LinkedIn και το Tribe.net. Τα μεγαλύτερα φαινόμενα κοινωνικών δικτύων που εδραιώθηκαν μετέπειτα, ήταν το Friendster, το MySpace και το Facebook. Αυτοί οι τρεις ιστότοποι κοινωνικής δικτύωσης μορφοποίησαν τις επιχειρήσεις, την κουλτούρα και το ερευνητικό ενδιαφέρον σε όλα τα επίπεδα.

Με την άνοδο δημοτικότητας των κοινωνικών δικτύων πολλές ιστοσελίδες ενσωμάτωναν χαρακτηριστικά κοινωνικής δικτύωσης. Διακριτά παραδείγματα ιστοσελίδων δημοσίευσης πολυμεσικού περιεχομένου όπως το Flickr, το YouTube και το Zoom, ιστοσελίδες με ιστολόγια όπως το Live-Journal και το BlogSpot, ιστοσελίδες συλλογής ειδήσεων όπως το Digg, το Reddit καθώς και ιστοσελίδες επαγγελματικής δικτύωσης όπως το LinkedIn και το Ryze[7]. Τα παραπάνω παραδείγματα αναφέρονται στις πιο γνωστές ιστοσελίδες που χρησιμοποιούνται από τους περισσότερους χρήστες, ωστόσο εξακολουθεί να υπάρχει μια μεγάλη λίστα με ιστοσελίδες που χρησιμοποίησαν την ίδια στρατηγική εκμετάλλευσης των κοινωνικών δικτύων προκειμένου να βελτιωθούν.

1.3 Κοινωνική δικτύωση

Με τον όρο κοινωνική δικτύωση εννοούνται τα μέσα αλληλεπίδρασης και επικοινωνίας ομάδων ανθρώπων μέσω διαδικτυακών κοινοτήτων. Υπάρχουν έξι διαφορετικοί τύποι μέσων κοινωνικής δικτύωσης : τα ιστολόγια, τα συνεργατικά έργα, οι ιστοσελίδες κοινωνικής δικτύωσης (social networking sites), οι κοινότητες

περιεχομένου, οι εικονικοί κόσμοι και τα forums[10].

Τα ιστολόγια είναι η παλαιότερη μορφή των μέσων κοινωνικής δικτύωσης και έχουν τη μορφή ιστοτόπου. Είναι μια λίστα καταχωρήσεων που ταξινομούνται με την ημερομηνία καταχώρησης. Αυτές οι καταχωρήσεις μπορούν να διαφέρουν σε περιεχόμενο και θεματολογία όπως για παράδειγμα ειδήσεις, πολιτικοκοινωνικός σχολιασμός, αθλητικός σχεδιασμός, τεχνολογία, μόδα.

Τα συνεργατικά έργα αποτελούνται από ιστοσελίδες τις οποίες οι χρήστες του διαδικτύου συνεργατικά, μπορούν να επεξεργαστούν και να τροποποιήσουν το περιεχόμενο για μια συγκεκριμένη θεματολογία. Ένα παράδειγμα είναι η ηλεκτρονική εγκυκλοπαίδεια Wikipedia.

Εκατοντάδες εκατομμύρια χρήστες σε όλο το κόσμο, χρησιμοποιούν τις λεγόμενες ιστοσελίδες κοινωνικής δικτύωσης (social networking sites-sns), τα οποία επιτρέπουν σε όλους τους χρήστες να επικοινωνούν μεταξύ τους, να δημιουργούν και να ανταλλάζουν υλικό.

Οι κοινότητες περιεχομένου είναι οργανωμένοι ιστότοποι οι οποίοι περιέχουν συγκεκριμένο περιεχόμενο με αρχεία ήχου, φωτογραφίες ή και βίντεο, όπου οι χρήστες μπορούν να αλληλοεπιδράσουν με αυτά και να τα μοιραστούν με άλλους χρήστες. Ένας ενεργός χρήστης δεν είναι υποχρεωμένος να δημιουργήσει το δικό του προφίλ στις κοινότητες περιεχομένου. Τέτοια παραδείγματα είναι το YouTube, το Flickr και το Pinterest.

Οι εικονικοί κόσμοι (virtual worlds) είναι πλατφόρμες όπου οι χρήστες συνδέονται και αλληλοεπιδρούν σε ένα τρισδιάστατο περιβάλλον. Εικονικοί κόσμοι υπάρχουν κυρίως σε παιχνίδια εικονικής πραγματικότητας.

Τα Forums είναι ιστότοποι ανταλλαγής απόψεων, ιδεών και πληροφοριών μεταξύ χρηστών για ποικίλα θέματα.

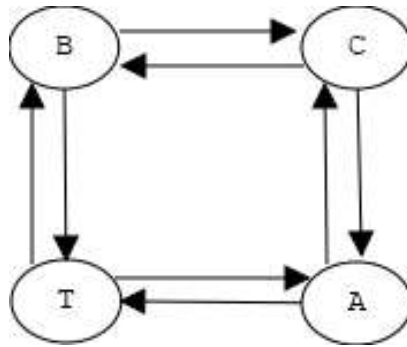
1.4 Αναπαράσταση και μοντελοποίηση των κοινωνικών δικτύων

Ένα κοινωνικό δίκτυο είναι μία κοινωνική δομή που αποτελείται από ένα πεπερασμένο σύνολο οντοτήτων και ακμών που συνδέονται μεταξύ τους. Η απεικόνιση των κοινωνικών δικτύων γίνεται με γράφους και πίνακες. Στους γράφους, οι κόμβοι απεικονίζουν τους ανθρώπους ή τις ομάδες ενώ οι ακμές δείχνουν τις σχέσεις ή τις ροές μεταξύ των κόμβων. Οι γράφοι χωρίζονται σε δυαδικούς προσημασμένους και πλειότιμους ενώ οι σχέσεις χωρίζονται σε κατευθυνόμενες, μη κατευθυνόμενες, μονοδιάστατες και πολυδιάστατες.

Οι πίνακες χρησιμοποιούνται για την αναπαράσταση των γράφων. Ένας πίνακας αποτελείται από γραμμές και στήλες τόσες όσες οι κόμβοι και τα στοιχεία του γράφου. Ένας πλήρης γράφος όπως αυτός απεικονίζεται στην εικόνα 1.2 σχηματίζεται όταν όλοι οι κόμβοι συνδέονται με όλους τους άλλους. Οι συνδέσεις

μεταξύ των κόμβων, όταν υπάρχουν, μπορεί να είναι είτε τόξα είτε γραμμές, αναλόγως των συνδέσεων μεταξύ των κόμβων του γράφου. Ένας γράφος $G = (V, E)$ ορίζεται ως ένα ζεύγος δύο συνόλων V και E όπου $V \neq \emptyset$ και E ένα σύνολο μη διατεταγμένων ζευγών των στοιχείων του V . Τα στοιχεία του $V \equiv \{V_1, V_2, \dots, V_n\}$ είναι οι κόμβοι (κορυφές) του γραφήματος G ενώ τα στοιχεία του $E \equiv \{E_1, E_2, \dots, E_k\}$ είναι οι συνδέσεις τους (ακμές)[11].

Εικόνα 1.2 Πλήρης Γράφος



Πηγή: [11]

ΚΕΦΑΛΑΙΟ 2

Θεωρητικό και Τεχνολογικό Υπόβαθρο

Στο παρόν κεφάλαιο θα αναλυθεί το θεωρητικό και τεχνολογικό υπόβαθρο που χρησιμοποιήθηκε στην ανάπτυξη της εφαρμογής.

2.1 Θεωρητικό Υπόβαθρο

Με την εμφάνιση των νέων τεχνολογιών πραγματοποιήθηκαν ριζικές αλλαγές στον κλάδο του τουρισμού, επαναπροσδιορίζοντας τους ρόλους της τουριστικής βιομηχανίας σε αυτόν.

Ο βασικός λόγος είναι η αμεσότητα, που προσφέρει το Διαδίκτυο και κατ' επέκταση τα κοινωνικά δίκτυα, στην επικοινωνία μεταξύ των χρηστών του. Η οικονομική ανταγωνιστικότητα των επιχειρήσεων από τις αυξανόμενες παρεχόμενες υπηρεσίες, ο καταγισμός των πληροφοριών από διάφορα μέσα, καθώς και η έλλειψη της σωστής οργάνωσης από μια πολυάσχολη καθημερινότητα είναι μερικοί από τους λόγους που ο κάθε χρήστης του Διαδικτύου το αξιοποιεί για την οργάνωση του ταξιδιού του.

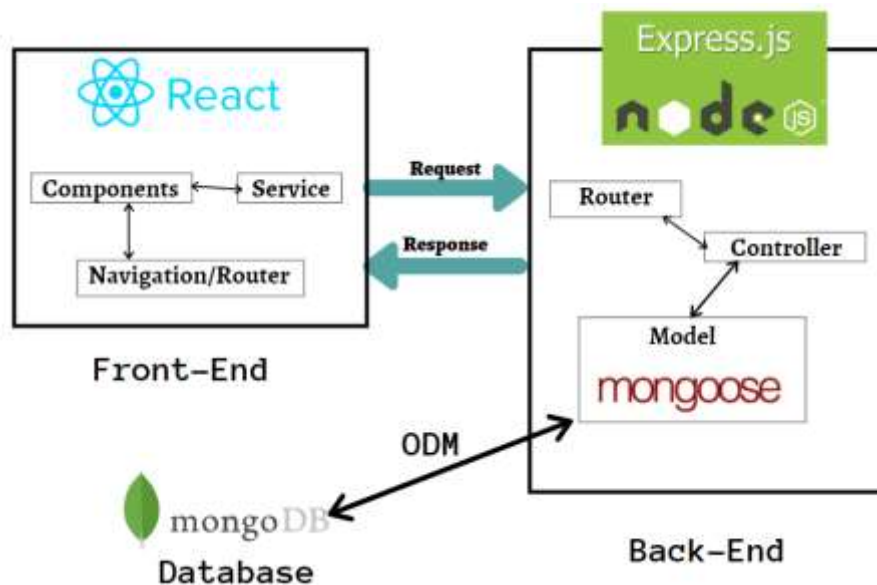
Τα τελευταία χρόνια ποικίλες εφαρμογές κοινωνικής δικτύωσης που έχουν βοηθήσει εκατομμύρια χρήστες να χτίσουν σχέσεις εμπιστοσύνης διευκολύνοντας την επικοινωνία μεταξύ τους, είτε με επιχειρήσεις οι οποίες προωθούν τις δικές τους προσφορές και υπηρεσίες, κυρίως με την χρήση του μάρκετινγκ, είτε με απλούς χρήστες που μοιράζουν συμβουλές και εμπειρίες. Αυτό βοηθά τους χρήστες να δημιουργήσουν μικρές κοινότητες που έχουν κοινά ενδιαφέροντα. Το φαινόμενο αυτό, έχει οδηγήσει τους χρήστες να αποφασίζουν τον επόμενο προορισμό τους μέσω των κοινωνικών δικτύων, αλλάζοντας εντελώς τον τρόπο λήψης αποφάσεων. Η συγκεκριμένη αλλαγή έχει επιφέρει ειδικά αποτελέσματα στη ζωή των χρηστών, αφού οι δυνατότητες για τον σχεδιασμό του ταξιδιού τους πολλαπλασιάζονται, καθώς έχουν αρκετές γνώσεις για τον κάθε προορισμό και κατ' επέκταση για το ταξίδι τους.

2.2 Τεχνολογικό Υπόβαθρο

Κατά την ανάπτυξη της εφαρμογής χρησιμοποιήθηκε η MERN Stack, που αποτελείται από συνδυασμό τεχνολογιών, καθώς και κάποιες επιπλέον τεχνολογίες που θα αναλυθούν παρακάτω.

2.2.1 MERN Stack

Η MERN Stack χρησιμοποιείται και πιο συχνά για την υλοποίηση εφαρμογών διαδικτύου καθώς είναι ένα ολοκληρωμένο σύστημα διαδικτύου (web system). Πρόκειται για μια στοίβα που συνδυάζει τεχνολογίες για την δημιουργία μιας ολοκληρωμένης εφαρμογής διαδικτύου[28][34]. Αυτή η στοίβα αποτελείται από την Βάση Δεδομένων MongoDB, τα framework ExpressJS, ReactJS και την πλατφόρμα λογισμικού NodeJS.



Εικόνα 2.1 Διάγραμμα αναπαράστασης λειτουργίας MERN Stack

Πηγή: [36]

Πλεονεκτήματα MERN Stack

Τα βασικά πλεονεκτήματα[26] της MERN Stack είναι:

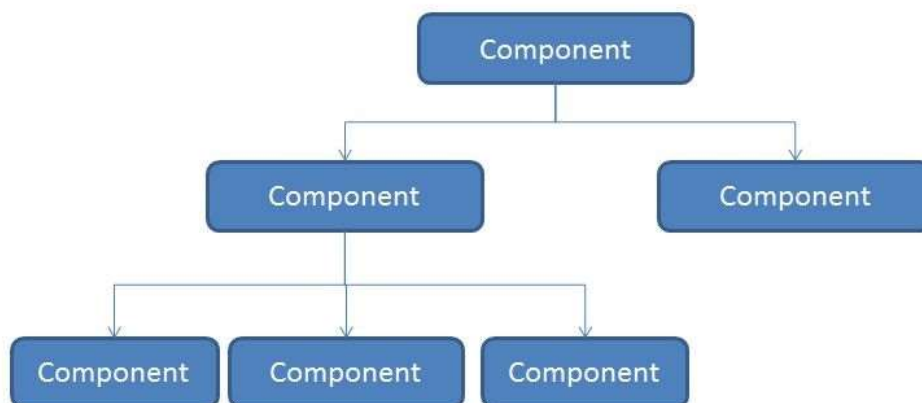
- Το πιο σημαντικό πλεονέκτημα αφορά την χρήση μόνο βιβλιοθηκών της γλώσσας Javascript, από το frontend έως το backend.
- Η εφαρμογή που αναπτύσσεται με τη MERN Stack επωφελείται τα πλεονεκτήματα του SPA (Single Page Application) με το κυριότερο να είναι η ταχύτητα φόρτωσης περιεχομένου.
- Αξιοποιεί τα REST APIs.

2.2.2 MERN - FRONTEND

2.2.2.1 ReactJS

Η ReactJS είναι μια βιβλιοθήκη της γλώσσας Javascript ανοιχτού κώδικα και είναι σχεδιασμένη για τη δημιουργία του front-end της εφαρμογής. Χρησιμοποιεί αντικείμενα (objects) και είναι φτιαγμένη με Javascript scripts. Ο σκοπός της ReactJS είναι να παρέχει μια εύκολη ανάπτυξη του UI (User Interface) και μπορεί να χρησιμοποιηθεί ως βάση στην ανάπτυξη εφαρμογών μιας σελίδας[13][27]. Το κύριο χαρακτηριστικό της ReactJS είναι ότι αποτελείται από components τα οποία συνδυασμένα αποτελούν το front-end της εφαρμογής[24][25]. Παρακάτω ακολουθεί διάγραμμα αναπαράστασης των components που χρησιμοποιούνται για την υλοποίηση μιας εφαρμογής, το οποίο είναι και ο σκελετός του UI.

Εικόνα 2.2 Διάγραμμα αναπαράστασης components της React



Πηγή: [21]

2.2.2.2 Material-UI Library

Για να ενισχυθεί το UX (User Experience) των χρηστών και να υπάρχει μια ομοιομορφία στο design αξιοποιήθηκε η Material-UI βιβλιοθήκη. Αυτή η βιβλιοθήκη παρέχει έτοιμα προς χρήση components τα οποία μπορούν να δεχθούν κλάσεις για να προστεθεί CSS. Τα material-UI components είναι βασισμένα στυλιστικό στο Google material design.

Σε καθένα από τα components της Material-UI μπορεί να διοριστούν κάποια properties τα οποία θα ορίζουν το στυλιστικό αποτέλεσμα αυτών. Έπειτα, θα δοθεί ένα παράδειγμα από properties που μπορούν να χρησιμοποιηθούν. Τα παρακάτω properties ορίζουν το width των components:

xs >= 0px

sm >= 600px

md >= 960px

lg >= 1280px

xl >= 1920px

Το κάθε property παίρνει τιμές από το 1 έως το 12 που ορίζουν πόσες φορές θα χωράει το component στην οθόνη[14].

Στη συνέχεια θα αναλυθεί η χρήση των components της βιβλιοθήκης material-UI που χρησιμοποιήθηκαν στην ανάπτυξη της εφαρμογής[15].

Button

Το component Button επιτρέπει στους χρήστες να πραγματοποιούν διάφορα actions μέσα στην εφαρμογή.

TextField

Το component TextField επιτρέπει στους χρήστες να εισάγουν και να τροποποιούν κείμενο.

Avatar

Το component Avatar χρησιμοποιείται για να αναπαραστήσει δεδομένα.

ChipInput

Το component ChipInput χρησιμοποιείται για την εισαγωγή κειμένου και για την αναπαράσταση του με διαφορετικό τρόπο. Το ChipInput οι χρήστες μπορούν να κατηγοριοποιήσουν δεδομένα, να κάνουν επιλογές και αναζητήσεις βάσει του συγκεκριμένου component.

Divider

Το component Divider είναι μια οριζόντια γραμμή που διαχωρίζει ή ομαδοποιεί το περιεχόμενο.

Typography

Το component Typography χρησιμοποιείται για να αναπαραστήσει το design και το περιεχόμενο όπως για παράδειγμα HTML Tags.

AppBar

Το component AppBar αναπαριστά σημαντικές πληροφορίες που αφορούν την οθόνη που βρίσκεται την εκάστοτε στιγμή ο κάθε χρήστης.

Paper

Το component Paper χρησιμοποιείται για το design της σελίδας και εμπεριέχει άλλα components.

Card

Το component Card περιέχει άλλα components για το ίδιο θέμα, τα οποία στυλιστικά φέρουν σε κάρτα. Άλλα components είναι τα CardMedia για την εισαγωγή πολυμέσων, CardContent για την εισαγωγή του κυρίως θέματος, CardActions για την εισαγωγή μέσων αλληλεπίδρασης με την εφαρμογή.

Container

Το component Container είναι από τα πιο βασικά components που χρησιμοποιούνται για το layout της σελίδας. Το συγκεκριμένο στοιχίζει το περιεχόμενο οριζόντια.

Grid

Το component Grid χρησιμοποιείται για το responsiveness της σελίδας, δηλαδή το

περιεχόμενο της σελίδας να εφαρμόζει σωστά στο width και στο height της, σε συνδυασμό με τον ορισμό διαφόρων properties.

Pagination

Το component Pagination χρησιμοποιείται για την κατηγοριοποίηση των δεδομένων σε σελίδες.

ButtonBase

Το component ButtonBase χρησιμοποιείται για τη δημιουργία ενός Button.

CircularProgress

Το component CircularProgress χρησιμοποιείται για να υποδείξει στον χρήστη ότι κάποιο action που έχει πραγματοποιήσει στην εφαρμογή φορτώνεται στο back-end της εφαρμογής για να επέλθει το αποτέλεσμα.

Toolbar

Το component Toolbar χρησιμοποιείται στο εσωτερικό του component AppBar για να εμφανίζει τα δεδομένα οριζόντια.

2.2.3 MERN - BACKEND

2.2.3.1 NodeJS

Το NodeJS είναι μια πλατφόρμα ανάπτυξης λογισμικού χτισμένο σε Javascript περιβάλλον που τροφοδοτεί το client application της κάθε εφαρμογής, εκτελώντας Javascript κώδικα στην πλευρά του διακομιστή. Είναι ιδανικό για real-time εφαρμογές αφού κάνει χρήση ασύγχρονου προγραμματισμού[29]. Αυτό σημαίνει ότι είναι κατάλληλο για εφαρμογές που χρειάζονται να διαχειριστούν ένα μεγάλο όγκο ταυτόχρονων συνδέσεων σε αυτές. Στόχος του NodeJS είναι να παρέχει εύκολα λύση στη δημιουργία κλιμακωτών διαδικτυακών εφαρμογών[27]. Με την εγκατάσταση της πλατφόρμας μπορούν να εγκατασταθούν πολλά modules χρησιμοποιώντας το built-in package manager του Node (npm). Με την βοήθεια του Node μπορούν να χρησιμοποιούν πολλές βιβλιοθήκες ανοιχτού κώδικα[13][16][25].

2.2.3.2 ExpressJS

Το ExpressJS είναι ένα web framework που βασίζεται στον πυρήνα του NodeJS, HTTP Module, και connect components. Αυτό το framework χρησιμοποιείται για να ελέγχει τις συνεδρίες των χρηστών, καθώς φροντίζει για την σωστή λειτουργία της αυθεντικοποίησης και της σύνδεσης των χρηστών στην εφαρμογή, για όσο παραμένουν ενεργοί σε αυτή, και στη δημιουργία API[13]. Επίσης, ευθύνεται για την οργάνωση των routes της εφαρμογής. Μέσω του framework, τα routes της εφαρμογής έχουν συγκεκριμένα URL paths αφού υπάρχουν κατάλληλες συναρτήσεις που ελέγχουν τα εισερχόμενα HTTP αιτήματα (http requests), με ένα συγκεκριμένο URI το οποίο θα αντιστοιχηθεί σε ένα request handler (χειριστή αιτημάτων)[25][27][29].

2.2.3.3 MongoDB

Η Βάση Δεδομένων MongoDB είναι μια μη-σχεσιακή βάση δεδομένων η οποία έχει πιο ευέλικτο μοντέλο αποθήκευσης δεδομένων που αναπαριστά ένα πιο περίπλοκο ιεραρχικό μοντέλο το οποίο δεν είναι ένα προκαθορισμένο schema (σχήμα)[13][25]. Αυτό κάνει πιο εύκολη την προσθήκη ή την διαγραφή εγγραφών από την βάση. Η MongoDB αποθηκεύει τα δεδομένα σε μορφή JSON, όπου είναι η πιο εύχρηστη μορφή για την χρήση των δεδομένων στην εφαρμογή[27][30].

2.2.3.4 Axios

Το Axios είναι μια μοντέρνα βιβλιοθήκη της Javascript που έχει δημιουργηθεί έπειτα από το XMLHttpRequest για να πραγματοποιεί AJAX κλήσεις. Η κύρια χρήση του είναι να επιτρέπει τα HTTP αιτήματα (requests) και από τον φυλλομετρητή (browser) και από τον server (διακομιστή). Επιπλέον, υποστηρίζει το Promise API, αλλάζοντας τα δεδομένα του αιτήματος - απόκρισης (request - response) χρησιμοποιώντας υποσχέσεις (promises). Επίσης, αυτόματα μετατρέπει τα δεδομένα που είναι σε μορφή JSON καθιστώντας το ιδανικό για τις ασύγχρονες κλήσεις (asynchronous calls) στον server από το client-side application[32].

2.2.3.5 Mongoose

Το Mongoose είναι ένα ODM (Object Data Modeling) εργαλείο για την βάση δεδομένων MongoDB και την πλατφόρμα NodeJS. Πρακτικά, με το Mongoose υπάρχει η δυνατότητα να δηλωθεί στον κώδικα το data schema που θα χρησιμοποιηθεί στην βάση δεδομένων. Τα δεδομένα έχουν συγκεκριμένη οργάνωση και ιεραρχία (data structure) που παρέχουν μια καλύτερη διαχείριση στη βάση δεδομένων από ότι παρέχει το ίδιο το MongoDB. Το Mongoose, επίσης, παρέχει ενσωματωμένες βοηθητικές μεθόδους με πολλές λειτουργικότητες για να καλύψει όλες τις ανάγκες της εφαρμογής. Οπότε η χρηστικότητα του Mongoose είναι πολύ μεγάλη διότι υπάρχει η δυνατότητα ορισμού του data structure σε JSON μορφή μέσα στην εφαρμογή, με το οποίο πραγματοποιούνται κλήσεις της εφαρμογής στη βάση δεδομένων. Το data schema ορίζει το όνομα του κάθε αντικειμένου των δεδομένων, και τον τύπο του είτε αυτός είναι string, αριθμός, ημερομηνία ή boolean. Το Mongoose επιστρέφει τα δεδομένα σε JSON αντικείμενο που αντλούνται και χρησιμοποιούνται άμεσα, σε αντίθεση με το MongoDB που θα τα επέστρεφε σε JSON string. Μία ακόμη σημαντική χρήση του Mongoose είναι ότι μπορεί να επιστρέψει ένα μοντέλο (model) το οποίο είναι αναπαράσταση ενός αντικειμένου που αντιστοιχεί σε ένα data schema[27][31][33].

2.3 Ανασκόπηση παρόμοιων εφαρμογών

Σε αυτή την ενότητα θα αναλυθούν παρόμοιες εφαρμογές με την εφαρμογή της εργασίας, Travel Light, οι οποίες παρέχουν γνώσεις για ταξιδιωτικούς προορισμούς είτε με αξιολογήσεις χρηστών, είτε με έτοιμες πληροφορίες μέσω της εφαρμογής. Ονομαστικά, κάποιες από τις διαδικτυακές εφαρμογές είναι: Google Maps, Trip Advisor, Travello Travel Social Network. Στη συνέχεια θα παρουσιαστούν αναλυτικά.

2.3.1 Google Maps

Μία από τις πιο δημοφιλείς υπηρεσίες της Google είναι η εφαρμογή Google Maps, η οποία προσφέρει λεπτομερή χαρτογράφηση των δρόμων καθώς και πολλές δυνατότητες.

Η προβολή δορυφορικών εικόνων, οι οδηγίες μετάβασης σε ένα προορισμό με διάφορα μέσα (αυτοκίνητο, πεζός, MMM), η μέτρηση των αποστάσεων μεταξύ σημείων στο χάρτη, η αναζήτηση επιχειρήσεων σε κοντινή απόσταση, οι γενικές πληροφορίες για ένα σημείο καθώς και για τις καιρικές συνθήκες που επικρατούν, είναι μερικές εξ' αυτών. Επίσης, οι χρήστες μπορούν να παρέχουν σημαντικές πληροφορίες για το σημείο ενδιαφέροντος καθώς και να προσθέσουν αξιολογήσεις τόσο για τις τοποθεσίες στον χάρτη, όσο και για επιχειρήσεις, ωστόσο δεν θεωρείται ταξιδιωτικός οδηγός. Ο κάθε χρήστης, μπορεί να συνεισφέρει σε αξιολογήσεις αρκεί να είναι συνδεδεμένος με τον λογαριασμό της υπηρεσίας Google. Με αυτό τον τρόπο οι χρήστες αλληλεπιδρούν μεταξύ τους αφού μπορούν να ακολουθούν συγκεκριμένους χρήστες που έχουν το σήμα τοπικού οδηγού της Google ή ακόμα και άλλους χρήστες που γνωρίζουν. Με την λειτουργικότητα της ακολούθησες χρηστών δημιουργείται μια αρχική σελίδα στο Maps όπου αναρτώνται οι πιο πρόσφατες αξιολογήσεις των χρηστών, οι κοινοποιήσεις τοποθεσίας ή η διαμοίραση πολυμεσικού υλικού[20].

Εικόνα 2.3 Αρχική σελίδα Google Maps



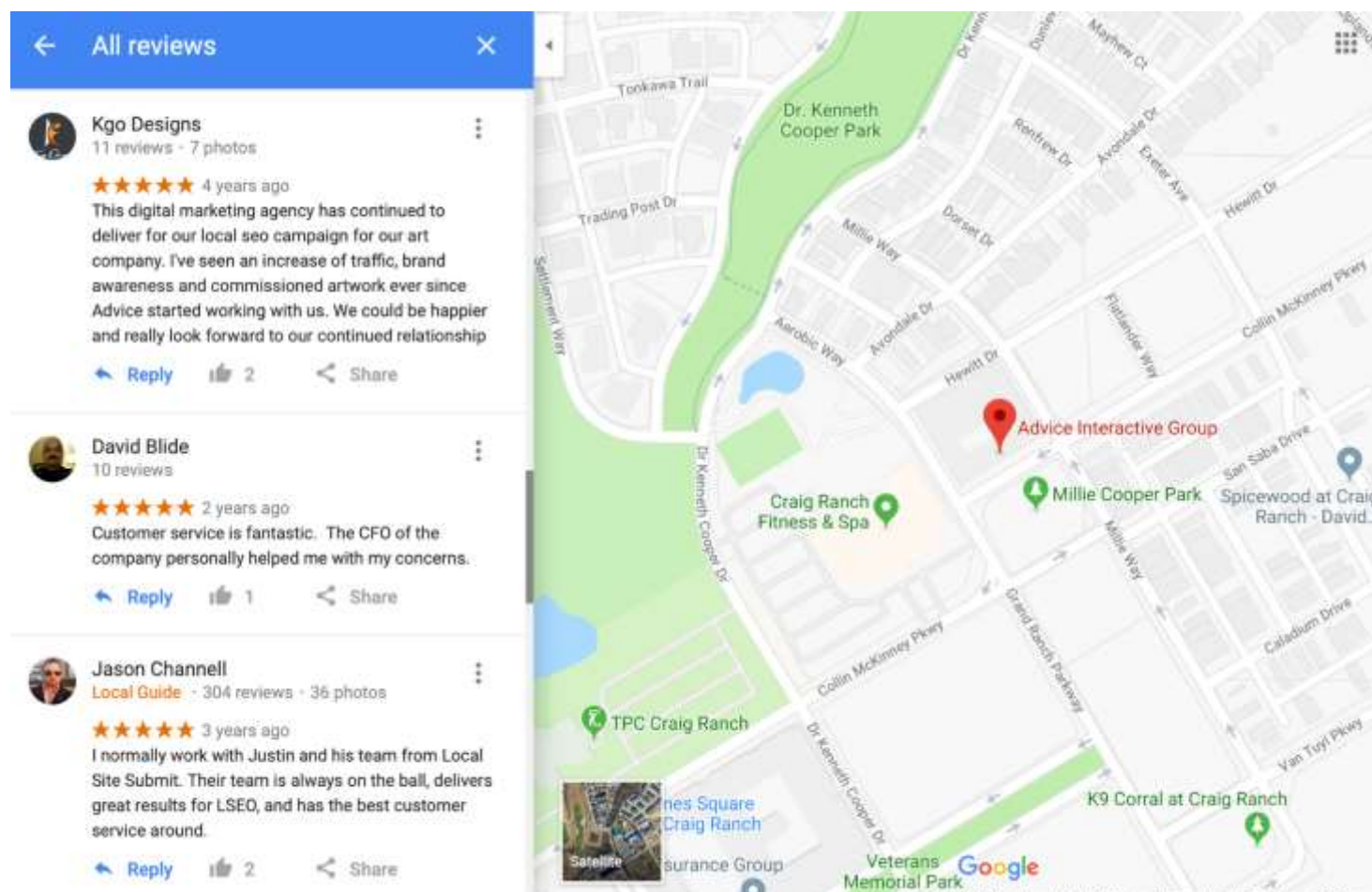
Πηγή: [17]

Εικόνα 2.4 Οδηγίες μετάβασης σε ένα προορισμό με διάφορα μέσα



Πηγή: [18]

Εικόνα 2.5 Αξιολογήσεις χρηστών για ένα συγκεκριμένο μέρος



Πηγή: [19]

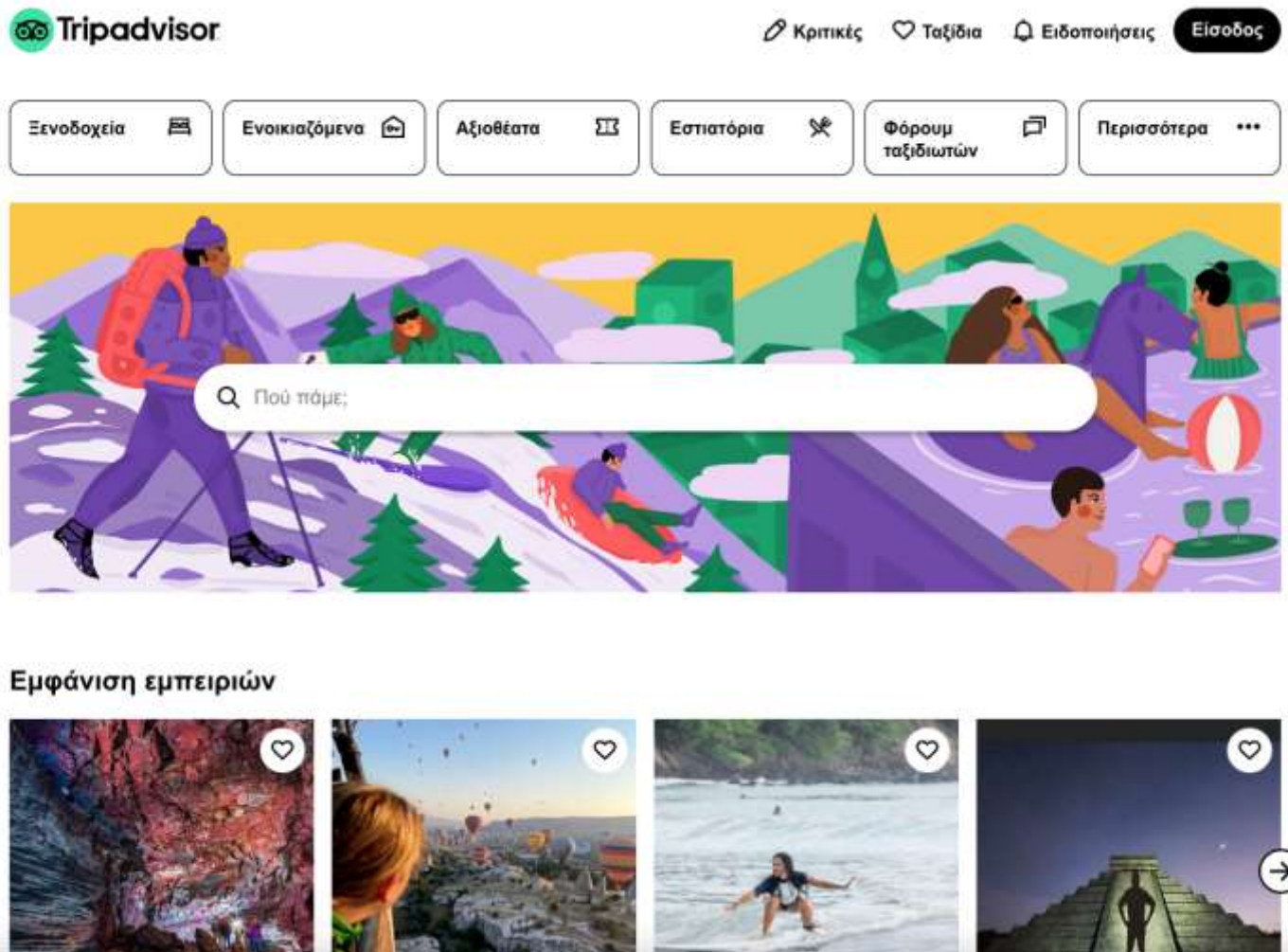
2.3.2 Trip Advisor

Η πιο δημοφιλής εφαρμογή και ιστοσελίδα, το Trip Advisor, προσφέρει μια ολοκληρωμένη υπηρεσία συστάσεων για καταλύματα, εστιατόρια, μέσα μεταφοράς, δραστηριότητες και πολλές άλλες επιλογές. Οι συστάσεις αυτές, βασίζονται στην συνεχή αξιολόγηση των χρηστών για κάθε επιλεγμένο μέρος. Βοηθά τους ταξιδιώτες να σχεδιάζουν τα ταξίδια τους αντλώντας εμπειρίες και κριτικές από άλλους ταξιδιώτες.

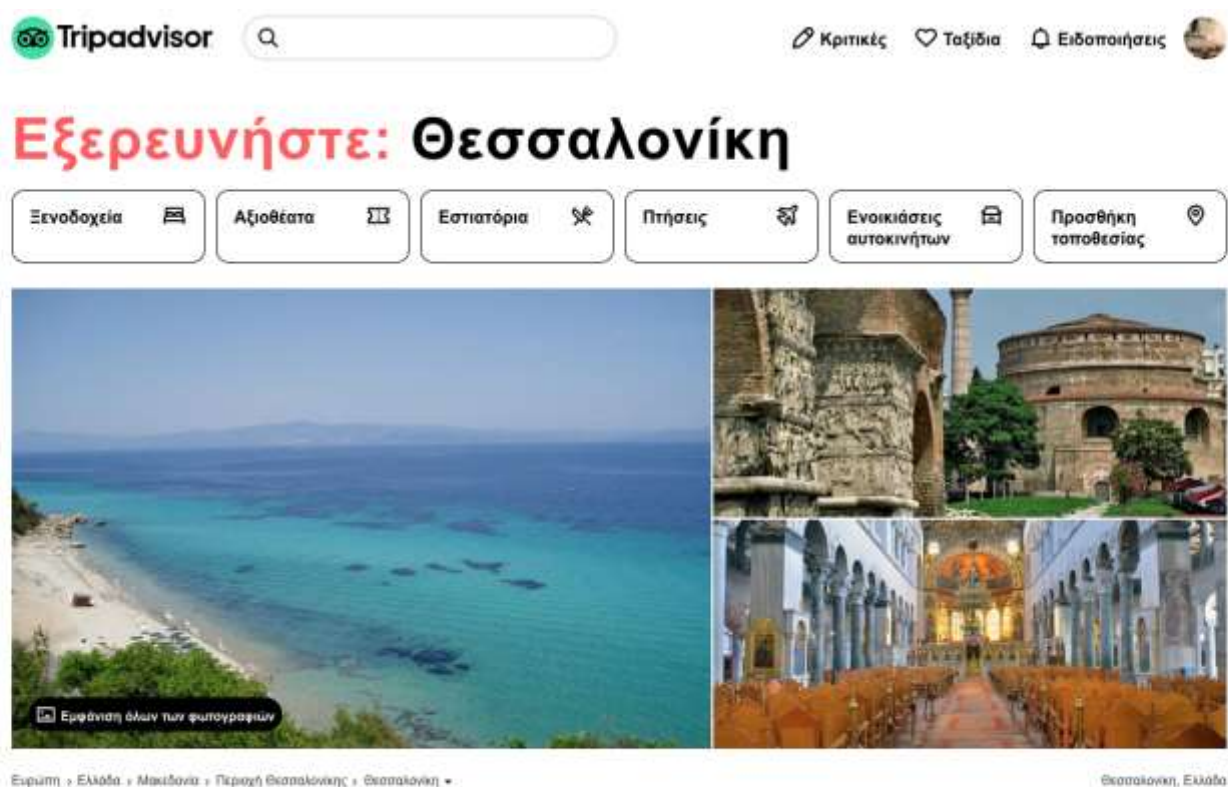
Ο κάθε χρήστης έχει τη δυνατότητα να χρησιμοποιεί την εφαρμογή και offline για την εύρεση πληροφοριών, όπως για παράδειγμα αξιοθέατα και καταλύματα. Η εφαρμογή για να ενισχύσει την συνεχή ενασχόληση(engagement) με τους χρήστες, παρέχει την λειτουργικότητα της προσθήκης φωτογραφιών του κάθε χρήστη από προσωπικά του ταξίδια, με σκοπό την δημιουργία ενός ημερολογίου που θα του προσφέρει ανά πάσα στιγμή πληροφορίες για τα ταξίδια του. Μία ακόμη λειτουργικότητα του Trip Advisor είναι η σύγκριση τιμών που πραγματοποιεί σε πάνω από 200 ιστοσελίδες για τις προτάσεις καταλυμάτων ή αεροπορικών εισιτηρίων που παρουσιάζει στους χρήστες.

Η εφαρμογή δεν είναι ταξιδιωτικό πρακτορείο ωστόσο συνεργάζεται με ταξιδιωτικά πρακτορεία, έτσι ώστε οι χρήστες που το επιθυμούν να κανονίζουν ολοκληρωμένα ταξίδια μέσω της εφαρμογής.

Εικόνα 2.6 Αρχική σελίδα Trip Advisor



Εικόνα 2.7 Εξερεύνηση προορισμού στο Trip Advisor



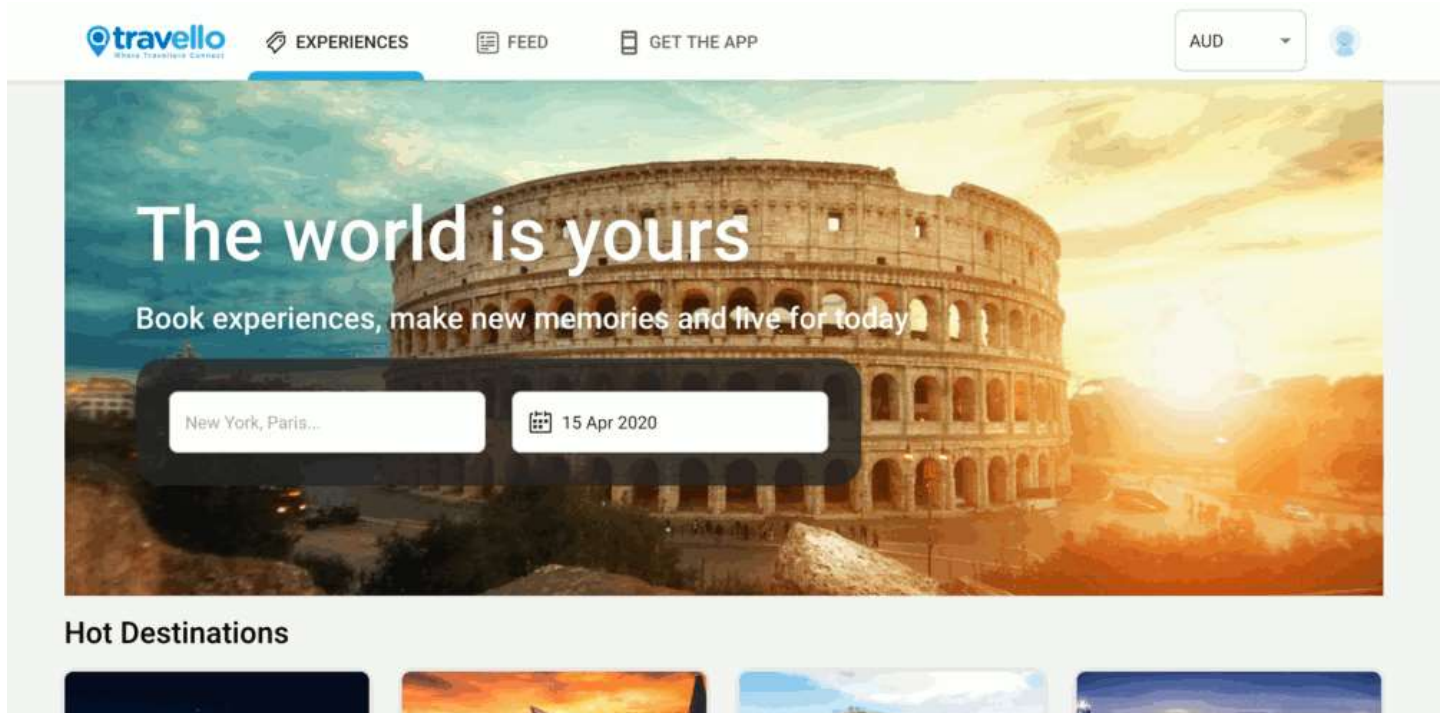
2.3.3 Travello, Travel Social Network

Η εφαρμογή Travello Social Network είναι μια online κοινότητα που χρησιμοποιούν όσοι ταξιδεύουν για να μοιράζονται εμπειρίες, γνώσεις και απόψεις. Η εφαρμογή χρησιμοποιείται σε περισσότερες από 180 χώρες και προσφέρει αρκετές δυνατότητες στους χρήστες της. Η σύνδεση στην εφαρμογή μπορεί να πραγματοποιηθεί με την δημιουργία λογαριασμού ή με την σύνδεση λογαριασμού Google, Facebook. Οι χρήστες κατηγοριοποιούνται σε ομάδες ανάλογα με τα ενδιαφέροντα τους και τις προτιμήσεις που έχουν δηλώσει κατά την εγγραφή τους στην εφαρμογή. Έχουν δημιουργηθεί ομάδες για τους χρήστες που αποτελούν τις ψηφιακές νομάδες, για εκείνους που ταξιδεύουν μόνοι, για εκείνους που ταξιδεύουν για επαγγελματικές φωτογραφίες και πολλά ακόμα.

Στην εφαρμογή ο κάθε χρήστης μπορεί να γνωρίσει, να ακολουθήσει και να συνομιλήσει με χρήστες διαφόρων εθνικοτήτων με κοινά ενδιαφέροντα, μπορεί να ανακαλύψει δραστηριότητες που υπάρχουν στην τοποθεσία του ή στα προγραμματισμένα ταξίδια του και έχει την δυνατότητα να τις κλείσει μέσω της εφαρμογής. Επίσης, παρέχει και τις κλασικές λειτουργίες των κοινωνικών δικτύων όπως like, comment, follow και share, καθώς και αρχικό feed με δημοσιεύσεις χρηστών. Ένα σημαντικό πλεονέκτημα της εφαρμογής για τους χρήστες της είναι οι πόνοι που πιστώνονται στον λογαριασμό τους όσο περισσότερο χρησιμοποιούν την εφαρμογή. Με τους πόντους έχουν εκπτώσεις στις τιμές των δραστηριοτήτων που

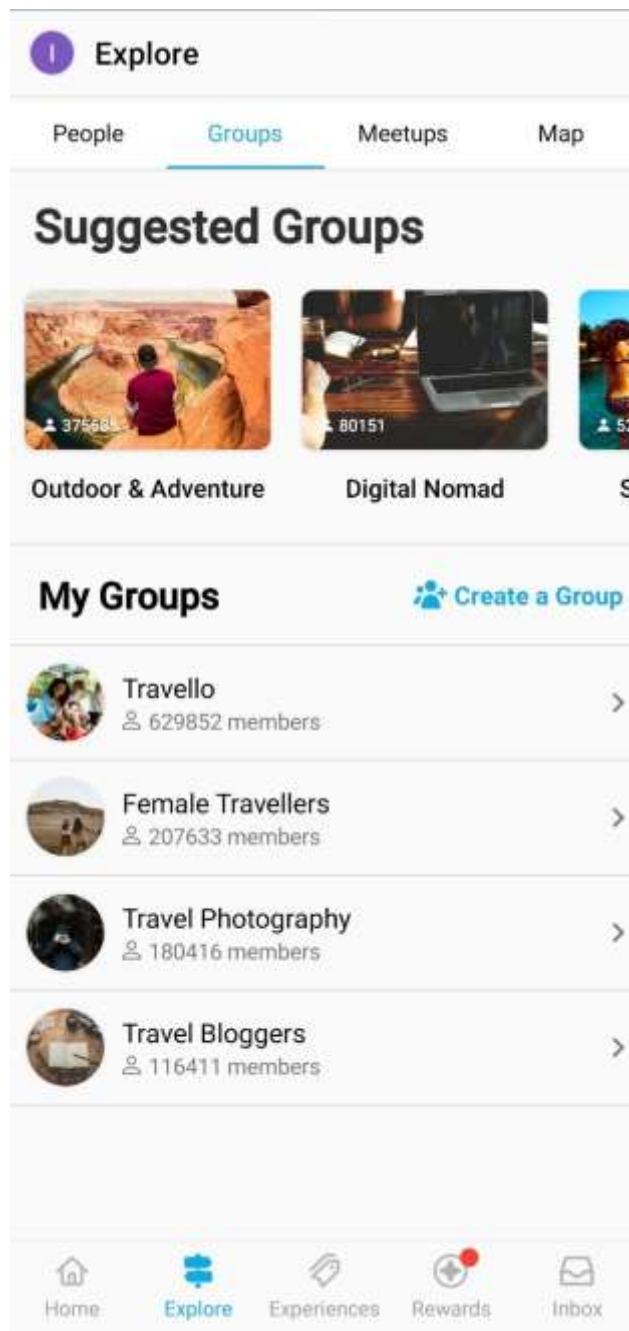
προσφέρονται από την εφαρμογή αλλά και προαγωγές στα status των προφίλ τους με το ανώτερο να είναι Guru της εφαρμογής.

Εικόνα 2.8 Αρχική σελίδα της εφαρμογής Travello

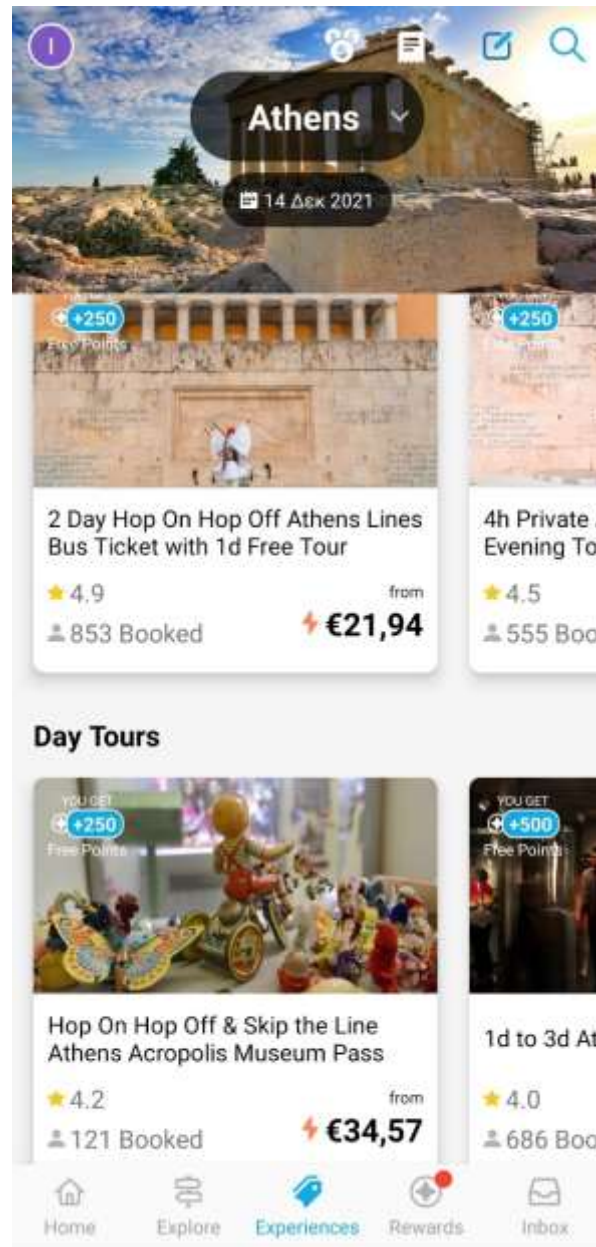


Πηγή: [22]

Εικόνα 2.9 Μενού της εφαρμογής Travello



Εικόνα 2.10 Εξερεύνηση δραστηριοτήτων που προσφέρονται μέσω της εφαρμογής για συγκεκριμένο μέρος και ημερομηνία



ΚΕΦΑΛΑΙΟ 3

Σχεδίαση και υλοποίηση εφαρμογής Travel Light

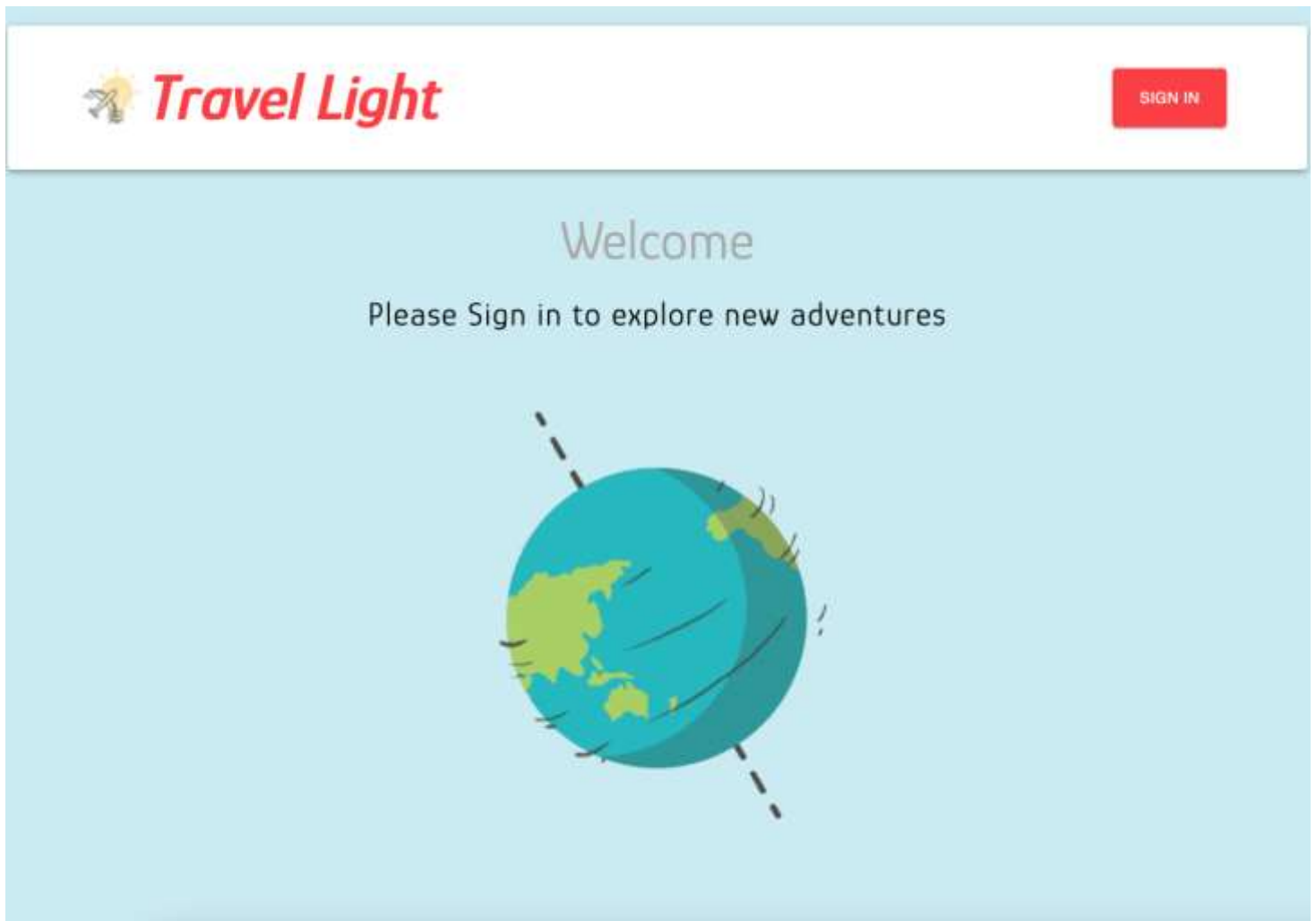
3.1 Ανάπτυξη εφαρμογής Travel Light

Η web εφαρμογή Travel Light δημιουργήθηκε με σκοπό να διευκολύνει τους χρήστες στα ταξίδια τους με τη διαμοίραση πληροφοριών. Πρόκειται για ένα ολοκληρωμένο σύστημα το οποίο βοηθά τους χρήστες να αναζητήσουν πληροφορίες και εμπειρίες χρηστών για ένα συγκεκριμένο μέρος. Η σύνδεση στην εφαρμογή πραγματοποιείται είτε με τη δημιουργία λογαριασμού δηλώνοντας το όνομα, το e-mail και κωδικό πρόσβασης είτε με σύνδεση του Google λογαριασμού. Ο εγγεγραμμένος χρήστης έχει στη διάθεση του όλες τις δυνατότητες της εφαρμογής οι οποίες είναι να δημιουργεί δημοσιεύσεις (Posts), να βλέπει όλες τις δημοσιεύσεις που έχουν δημοσιευθεί στην εφαρμογή με χρονολογική σειρά, καθώς και να πατά like σε αυτά που του αρέσουν. Μια σημαντική λειτουργία είναι η αναζήτηση που μπορεί να χρησιμοποιήσει ο χρήστης η οποία δεν αφορά μόνο μέρη ή δραστηριότητες αλλά και ετικέτες (hashtags) που υπάρχουν σε κάθε δημοσίευση. Το link της εφαρμογής είναι το εξής <https://travel-light-app.netlify.app/>.

3.1.1 Αρχική σελίδα εφαρμογής

Η αρχική σελίδα της εφαρμογής (landing page) παρέχει πληροφορίες για τον σκοπό της εφαρμογής καθώς και ένα σύνδεσμο που παραπέμπει στη σελίδα σύνδεσης χρήστη.

Εικόνα 3.1 Αρχική σελίδα εφαρμογής



What is Travel Light ?

SHARING YOUR FAVORITE MOMENTS



EXPLORING EXPERIENCES AND PLACES



PLANNING YOUR OWN EXPERIENCES



“

Experience, Travel - These are as education in themselves

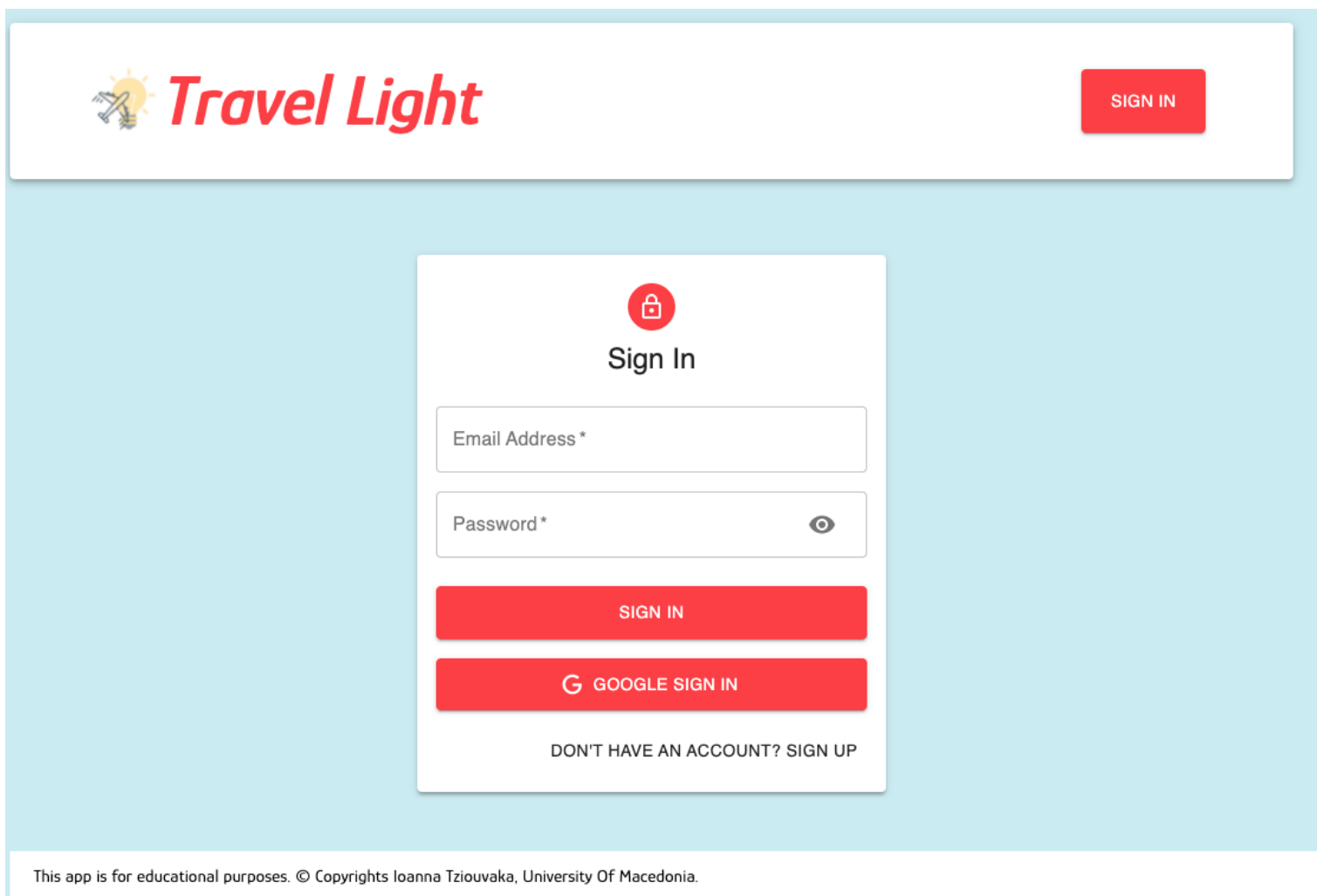
EURIPIDES

”

3.1.2 Σελίδα σύνδεσης και εγγραφής χρήστη

Ο χρήστης έχει την επιλογή να συνδεθεί με τον Google λογαριασμό του στην εφαρμογή, ή να δημιουργήσει λογαριασμό στην εφαρμογή συμπληρώνοντας τα στοιχεία του έτσι ώστε την επόμενη φορά που θα συνδεθεί να υπάρχει ο λογαριασμός.

Εικόνα 3.2 Σελίδα σύνδεσης λογαριασμού



Travel Light SIGN IN

Sign In

Email Address *

Password * 

SIGN IN

G GOOGLE SIGN IN

DON'T HAVE AN ACCOUNT? SIGN UP

This app is for educational purposes. © Copyrights Ioanna Tziouvaka, University Of Macedonia.

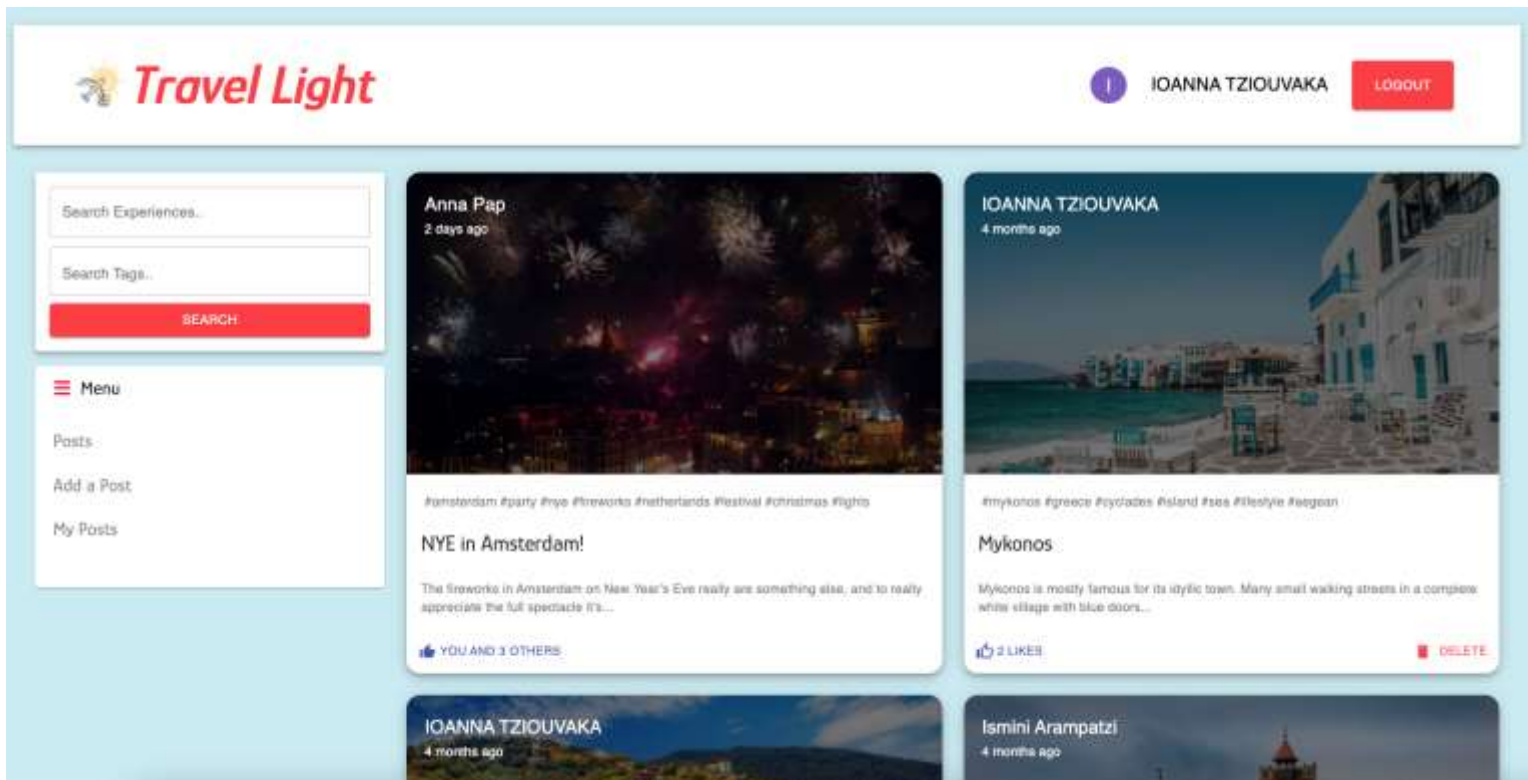
3.1.3 Σελίδα δημοσιεύσεων (Posts)

Στην κεντρική σελίδα της εφαρμογής προβάλλονται όλες οι δημοσιεύσεις που έχουν δημοσιευθεί από όλους τους εγγεγραμμένους χρήστες της εφαρμογής. Η δομή της κάθε δημοσίευσης αποτελείται από το όνομα του χρήστη, η χρονική στιγμή δημοσίευσης, η φωτογραφία που έχει ανεβάσει, οι ετικέτες, ο τίτλος της δημοσίευσης, μία προεπισκόπηση από το περιεχόμενο της δημοσίευσης και τέλος ο αριθμός των likes της δημοσίευσης.

Επιπλέον, στο sidebar της αρχικής σελίδας υπάρχει το μενού της εφαρμογής όπου υπάρχουν οι επιλογές στους εγγεγραμμένους χρήστες να περιηγηθούν στην

εφαρμογή. Από τις πιο σημαντικές λειτουργικότητες της εφαρμογής είναι αυτή της αναζήτησης. Υπάρχουν δύο πεδία αναζήτησης, το ένα αφορά εμπειρίες ή προορισμούς και πραγματοποιείται αναζήτηση στους τίτλους των δημοσιεύσεων και το δεύτερο πεδίο αφορά αναζήτηση λέξεων-κλειδιών της δημοσίευσης και πραγματοποιεί αναζήτηση στις ετικέτες που υπάρχουν σε κάθε δημοσίευση.

Εικόνα 3.3 Σελίδα δημοσιεύσεων



3.1.4 Σελίδα δημιουργίας δημοσίευσης(Add a Post)

Ο εγγεγραμμένος χρήστης κατά τη δημιουργία δημοσίευσης προσθέτει τον τίτλο της δημοσίευσης, την εμπειρία που θέλει να μοιραστεί, και τις ετικέτες (hashtags) που θα κάνουν τη δημοσίευση του να ξεχωρίζει και θα περιγράφουν καλύτερα την εμπειρία του έτσι ώστε να βοηθήσει άλλους χρήστες να τη βρουν από την αναζήτηση της εφαρμογής. Επίσης, ο χρήστης έχει τη δυνατότητα να ανεβάσει μια σχετική φωτογραφία με την δημοσίευση του, η οποία θα εμφανίζεται και στην προεπισκόπηση της δημοσίευσης αλλά και στην ίδια τη σελίδα της δημοσίευσης.

Εικόνα 3.4 Σελίδα δημιουργίας δημοσίευσης

 **Travel Light**

IO

IOANNA TZIOUVAKA

LOGOUT

[Back to Posts](#)



Tell people a few things about your upcoming post!

You can detail a beautiful city, or describe your unique experience when skydiving. Tell people about your Sunday walk by your favorite river. Show them your favorite spots that are hidden gems. You have the chance to use hashtags to assist users who search posts like yours.

Make sure to share special tips to help other users!

Creating an Experience:

Title

Your Experience

Tags

Επιλογή αρχείου

Δεν επιλέχθηκε κανένα αρχείο.

SUBMIT

CLEAR

This app is for educational purposes. © Copyrights Ioanna Tziouvaka, University Of Macedonia.

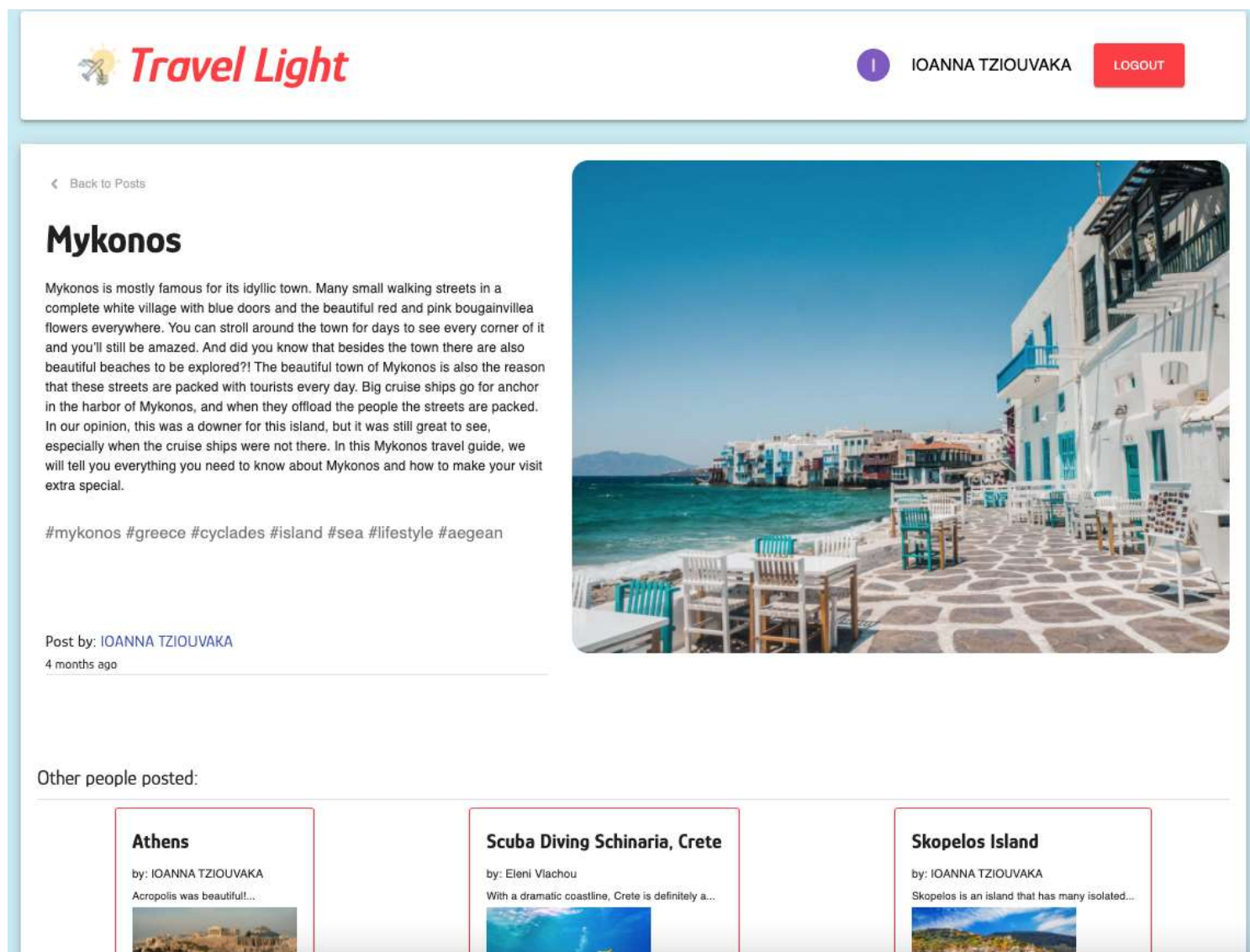
3.1.5 Σελίδα δημοσίευσης(Post)

Η κάθε δημοσίευση όταν επιλέγεται εμφανίζεται σε μία σελίδα η οποία περιέχει αναλυτικά την εμπειρία του χρήστη συνοδευόμενη από φωτογραφία, τον τίτλο και τις ετικέτες. Επίσης, στη σελίδα εμφανίζεται το όνομα του χρήστη στον οποίο ανήκει η δημοσίευση και η ημερομηνία που πραγματοποίησε την δημοσίευση.

Ο χρήστης έχει τη δυνατότητα να δει όλες τις δημοσιεύσεις που έχει πραγματοποιήσει ένας άλλος χρήστης πατώντας το όνομα του που υπάρχει σαν σύνδεσμος στην εκάστοτε δημοσίευση. Στο τέλος της σελίδας του post, προτείνονται στον χρήστη άλλες δημοσιεύσεις που έχουν κοινοποιήσει άλλοι χρήστες. Αυτό τους βοηθά να ανακαλύψουν περισσότερες δημοσιεύσεις στην εφαρμογή ακόμα και μη

σχετικές με τις αναζητήσεις του με αποτέλεσμα να μην υπάρχει μεγάλο ποσοστό εγκατάλειψης (bounce rate).

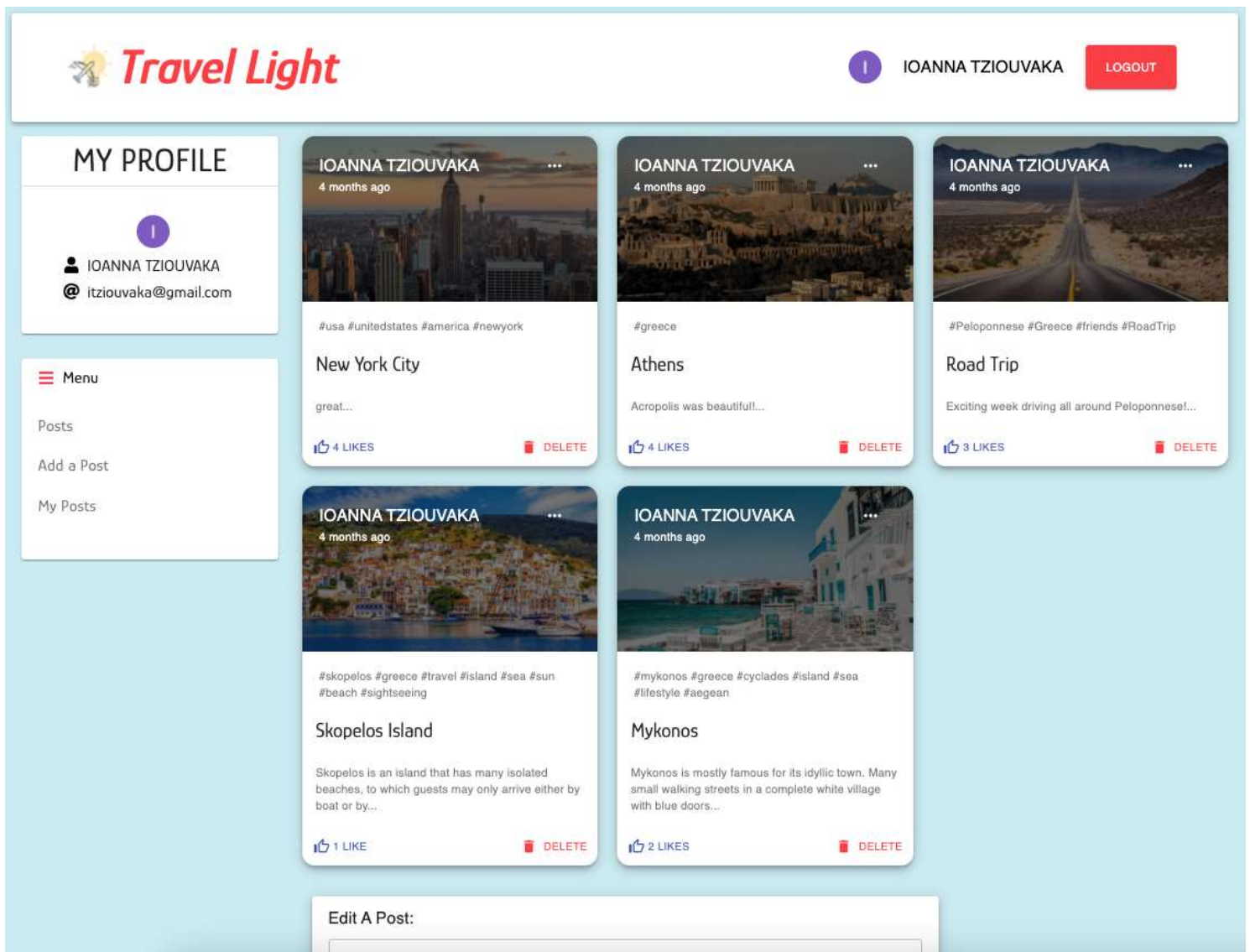
Εικόνα 3.5 Σελίδα ενδεικτικής δημοσίευσης



3.1.6 Σελίδα προφίλ χρήστη(My Posts)

Ο συνδεδεμένος χρήστης μπορεί να μπει στο προφίλ του όπου εκεί εμφανίζονται τα στοιχεία του όπως το e-mail του, το username του, το avatar του, αλλά και όλες τις δημοσιεύσεις που έχει κοινοποιήσει στην εφαρμογή. Αυτές οι δημοσιεύσεις μπορούν με την επιθυμία του χρήστη να διαγραφούν ή να τροποποιηθούν αλλάζοντας οτιδήποτε επιθυμεί από το εκάστοτε post.

Εικόνα 3.6 Προφίλ χρήστη



3.2 Περιπτώσεις Χρήσης

Για την περιήγηση στην εφαρμογή υπάρχουν ενδεικτικά σενάρια τα οποία θα παρουσιαστούν παρακάτω.

3.2.1 Πρώτο Σενάριο

Το πρώτο σενάριο αφορά τη περιήγηση ενός χρήστη που είναι συνδεδεμένος στην εφαρμογή.

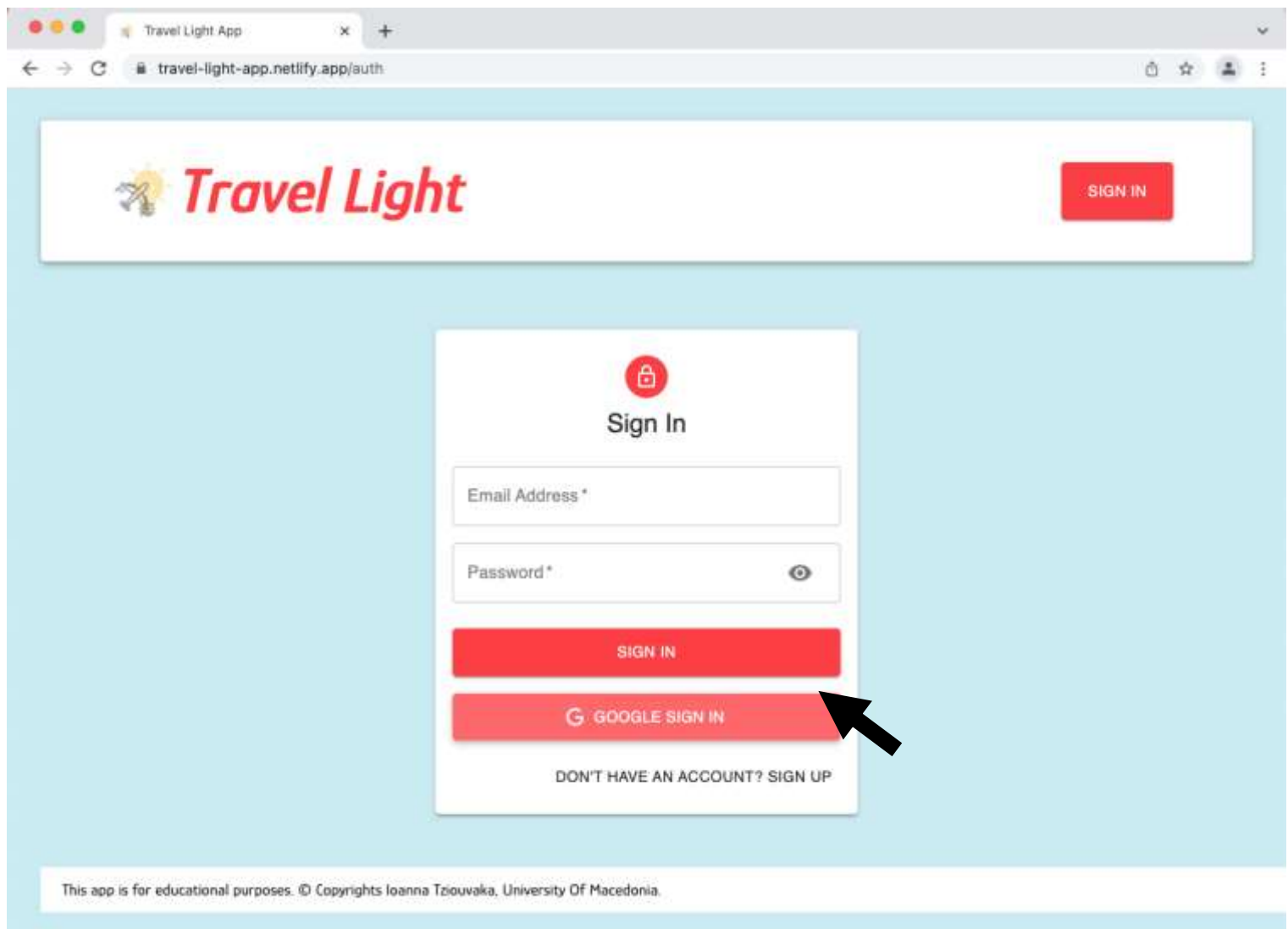
3.2.1.1 Σύνδεση στην εφαρμογή

Στην εφαρμογή Travel Light παρέχονται δύο τρόποι σύνδεσης, με δημιουργία λογαριασμού ή με Google Login.

Σύνδεση με Google Login

Ο χρήστης μπορεί να συνδεθεί με τον λογαριασμό Google που κατέχει.

Εικόνα 3.7 Σύνδεση με Google Login



Εγγραφή και σύνδεση στην εφαρμογή

Σε περίπτωση που ο χρήστης δεν επιθυμεί να συνδεθεί με τον λογαριασμό Google ή δεν έχει δημιουργήσει λογαριασμό Google, μπορεί να δημιουργήσει ένα λογαριασμό στην εφαρμογή δηλώνοντας το e-mail, το username και έναν κωδικό πρόσβασης.

Εικόνα 3.8 Εγγραφή στην εφαρμογή



SIGN IN



Sign Up

First Name *

User

Last Name *

Ioanna

Email Address *

userioanna@gmail.com

Password *

.....



Repeat Password *

.....

SIGN UP

 GOOGLE SIGN IN

ALREADY HAVE AN ACCOUNT? SIGN IN

Εικόνα 3.9 Σύνδεση στην εφαρμογή με τα στοιχεία της εγγραφής

Travel Light App

travel-light-app.netlify.app/auth

 **Travel Light**

SIGN IN


Sign In

Email Address*
userioanna@gmail.com

Password*
.....

SIGN IN

 **GOOGLE SIGN IN**

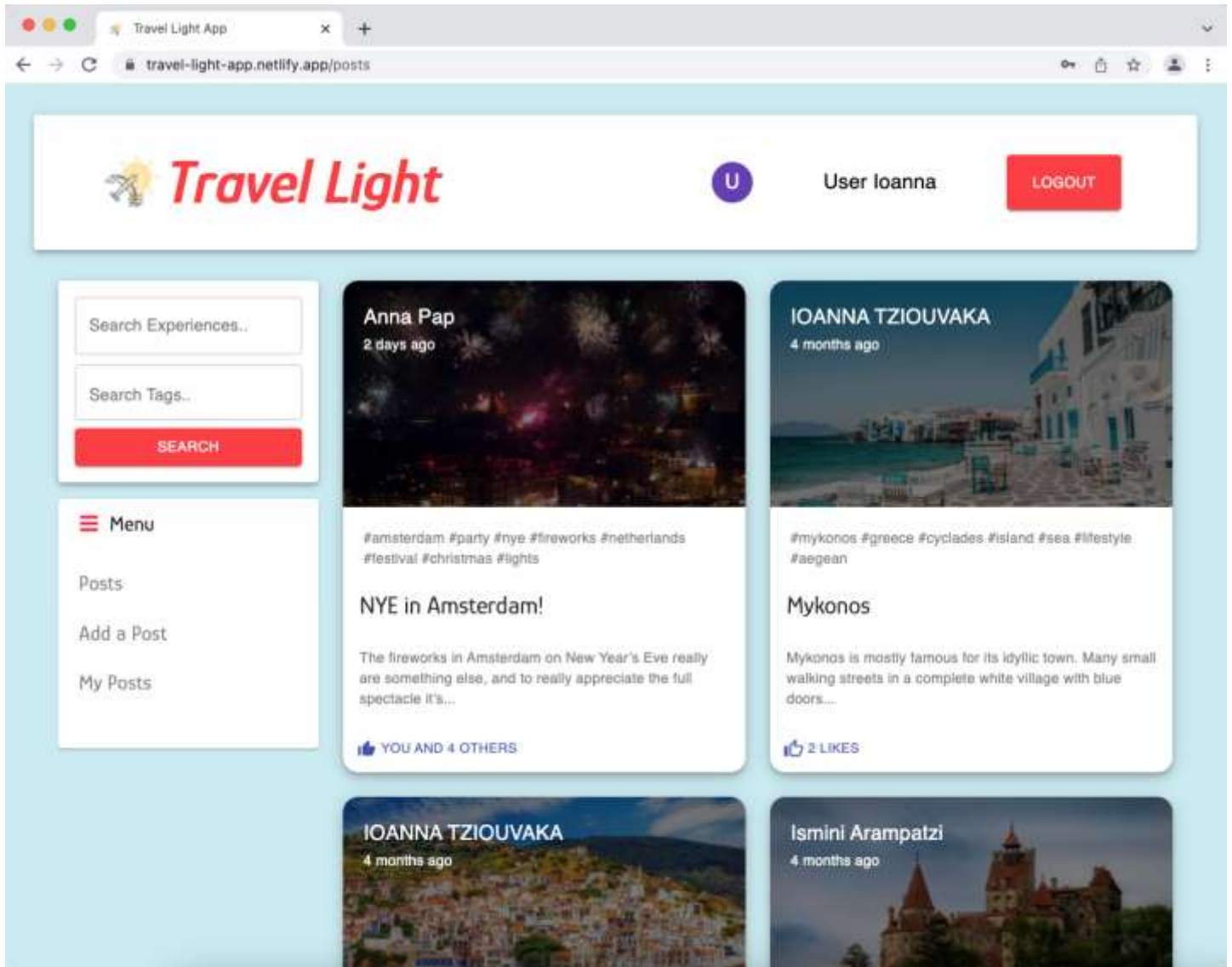
DON'T HAVE AN ACCOUNT? SIGN UP

This app is for educational purposes. © Copyrights Ioanna Tziouvala, University Of Macedonia.

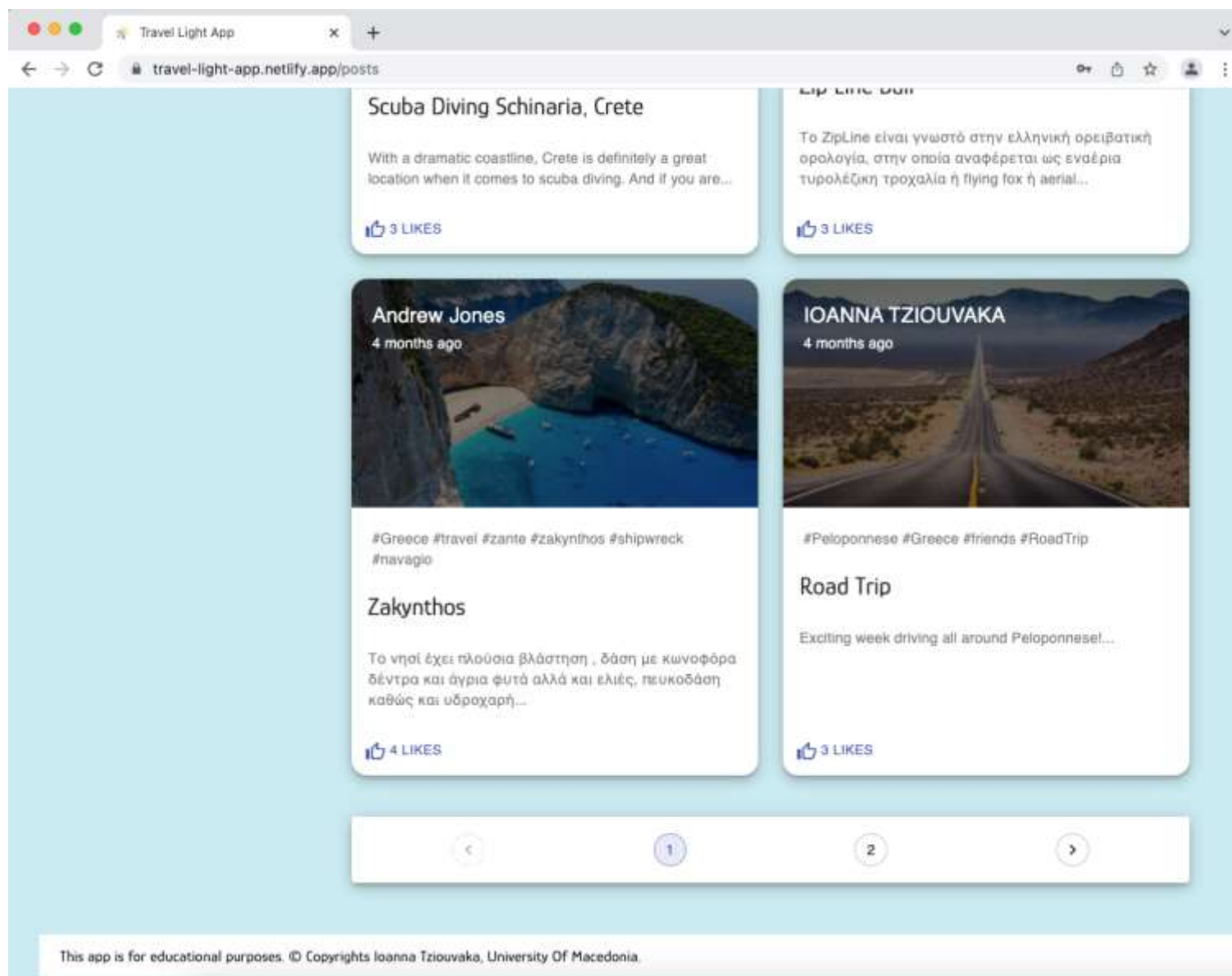
3.2.1.2 Περιήγηση στην αρχική σελίδα

Στην αρχική σελίδα εμφανίζεται το περιεχόμενο της εφαρμογής. Στα αριστερά υπάρχει το μενού καθώς και οι μπάρες αναζήτησης. Στο κέντρο εμφανίζονται όλες οι δημοσιεύσεις που υπάρχουν στην εφαρμογή από την πιο πρόσφατη στην πιο παλιά. Οι δημοσιεύσεις χωρίζονται σε σελίδες.

Εικόνα 3.10 Αρχική σελίδα δημοσιεύσεων συνδεδεμένου χρήστη



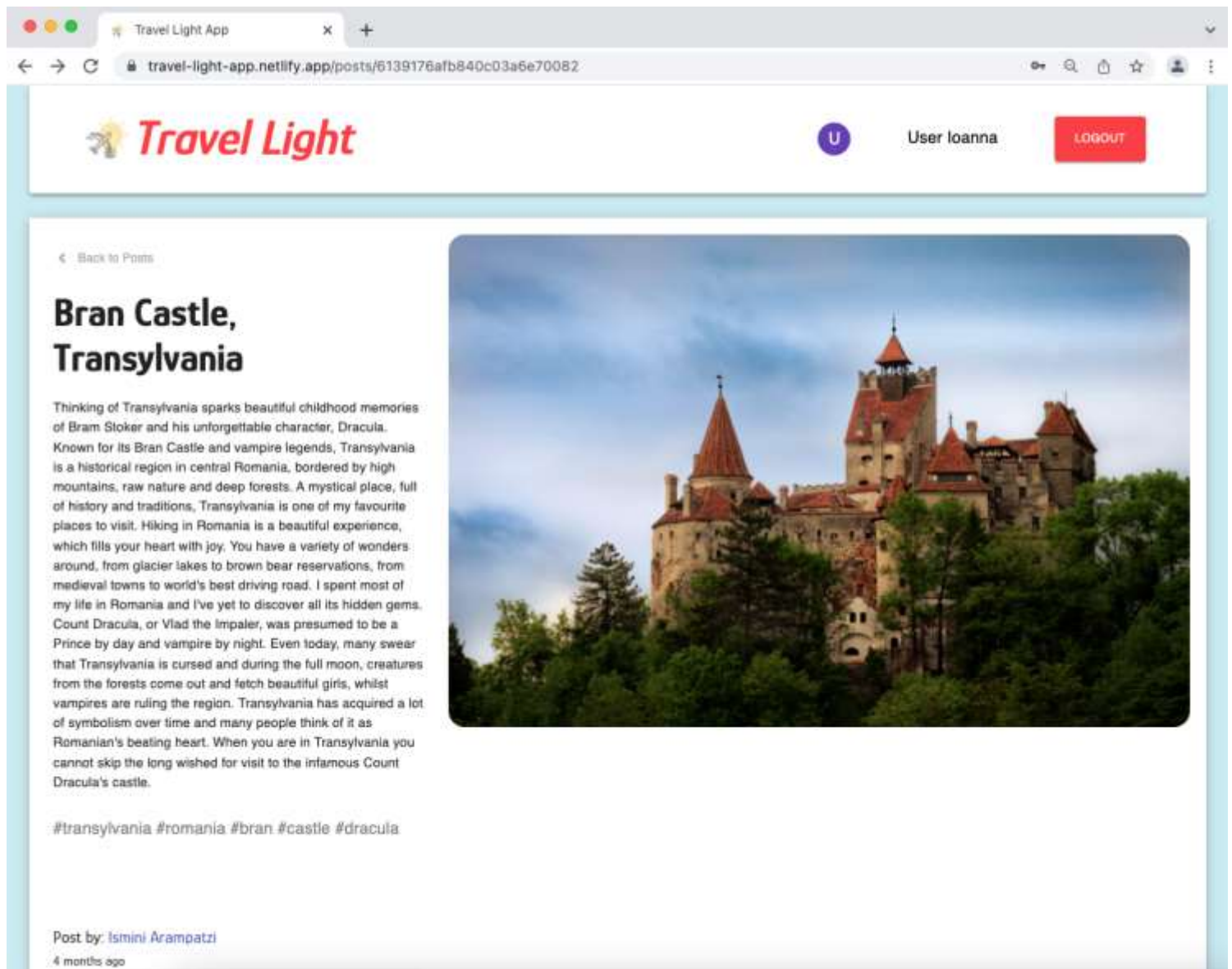
Εικόνα 3.11 Σελιδοποίηση δημοσιεύσεων



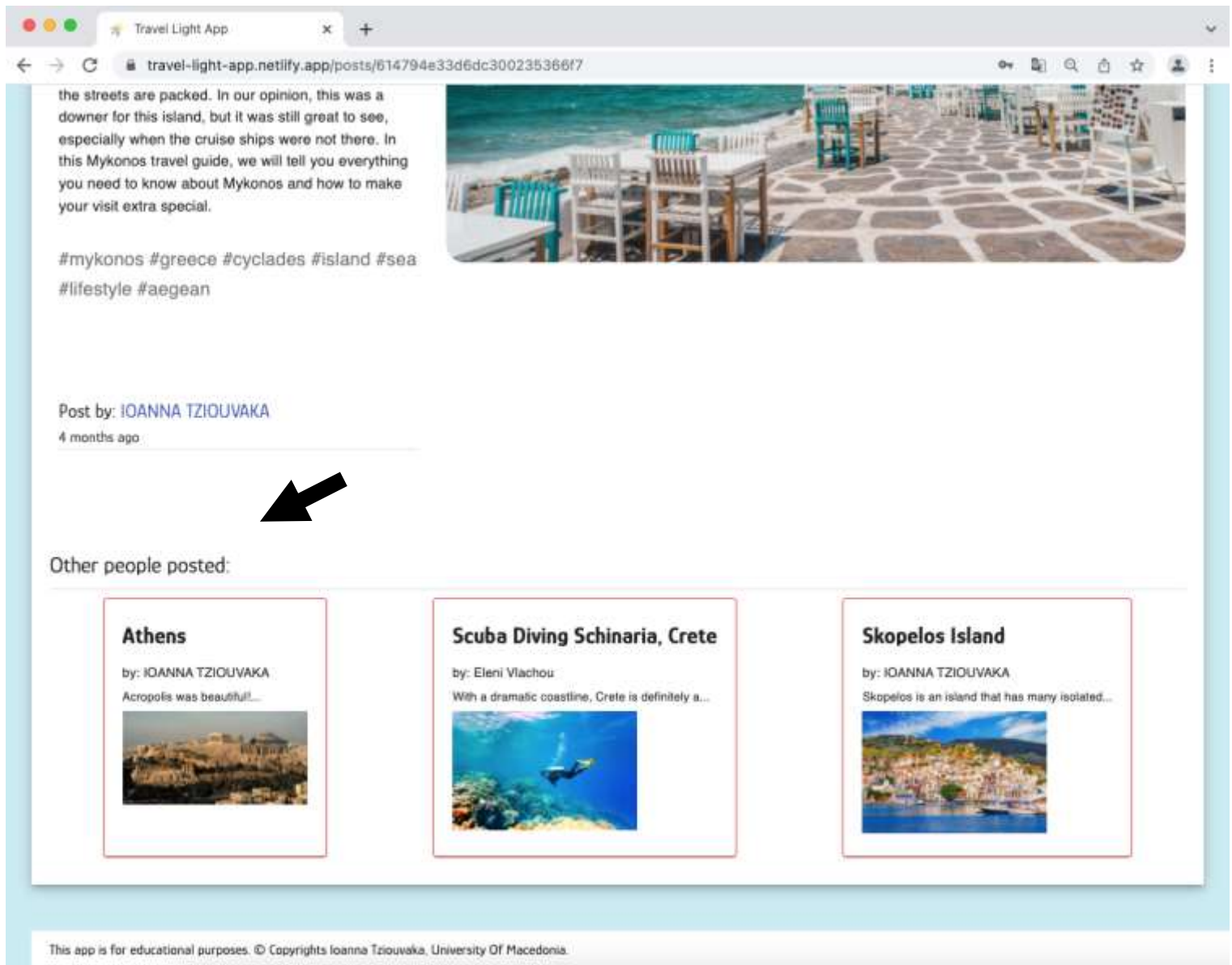
Επίσκεψη σε δημοσίευση

Η προεπισκόπηση κάθε δημοσίευση, που υπάρχει στην αρχική σελίδα, οδηγεί σε σελίδα δημοσίευσης. Σε αυτή τη σελίδα υπάρχουν ο τίτλος της δημοσίευσης, το username του χρήστη που την κοινοποίησε καθώς και η χρονική στιγμή που κοινοποιήθηκε, οι ετικέτες, η φωτογραφία στα δεξιά της σελίδας και το περιεχόμενο της δημοσίευσης. Στο τέλος της σελίδας υπάρχουν προτάσεις για τον χρήστη.

Εικόνα 3.12 Σελίδα δημοσίευσης συνδεδεμένου χρήστη



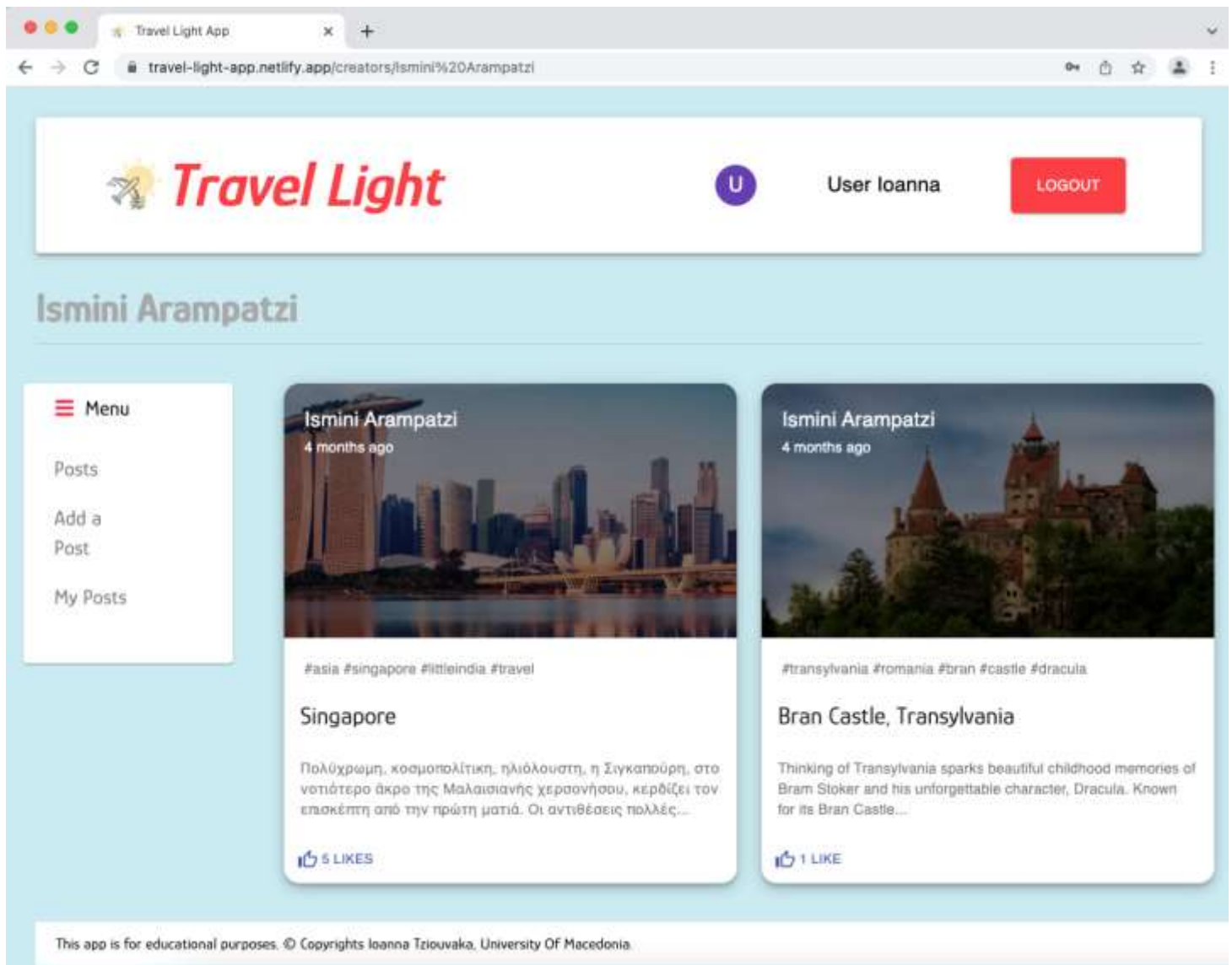
Εικόνα 3.13 Προτάσεις δημοσιεύσεων εφαρμογής για τον χρήστη



Επίσκεψη σε προφίλ χρηστών

Στην σελίδα της δημοσίευσης, περιέχεται σε σύνδεσμο το username του χρήστη που ανήκει η δημοσίευση. Ο σύνδεσμος αυτός οδηγεί στο προφίλ του χρήστη όπου εμφανίζονται όλες οι δημοσιεύσεις που έχει αναρτήσει.

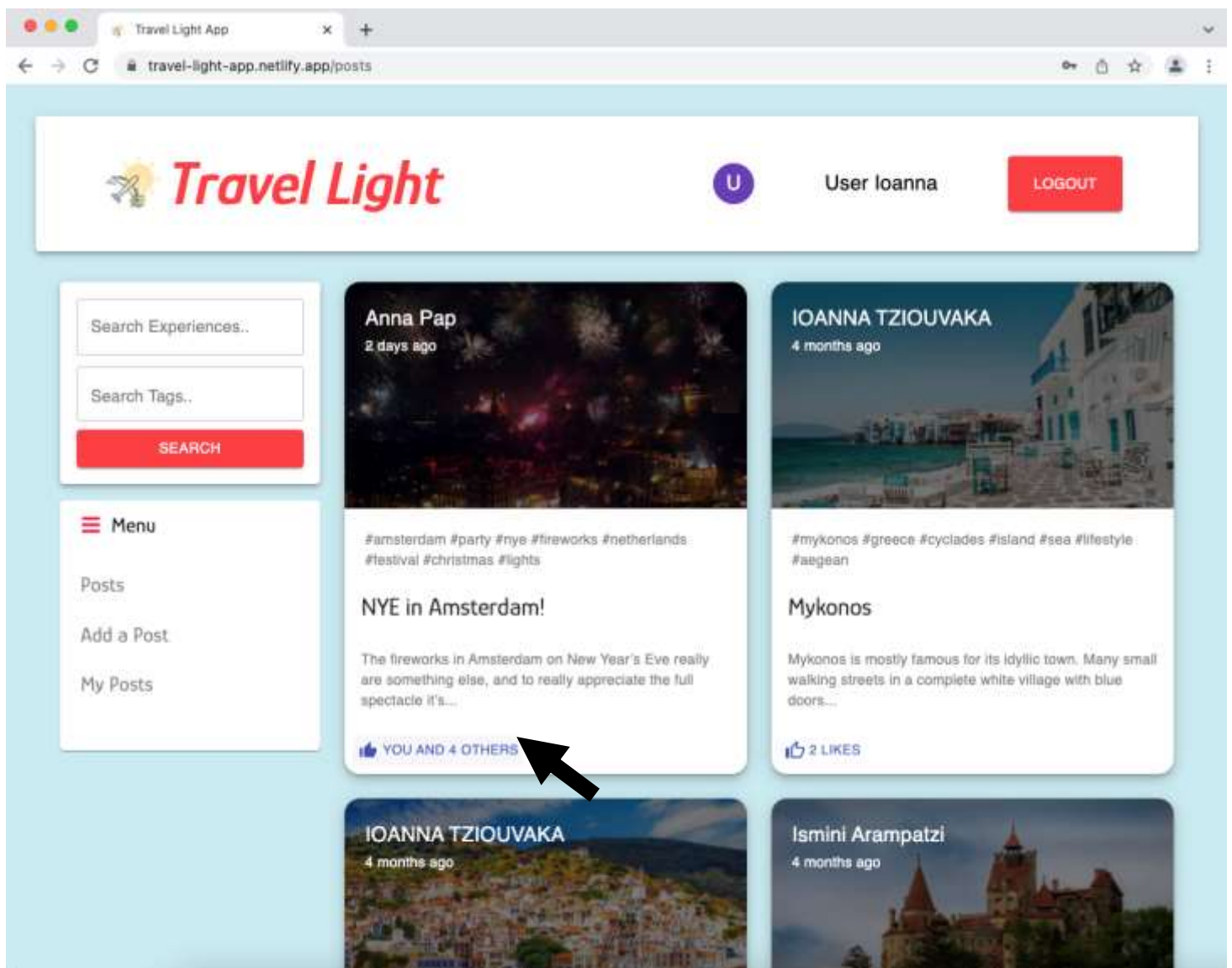
Εικόνα 3.14 Προφίλ άλλου χρήστη



Εκδήλωση Like σε δημοσίευση

Στην προεπισκόπηση των δημοσιεύσεων υπάρχει η λειτουργικότητα του Like. Ο συνδεδεμένος χρήστης μπορεί να δηλώσει ότι του αρέσει η δημοσίευση ενός άλλου χρήστη. Επίσης είναι ορατός και ο αριθμός των Likes της δημοσίευσης.

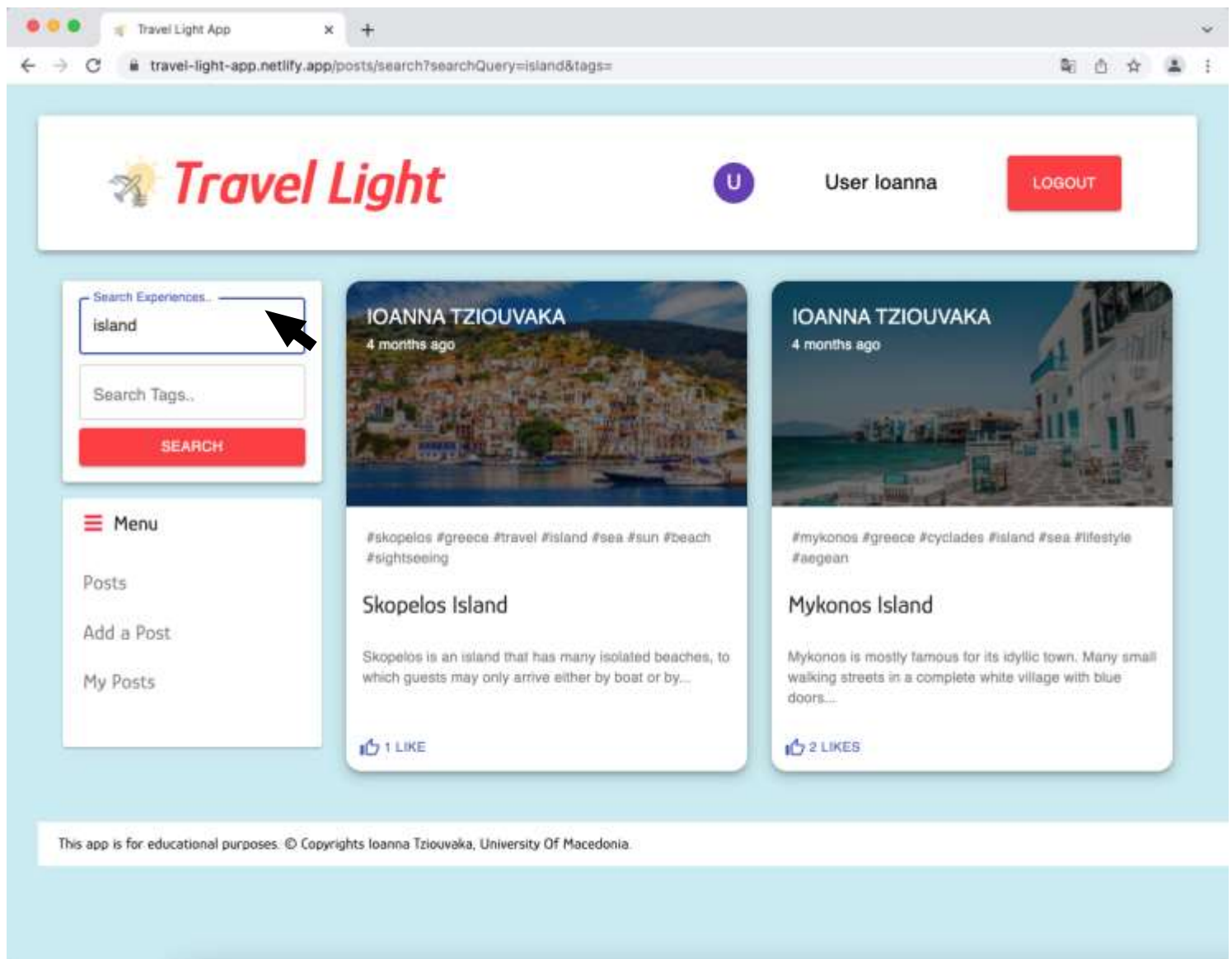
Εικόνα 3.15 Λειτουργικότητα Like δημοσίευσης



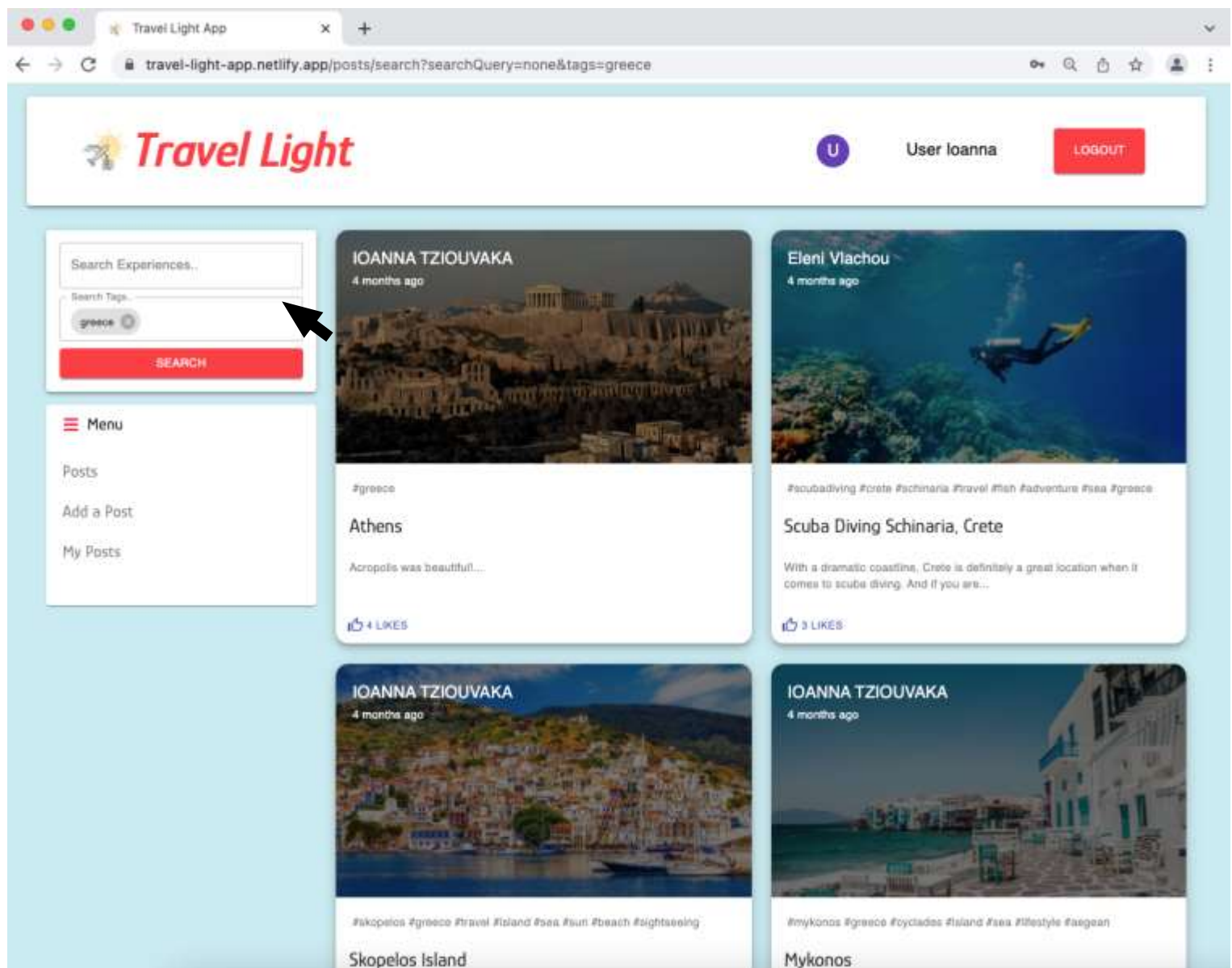
3.2.1.3 Αναζήτηση τίτλων και ετικετών

Στην αριστερή πλευρά της αρχικής σελίδας υπάρχουν οι μπάρες αναζήτησης. Στο πεδίο “Search Experiences” η εφαρμογή πραγματοποιεί αναζήτηση στους τίτλους των δημοσιεύσεων προκειμένου να εμφανίσει αποτελέσματα. Στο πεδίο “Search Tags” η εφαρμογή πραγματοποιεί αναζήτηση στις ετικέτες που υπάρχουν σε κάθε δημοσίευση προκειμένου να εμφανίσει αποτελέσματα.

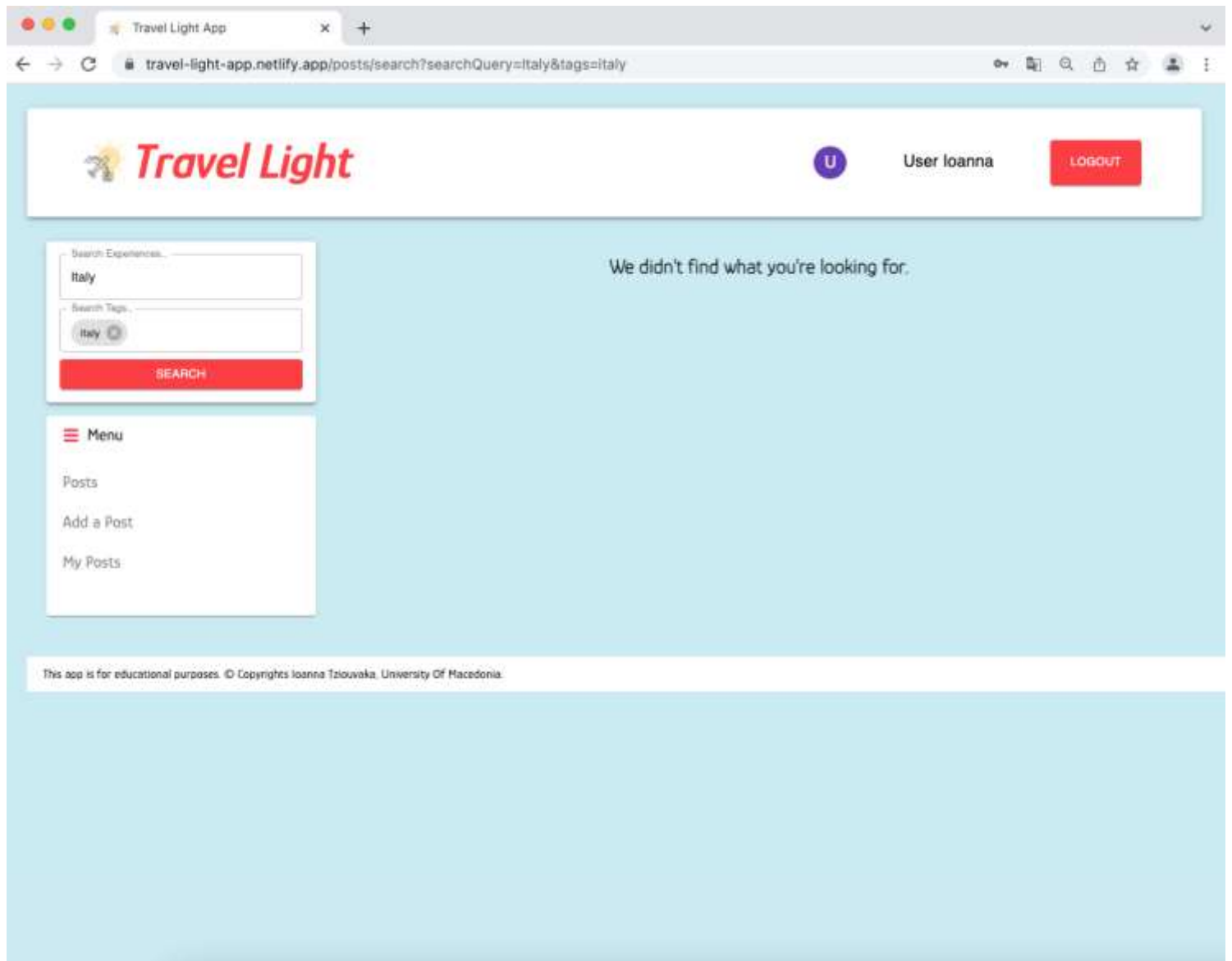
Εικόνα 3.16 Αναζήτηση τίτλων συνδεδεμένου χρήστη



Εικόνα 3.17 Αναζήτηση ετικετών συνδεδεμένου χρήστη



Εικόνα 3.18 Αναζήτηση ετικετών συνδεδεμένου χρήστη χωρίς αποτελέσματα



3.2.1.4 Δημιουργία δημοσίευσης

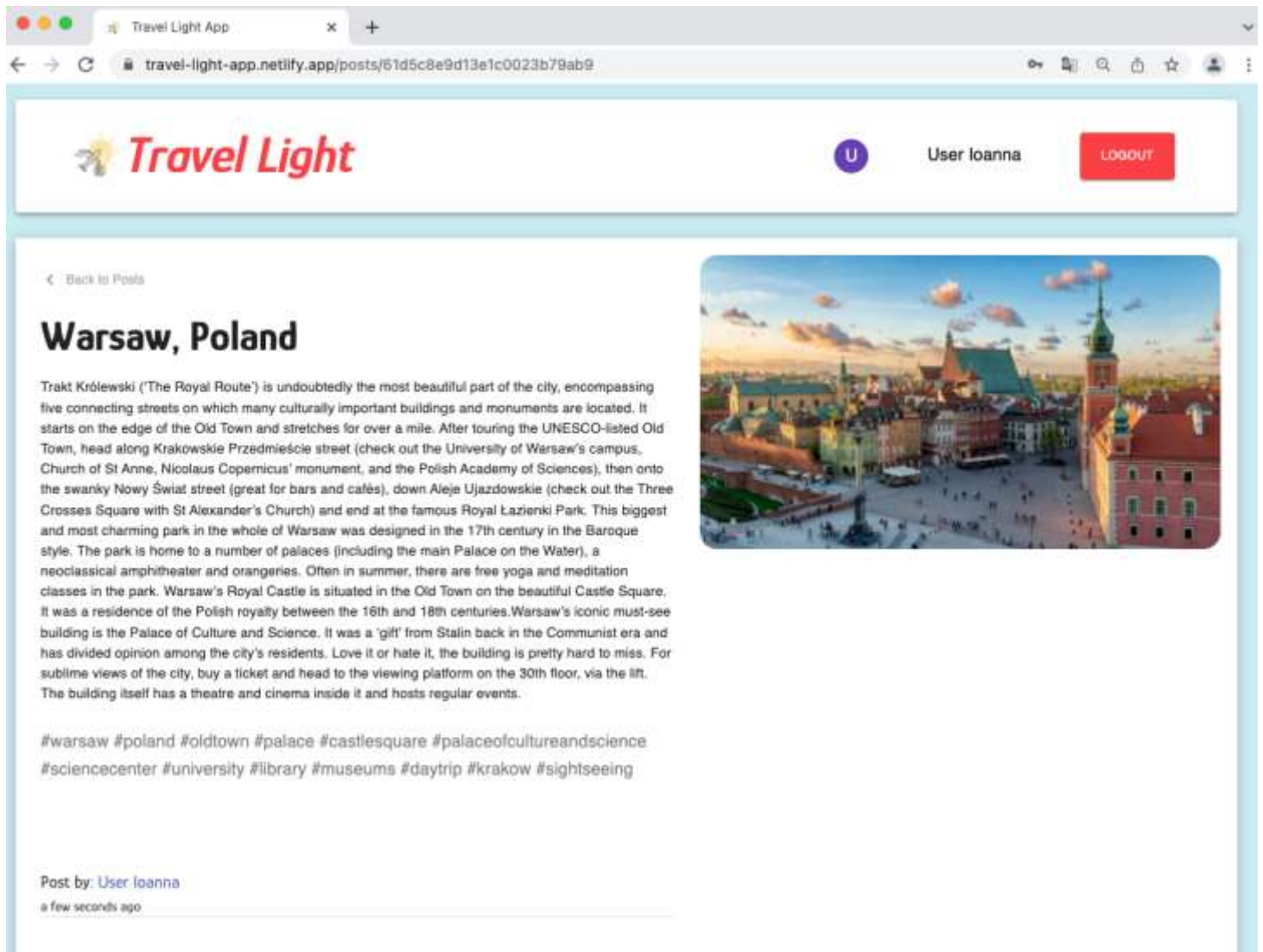
Η επιλογή στο μενού “Add a Post” οδηγεί στη σελίδα που αφορά τη δημιουργία μιας δημοσίευσης. Στη σελίδα υπάρχει μια φόρμα όπου ο συνδεδεμένος χρήστης καλείται να συμπληρώσει ορισμένα πεδία για να δημιουργήσει την ανάρτηση. Αυτά τα πεδία είναι ο τίτλος, το κείμενο της ανάρτησης, οι ετικέτες και η επισύναψη φωτογραφίας.

Εικόνα 3.19 Δημιουργία δημοσίευσης

The screenshot shows the 'Travel Light App' interface for creating a post. The page has a light blue background. At the top, there's a header with the 'Travel Light' logo, a user profile icon labeled 'User Ioanna', and a red 'LOGOUT' button. Below the header, there's a 'Back to Posts' link. The main content area features a globe illustration with various landmarks and speech bubbles. To the right of the globe, there's a text box with instructions: 'Tell people a few things about your upcoming post!', 'You can detail a beautiful city, or describe your unique experience when skydiving. Tell people about your Sunday walk by your favorite river. Show them your favorite spots that are hidden gems. You have the chance to use hashtags to assist users who search posts like yours.', and 'Make sure to share special tips to help other users!'. Below this, there's a 'Creating an Experience:' section with a form. The form has a 'Title' field with 'Warsaw, Poland' entered. Below the title is a 'Your Experience' text area containing a paragraph about Warsaw's Royal Castle. Underneath the text area is a 'Tags' section with a grid of tags: warsaw, poland, oldtown, palace, castlesquare, palaceofcultureandscience, sciencecenter, university, library, museums, daytrip, krakow, sightseeing. Below the tags is a 'Επισύνηψη αρχείου' (File Upload) field with 'warsaw.jpg' entered. At the bottom of the form, there are two buttons: a red 'SUBMIT' button and a grey 'CLEAR' button. A black arrow points to the 'SUBMIT' button.

Αφού ο χρήστης υποβάλει την δημοσίευση του η εφαρμογή τον ανακατευθύνει στην δημοσίευση που δημιούργησε.

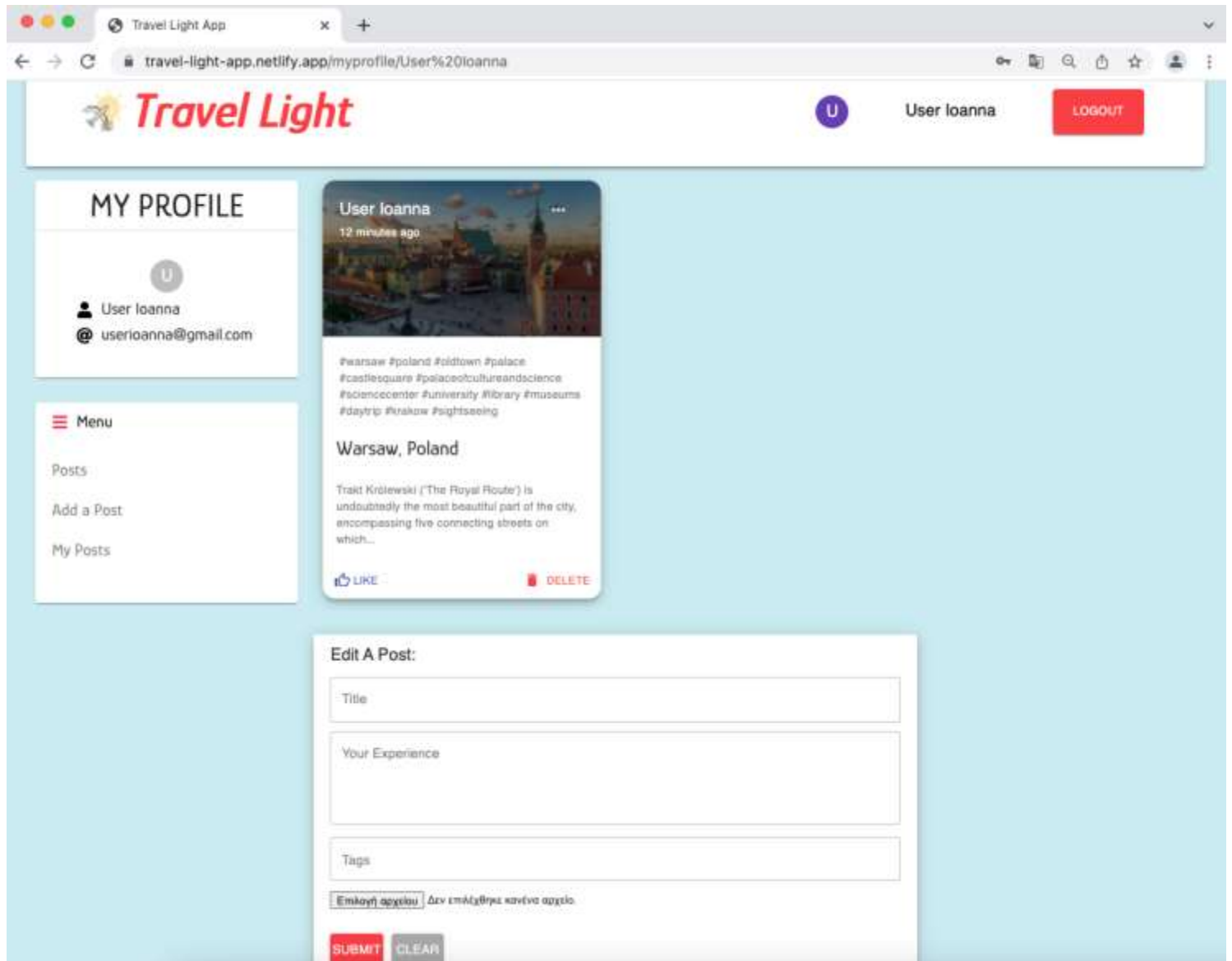
Εικόνα 3.20 Δημιουργία δημοσίευσης



3.2.1.5 Προφίλ χρήστη

Εφόσον ο χρήστης έχει λογαριασμό στην εφαρμογή, στο προφίλ του προβάλλονται όλες οι δημοσιεύσεις που έχει αναρτήσει καθώς και τα προσωπικά του στοιχεία (username, e-mail, avatar).

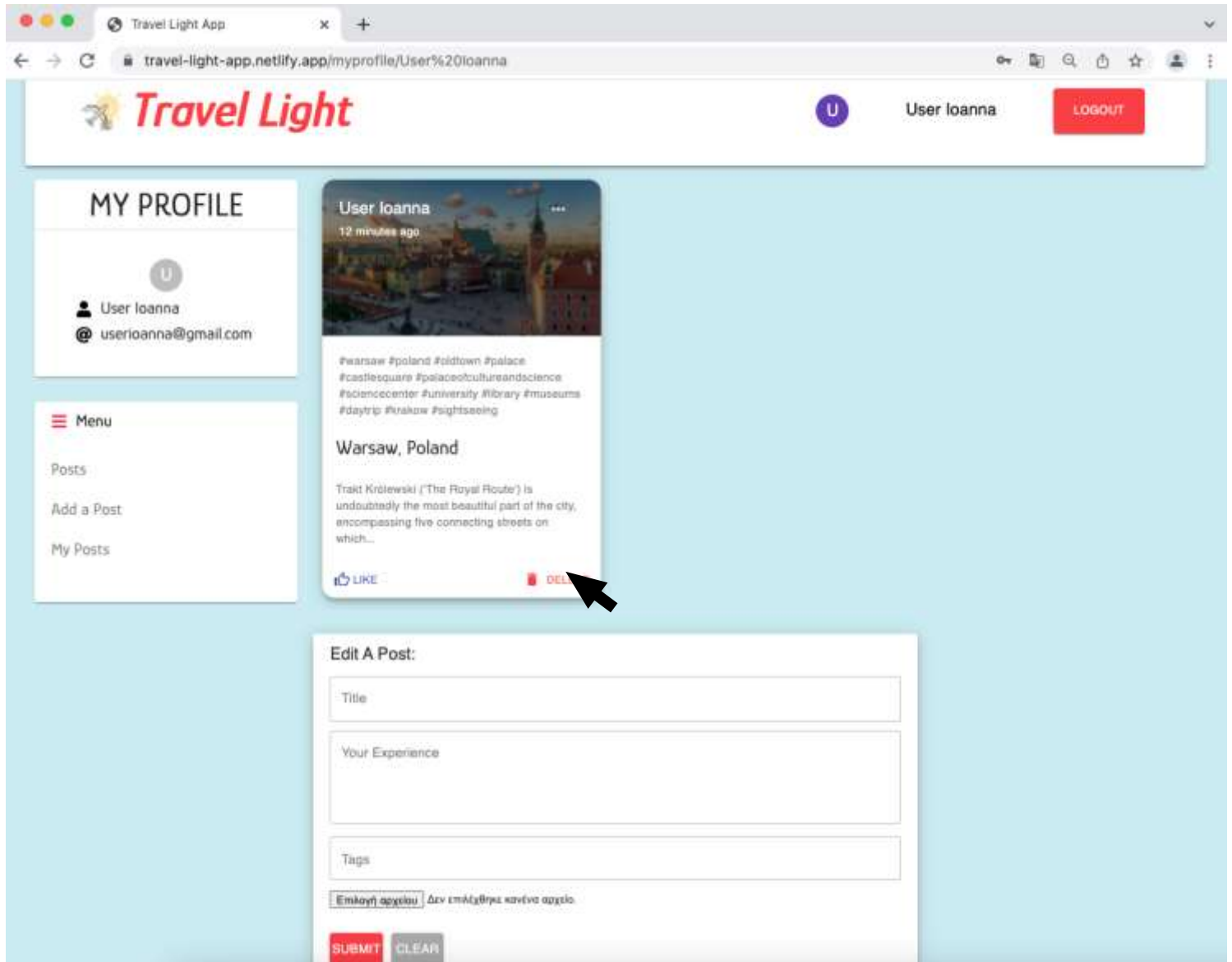
Εικόνα 3.21 Προφίλ συνδεδεμένου χρήστη



Διαγραφή μιας δημοσίευσης

Στο προφίλ του χρήστη υπάρχει η δυνατότητα να διαγράψει μία ανάρτηση του.

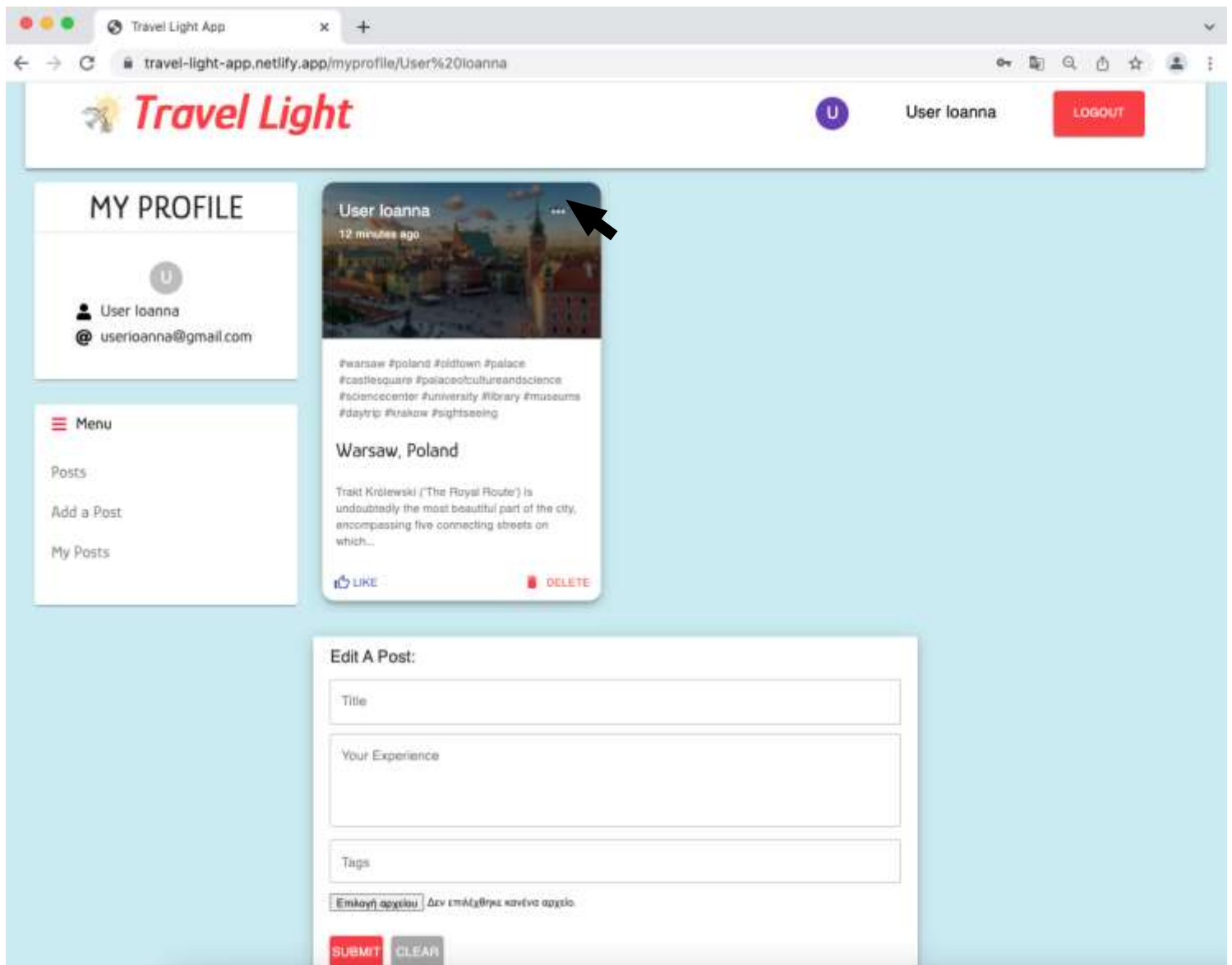
Εικόνα 3.22 Διαγραφή δημοσίευσης



Τροποποίηση μιας δημοσίευσης

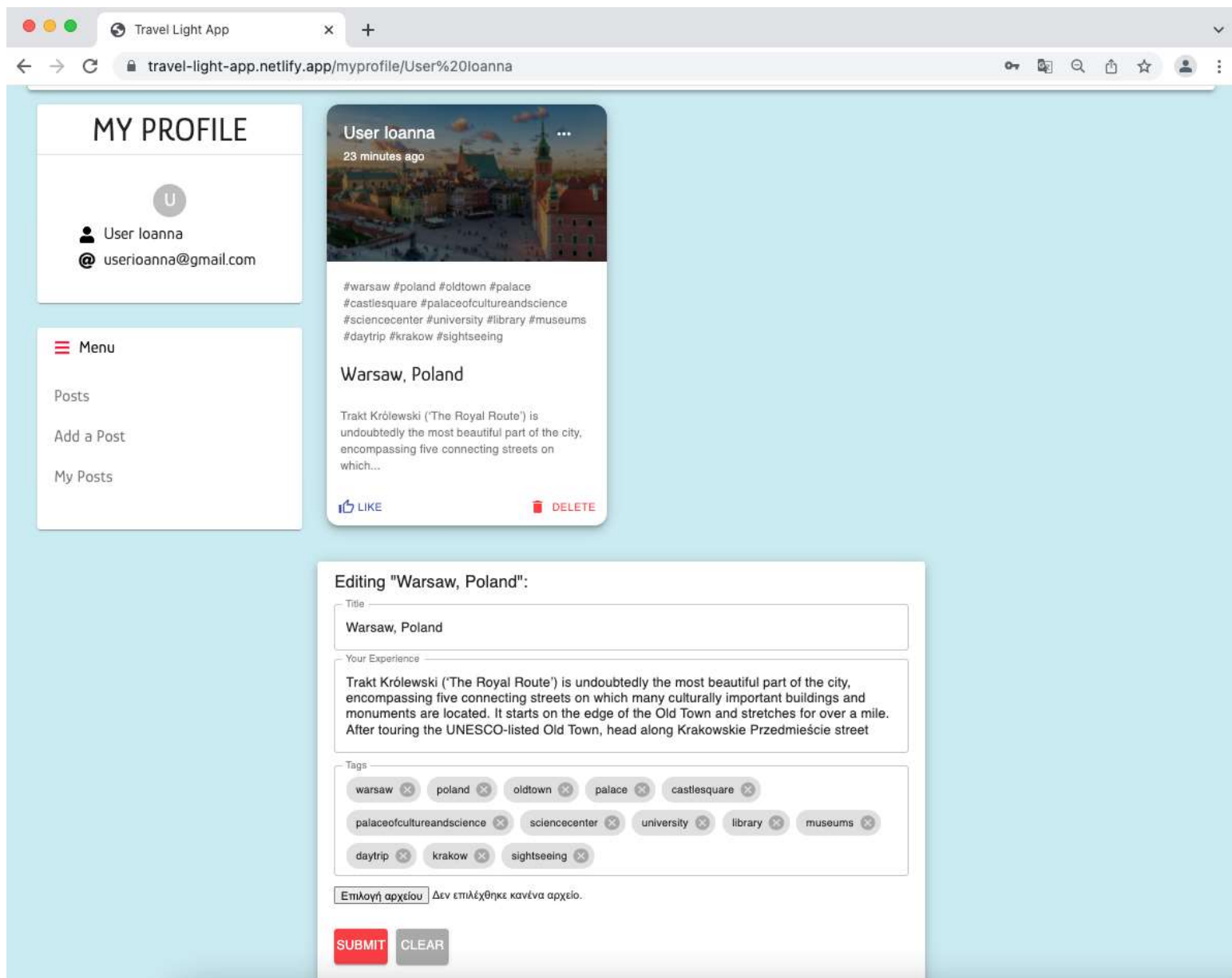
Μια σημαντική λειτουργικότητα στο προφίλ του χρήστη είναι η δυνατότητα της τροποποίησης της δημοσίευσης. Ο χρήστης επιλέγει την δημοσίευση που θέλει να τροποποιήσει και εκείνη εμφανίζεται στην φόρμα που υπάρχει κάτω από τις δημοσιεύσεις. Όταν η τροποποιημένη ανάρτηση είναι έτοιμη ο χρήστης την υποβάλει και εκείνη αυτόματα ανανεώνεται.

Εικόνα 3.23 Τροποποίηση δημοσίευσης



Αφού ο χρήστης επιλέξει τις τρεις τελείες, τα πεδία της φόρμας στο κάτω μέρος αυτοσυμπληρώνονται έτσι ώστε ο χρήστης να διορθώσει το επιθυμητό πεδίο. Στη συνέχεια πατάει το κουμπί Submit και η δημοσίευση ανανεώνεται τροποποιημένη.

Εικόνα 3.24 Διορθωμένη φόρμα



3.2.2 Δεύτερο Σενάριο

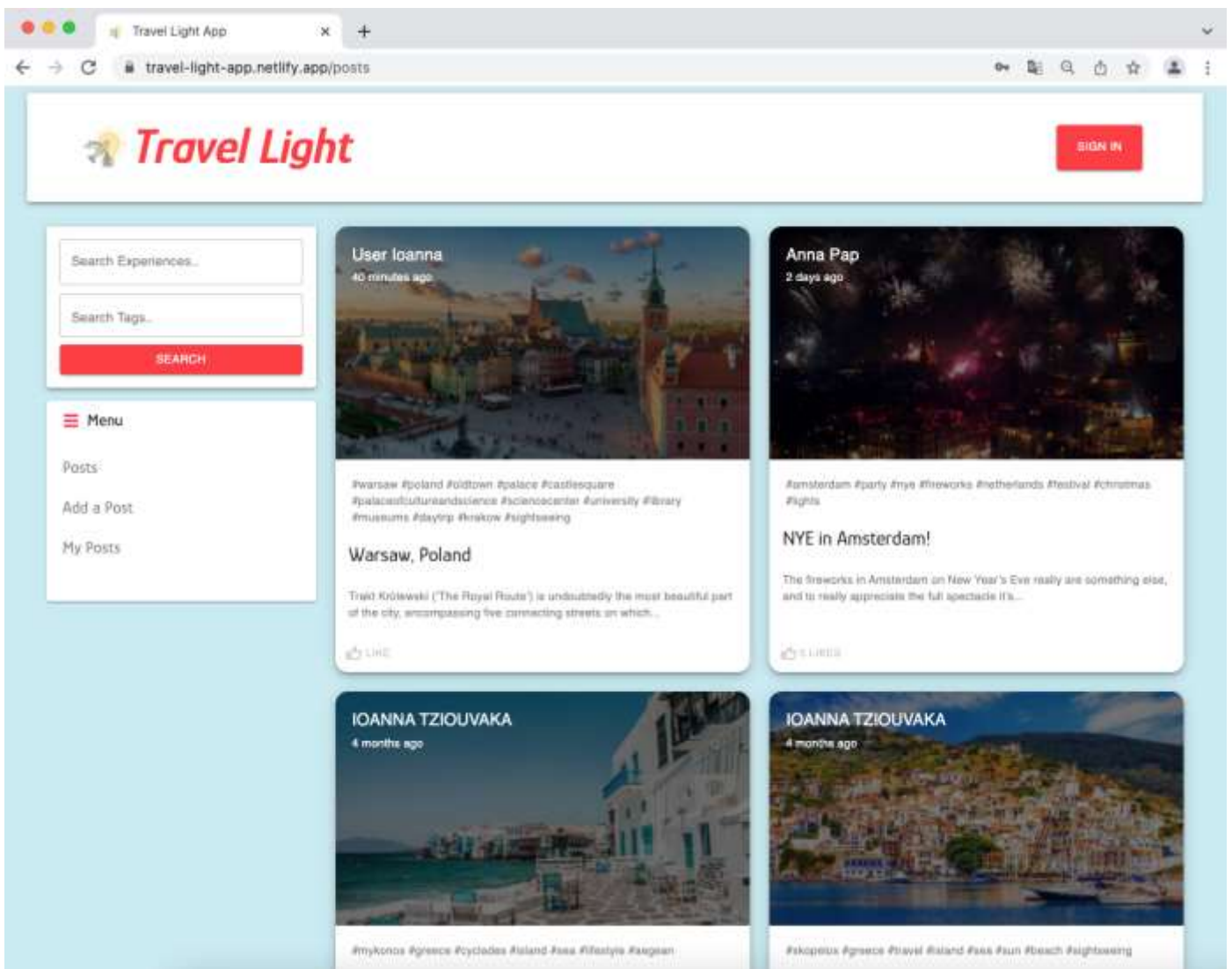
Το δεύτερο σενάριο αφορά τη περιήγηση ενός χρήστη που δεν είναι συνδεδεμένος στην εφαρμογή.

Ένας χρήστης ο οποίος δεν έχει συνδεθεί στην εφαρμογή και είναι απλώς επισκέπτης έχει περιορισμένες λειτουργικότητες στην διάθεση του.

3.2.2.1 Περιήγηση στην αρχική σελίδα

Ο επισκέπτης μπορεί να περιηγηθεί στην σελίδα των δημοσιεύσεων, βλέποντας κανονικά όλες τις δημοσιεύσεις των χρηστών και τα προφίλ του. Η λειτουργικότητα του Like είναι απενεργοποιημένη για τον ίδιο ωστόσο μπορεί να δει τον αριθμό των Likes σε κάθε δημοσίευση. Επίσης, έχει πρόσβαση και στο μενού χωρίς όμως περαιτέρω επιλογές.

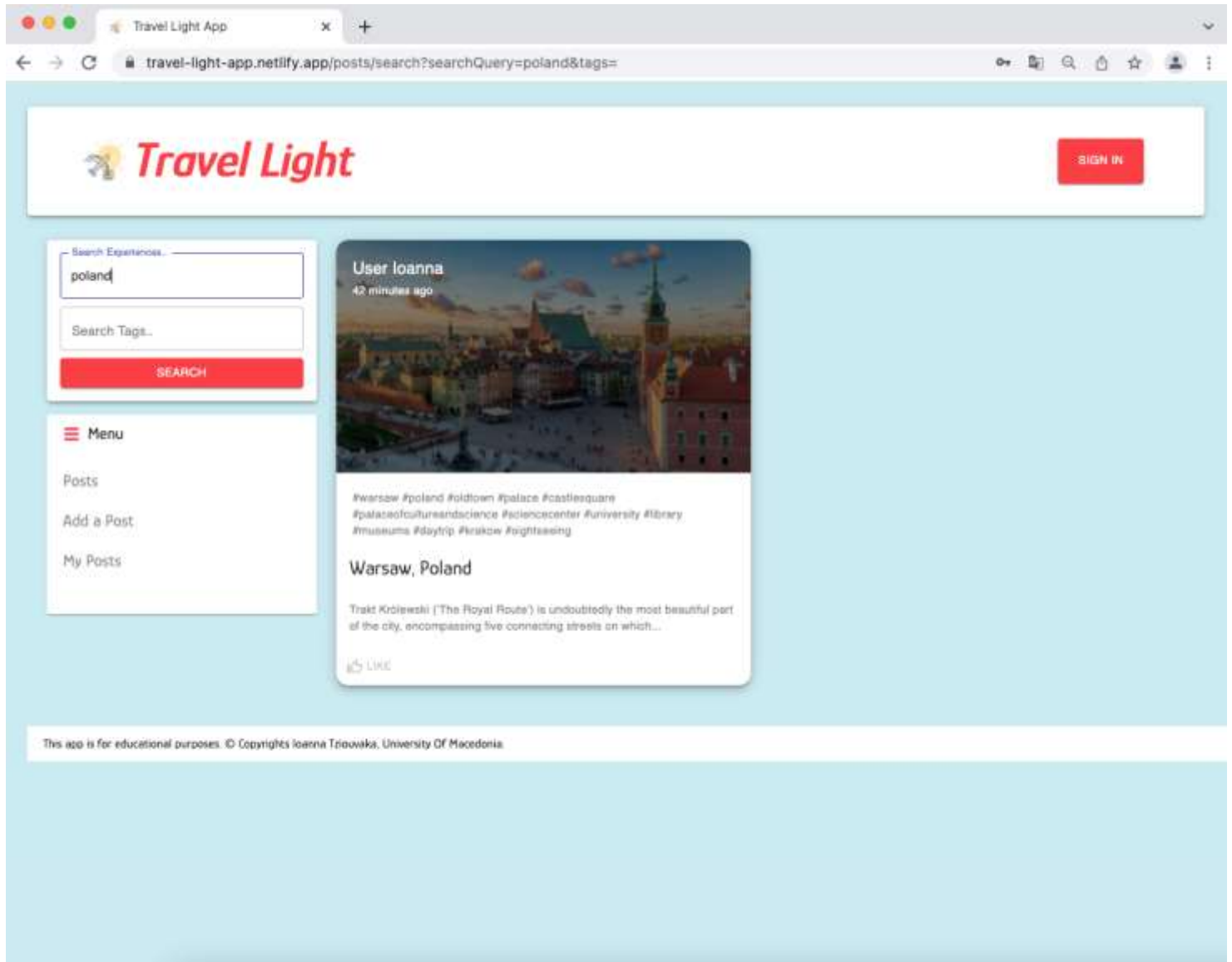
Εικόνα 3.25 Αρχική σελίδα επισκέπτη



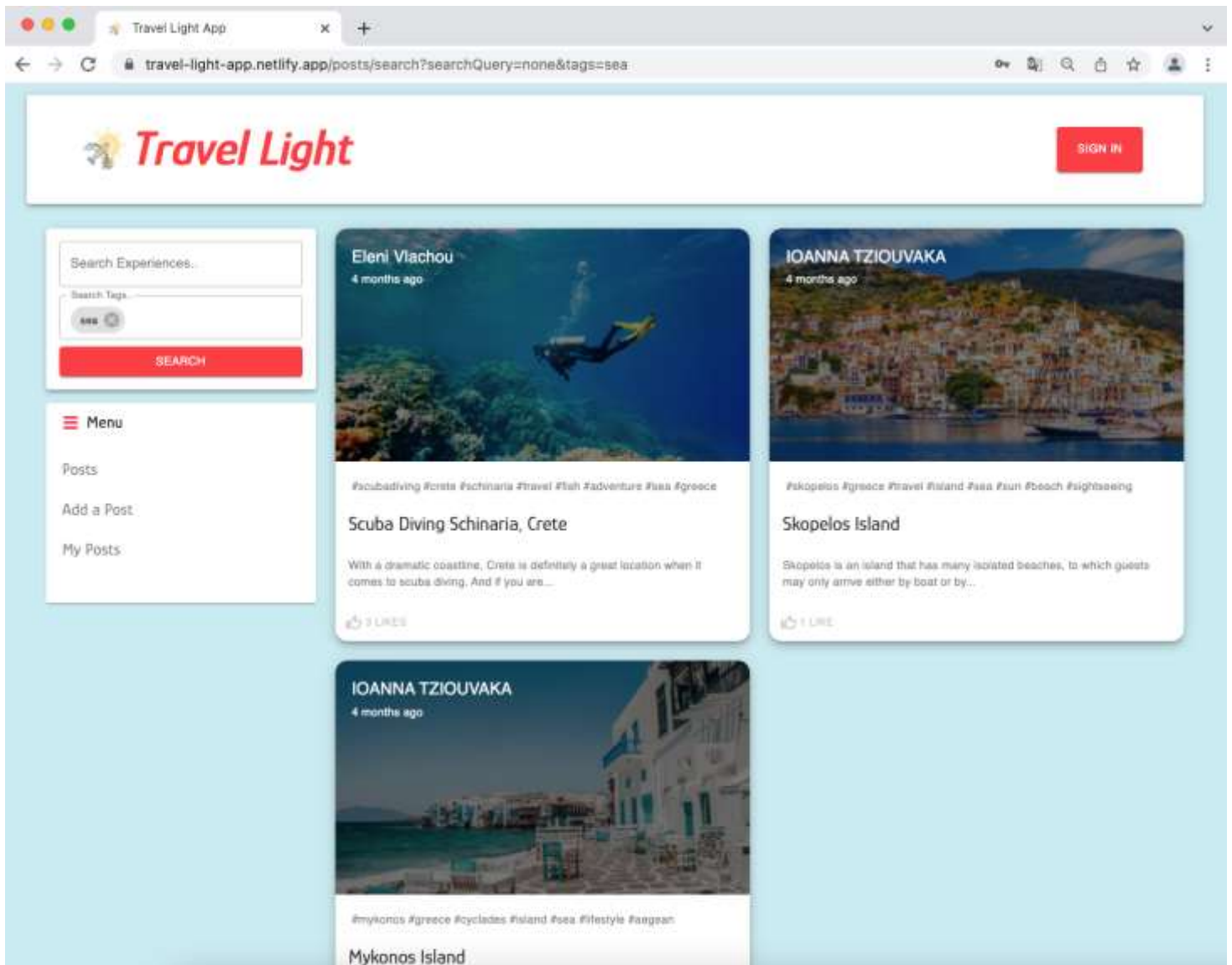
3.2.2.2 Αναζήτηση τίτλων και ετικετών

Ένας μη συνδεδεμένος χρήστης μπορεί να αναζητήσει τίτλους και ετικέτες χωρίς κανέναν περιορισμό.

Εικόνα 3.26 Αναζήτηση τίτλων επισκέπτη



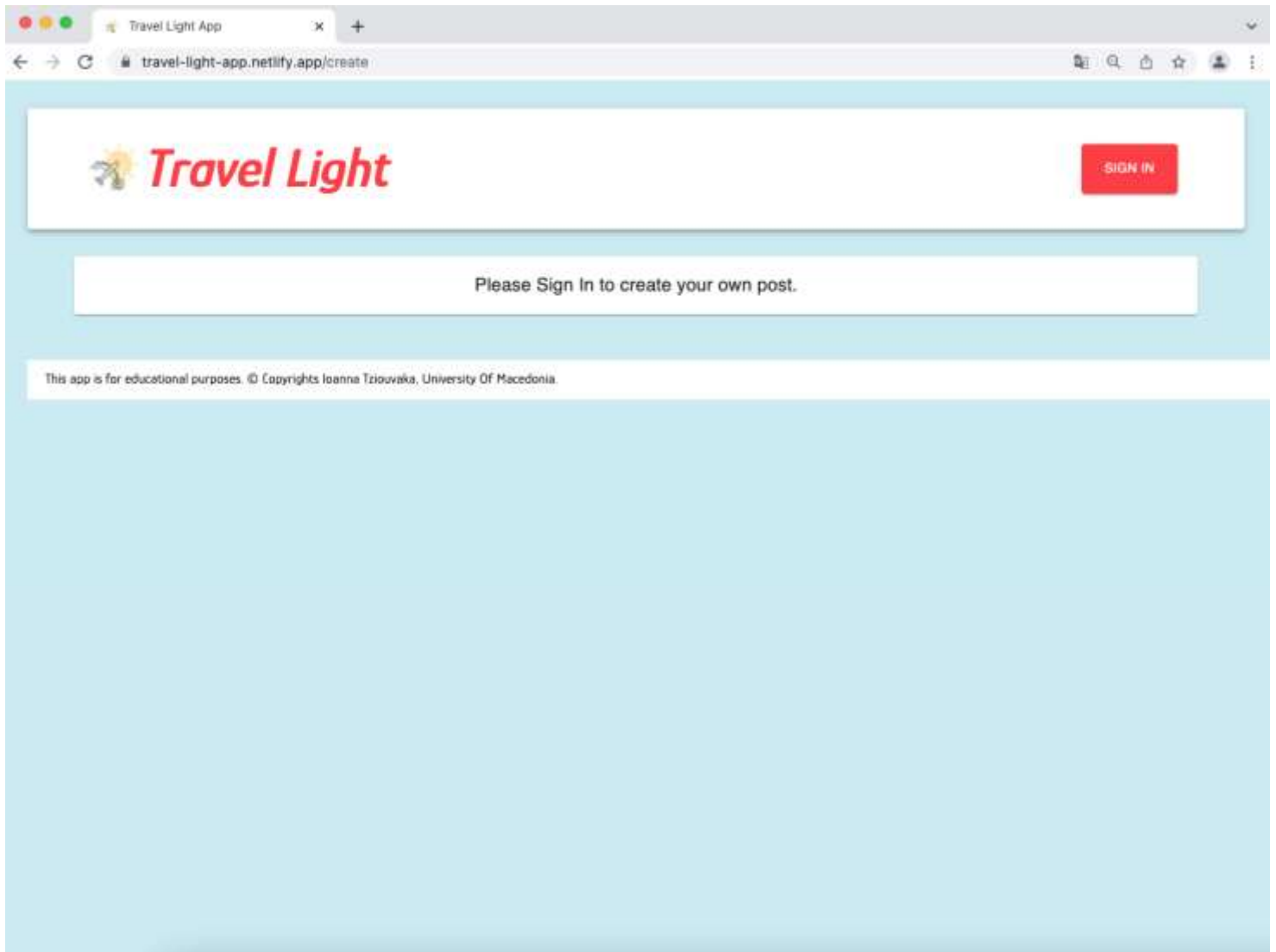
Εικόνα 3.27 Αναζήτηση ετικετών επισκέπτη



3.2.2.3 Δημιουργία δημοσίευσης

Ένας επισκέπτης της εφαρμογής δεν μπορεί να δημοσιεύσει περιεχόμενο, οπότε αν επισκεφθεί την σελίδα “Add a Post” θα του εμφανιστεί κατάλληλο μήνυμα που θα τον παροτρέπει να συνδεθεί στην εφαρμογή.

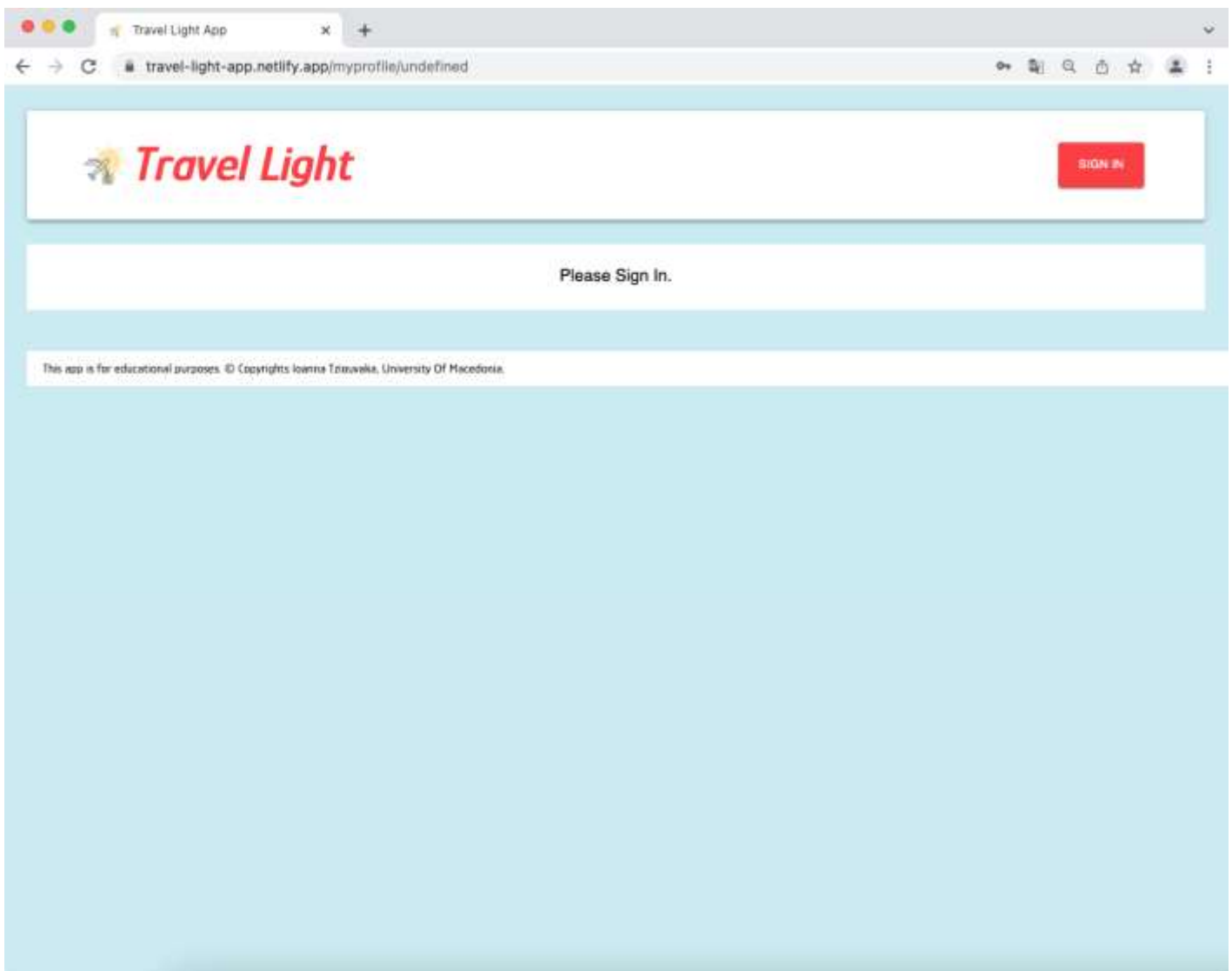
Εικόνα 3.28 Σελίδα δημιουργία δημοσίευσης επισκέπτη



3.2.2.4 Προφίλ χρήστη

Ο επισκέπτης δεν έχει προφίλ οπότε στην σελίδα “My Posts” εμφανίζεται μήνυμα που ενημερώνει τον χρήστη να συνδεθεί στην εφαρμογή.

Εικόνα 3.29 Προφίλ επισκέπτη

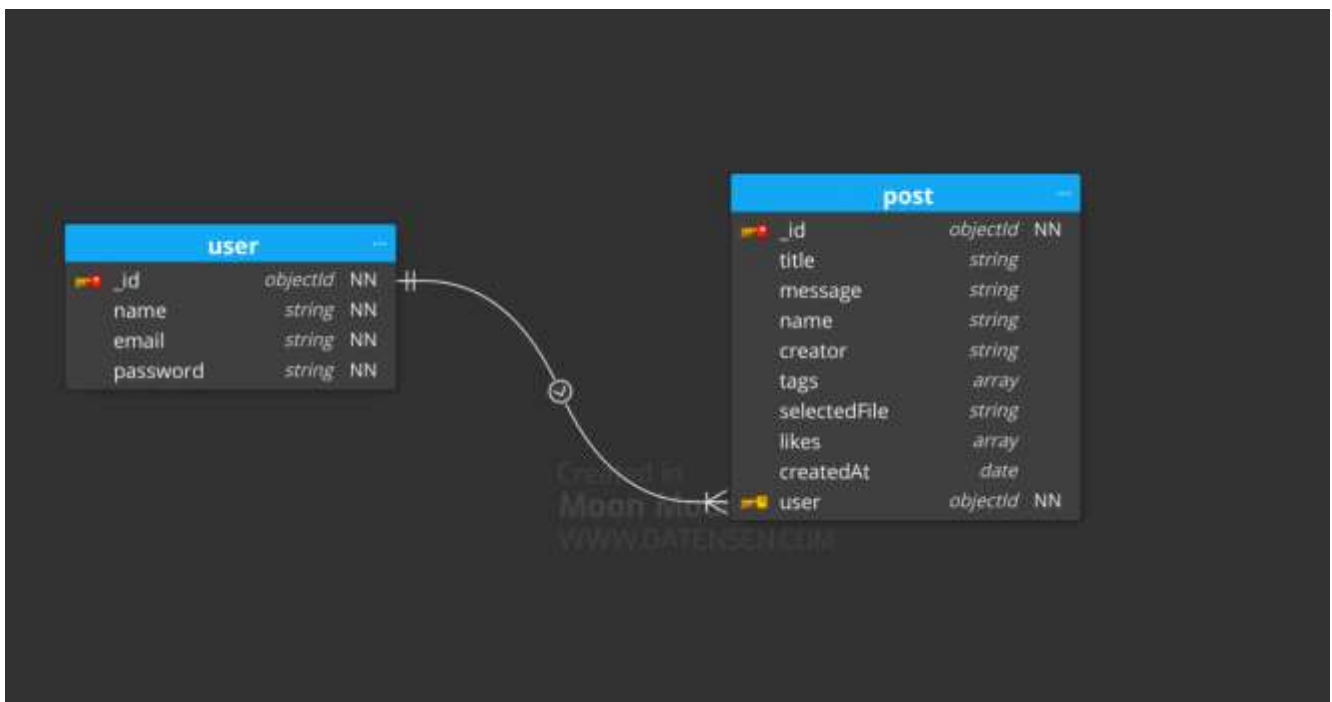


3.3 Υλοποίηση Εφαρμογής Travel Light

3.3.1 Βάση Δεδομένων Εφαρμογής

Για την δημιουργία της εφαρμογής χρησιμοποιήθηκε μια NoSQL βάση δεδομένων, το MongoDB. Το μοντέλο δεδομένων της βάσης δεδομένων επιτρέπει την αποθήκευση πιο περίπλοκων δομών, όπως πίνακες. Το MongoDB schema που δημιουργήθηκε κατά τη διάρκεια ανάπτυξης της εφαρμογής είχε ως στόχο τη σωστή αποθήκευση και διαχείριση των δεδομένων της εφαρμογής. Στην εφαρμογή δημιουργήθηκαν δύο schemas: postMessage και user. Παρακάτω ακολουθεί το ER Diagram της βάσης δεδομένων MongoDB.

Εικόνα 3.30 MongoDB ER Diagram



Η βάση δεδομένων υπάρχει σε cluster του Atlas MongoDB (<https://www.mongodb.com/atlas/database>) στο region Frankfurt (eu-central-1) και είναι προσβάσιμη μέσω ενός env αρχείου στην εφαρμογή που περιέχει το Connection URL όπως φαίνεται και στην εικόνα.

Εικόνα 3.31 Connection URL

```
CONNECTION_URL = mongodb://[username]:[password]@[hostname]:[port]/[database]
```

Στη συνέχεια ακολουθούν τα schemas.

Εικόνα 3.32 PostMessage Schema

```
import mongoose from 'mongoose';

const postSchema = mongoose.Schema({
  title: String,
  message: String,
  name: String,
  creator: String,
  tags: [String],
  selectedFile: String,
  likes: { type: [String], default: [] },
  createdAt: {
    type: Date,
    default: new Date(),
  },
});

var PostMessage = mongoose.model('PostMessage', postSchema);

export default PostMessage;
```

Εικόνα 3.33 User Schema

```
import mongoose from 'mongoose';

const userSchema = mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true },
  password: { type: String, required: true },
  id: { type: String },
});

export default mongoose.model("User", userSchema);
```

Τα δεδομένα με βάση τα schemas, αποθηκεύονται στη βάση δεδομένων Atlas όπως στις εικόνες που ακολουθούν.

Εικόνα 3.34 PostMessage Data

```
_id: ObjectId("6133efe6519b93192e22bfa7")
tags: Array
  0: "Peloponnese"
  1: "Greece"
  2: "friends"
  3: "RoadTrip"
likes: Array
  0: "101419248507845484501"
  1: "111404008320479650986"
  2: "101237171050584499155"
createdAt: 2021-09-04T22:15:02.252+00:00
title: "Road Trip"
message: "Exciting week driving all around Peloponnese!"
selectedFile: "data:image/jpeg;base64,/9j/4AAQSkZJRgABAQEAYABgAAD/2wBDAA0JCgsKCA0LCws..."
name: "IOANNA TZIOUVAKA"
creator: "111396912671832721875"
```

Εικόνα 3.35 User Data

```
_id: ObjectId("61d5b5f287439d0023b2ad6c")
email: "userioanna@gmail.com"
password: "$2a$12$8bpS5GW9u6d/k2FNB5j87.yK4F4rtJoeRh0fu0Z.mZtQxE1ll7ls6"
name: "User Ioanna"
```

3.3.2 Backend Εφαρμογής

Το backend της εφαρμογής επικοινωνεί με το frontend της, το οποίο στέλνει requests (αιτήματα). Οπότε το backend επιστρέφει τα δεδομένα στο frontend, λειτουργώντας ως restful API, σύμφωνα με το schema της βάσης δεδομένων. Παρακάτω παρουσιάζονται τα dependencies που χρησιμοποιήθηκαν για την ανάπτυξη του backend.

Εικόνα 3.36 Backend dependencies

```
"dependencies": {
  "bcryptjs": "^2.4.3",
  "body-parser": "^1.19.0",
  "cors": "^2.8.5",
  "dotenv": "^10.0.0",
  "esm": "^3.2.25",
  "express": "^4.17.1",
  "jsonwebtoken": "^8.5.1",
  "minimatch": "^3.0.4",
  "mongoose": "^5.13.2",
  "node-modules": "^1.0.1",
  "nodemon": "^2.0.12"
```

Το bcryptjs είναι μια βιβλιοθήκη που χρησιμοποιείται για κατακερματισμό κωδικών.

Το body-parser είναι μια βιβλιοθήκη που επεξεργάζεται τα δεδομένα που στέλνονται μέσω HTTP request body[23].

Το dotenv χρησιμοποιείται για να κρυφθούν σημαντικά στοιχεία, όπως κωδικοί, από τον πηγαίο κώδικα.

Το cors επιτρέπει στους servers να προσδιορίζουν ποιος θα έχει πρόσβαση και πως θα γίνεται η πρόσβαση.

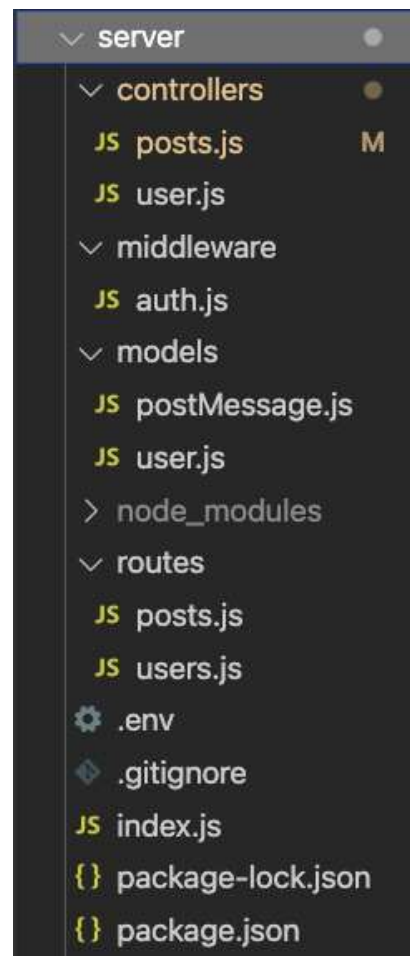
Το jsonwebtoken χρησιμοποιείται χρησιμοποιείται για ασφαλή επικοινωνία μεταξύ client-server.

Το mongoose είναι ένα εργαλείο που χρησιμοποιείται για object modeling.

Το nodemon χρησιμοποιείται για να κάνει επανεκκίνηση στον server όταν ανιχνεύει αλλαγή στον κώδικα.

Η δομή των φακέλων στον φάκελο του server είναι η ακόλουθη.

Εικόνα 3.37 Δομή φακέλων του backend



Στη συνέχεια θα αναλυθεί το περιεχόμενο του κάθε φακέλου.

Middleware

Ο φάκελος middleware περιέχει το αρχείο auth.js.

Εικόνα 3.38 Αρχείο middleware/auth.js

```
import jwt from 'jsonwebtoken';

const auth = async (req, res, next) => {
  try {
    const token = req.headers.authorization.split(" ")[1];
    const isCustomAuth = token.length < 500;

    let decodedData;

    if(token && isCustomAuth) {
      decodedData = jwt.verify(token, 'test');

      req.userId = decodedData?.id;
    } else {
      decodedData = jwt.decode(token);

      req.userId = decodedData?.sub;
    }

    next();
  } catch (error) {
    console.log(error);
  }
};

export default auth;
```

Η συνάρτηση auth χρησιμοποιείται για να ελέγχει αν τα αιτήματα (requests) είναι έγκυρα πριν από κάποιο action. Δηλαδή, πριν ο χρήστης πραγματοποιήσει ένα action στην εφαρμογή, το auth middleware ελέγχει αν είναι συνδεδεμένος σε αυτή είτε μέσω Google Login είτε με εγγραφή, και αν κληθεί η συνάρτηση next(), το action πραγματοποιείται. Το auth middleware έχει προστεθεί σε συγκεκριμένα actions στο routing της εφαρμογής όπου απαιτείται η ταυτοποίηση του χρήστη.

Routes

Το πιο σημαντικό μέρος της εφαρμογής είναι το routing, δηλαδή οι ανακατευθύνσεις σε σελίδες που υπάρχουν στην κεντρική σελίδα της εφαρμογής. Υπάρχουν δύο αρχεία στο φάκελο Routes: posts.js και user.js.

Εικόνα 3.39 Αρχείο routes/posts.js

```
import express from 'express';

import { getPostsBySearch, getPosts, getPost, createPost, updatePost,
  deletePost, likePost, getPostsByCreator, getProfilePosts } from '../controllers/posts.js';

const router = express.Router();
import auth from '../middleware/auth.js';

// localhost:5000/posts

router.get('/creator', getPostsByCreator);
router.get('/myprofile', getProfilePosts);
router.get('/search', getPostsBySearch);
router.get('/', getPosts);
router.get('/:id', getPost);

router.post('/', auth, createPost);
router.patch('/:id', auth, updatePost);
router.delete('/:id', auth, deletePost);
router.patch('/:id/likePost', auth, likePost);

export default router;
```

Στο αρχείο posts.js υπάρχουν όλα τα routes που σχετίζονται με actions που αφορούν τις δημοσιεύσεις όπως η σελίδα των δημοσιεύσεων, η σελίδα με τα αποτελέσματα αναζήτησης των δημοσιεύσεων, η σελίδα του προφίλ χρήστη με τις δημοσιεύσεις του, η σελίδα ενός χρήστη που περιέχει όλες τις αναρτήσεις του αλλά και σελίδες που πραγματοποιούν κάποιο action όπως δημιουργία post, διαγραφή post, τροποποίηση post, και like στο post. Στα routes που αφορούν τα actions υπάρχει και το auth middleware το οποίο διασφαλίζει ότι τα actions θα πραγματοποιηθούν από συνδεδεμένους χρήστες. Το κάθε route καλεί συγκεκριμένη συνάρτηση από τον φάκελο controllers και το αρχείο του posts.js, που θα αναλυθεί παρακάτω.

Εικόνα 3.40 Αρχείο routes/user.js

```
import express from 'express';
const router = express.Router();

import { signin, signup } from '../controllers/user.js';

router.post("/signin", signin);
router.post("/signup", signup);

export default router;
```

Σε αυτό το αρχείο υπάρχουν τα routes που σχετίζονται με τον χρήστη. Με την βοήθεια των συναρτήσεων των controllers, δημιουργούνται routes που διαχειρίζονται το sign in (σύνδεση) και sign up (εγγραφή) στην εφαρμογή.

Controllers

Ο φάκελος controllers περιέχει δύο αρχεία το posts.js και το user.js.

Εικόνα 3.41 Αρχείο controllers/posts.js

```
import express from 'express';
import mongoose from 'mongoose';
import PostMessage from '../models/postMessage.js';
const router = express.Router();

export const getPosts = async (req, res) => {
  const { page } = req.query;

  try {
    const LIMIT = 8;
    const startIndex = (Number(page) - 1) * LIMIT;
    const total = await PostMessage.countDocuments({});

    const posts = await PostMessage.find().sort({ _id: -1 }).limit(LIMIT).skip(startIndex);

    res.json({ data: posts, currentPage: Number(page), numberOfPages: Math.ceil(total / LIMIT)});
  } catch (error) {
    res.status(404).json({ message: error.message });
  }
};

export const getPostsBySearch = async (req, res) => {
  const { searchQuery, tags } = req.query;

  try {
    const title = new RegExp(searchQuery, 'i');

    const posts = await PostMessage.find({ $or: [ { title }, { tags: { $in: tags.split(',') } } ] });

    res.json({ data: posts });
  } catch (error) {
    res.status(404).json({ message: error.message });
  }
};

export const getPost = async (req, res) => {
  const { id } = req.params;

  try {
    const post = await PostMessage.findById(id);

    res.status(200).json(post);
  } catch (error) {
    res.status(404).json({ message: error.message });
  }
};
```

```

export const likePost = async (req, res) => {
  const { id } = req.params;

  if (!req.userId) {
    return res.json({ message: "Unauthenticated" });
  }

  if (!mongoose.Types.ObjectId.isValid(id)) return res.status(404).send(`No post with id: ${id}`);

  const post = await PostMessage.findById(id);
  const index = post.likes.findIndex((id) => id === String(req.userId));

  if (index === -1) {
    post.likes.push(req.userId);
  } else {
    post.likes = post.likes.filter((id) => id !== String(req.userId));
  }

  const updatedPost = await PostMessage.findByIdAndUpdate(id, post, { new: true });
  res.status(200).json(updatedPost);
};

export const getPostsByCreator = async (req, res) => {
  const { name } = req.query;

  try {
    const posts = await PostMessage.find({name});

    res.json({ data: posts });
  } catch (error) {
    res.status(404).json({ message: error.message });
  }
};

export const getProfilePosts = async (req, res) => {
  const { name } = req.query;

  try {
    const posts = await PostMessage.find({name});

    res.json({ data: posts });
  } catch (error) {
    res.status(404).json({ message: error.message });
  }
};

export default router;

```



```

export const createPost = async (req, res) => {
  const post = req.body;

  const newPost = new PostMessage({ ... post, creator: req.userId, createdAt: new Date().toISOString() });

  try {
    await newPost.save();

    res.status(201).json(newPost);
  } catch (error) {
    res.status(409).json({ message: error.message });
  }
};

export const updatePost = async (req, res) => {
  const { id } = req.params;
  const { title, message, creator, selectedFile, tags } = req.body;

  if (!mongoose.Types.ObjectId.isValid(id)) return res.status(404).send(`No post with id: ${id}`);

  const updatedPost = { creator, title, message, tags, selectedFile, _id: id };

  await PostMessage.findByIdAndUpdate(id, updatedPost, { new: true });

  res.json(updatedPost);
};

export const deletePost = async (req, res) => {
  const { id } = req.params;

  if (!mongoose.Types.ObjectId.isValid(id)) return res.status(404).send(`No post with id: ${id}`);
  await PostMessage.findByIdAndRemove(id);

  res.json({ message: 'Post deleted successfully.' });
};

```

Σε αυτό το αρχείο υλοποιείται το backend του κάθε action που υπάρχει στην εφαρμογή. Το κάθε action γίνεται γίνεται import και χρησιμοποιείται για το routing της εφαρμογής. Η κάθε συνάρτηση επιστρέφει ένα αίτημα του χρήστη. Επίσης, για την υλοποίηση της κάθε συνάρτησης χρησιμοποιήθηκε και το schema του PostMessage που υπάρχει στον φάκελο models.

getPosts: επιστρέφει όλες τις δημοσιεύσεις.

getPostsBySearch: αναζητά και επιστρέφει αποτελέσματα αναζήτησης του χρήστη.

getPost: αναζητά και επιστρέφει την δημοσίευση που έχει ζητήσει ο χρήστης με βάση το id της.

createPost: αποθηκεύει τα δεδομένα της δημοσίευσης που έχει δημιουργήσει ο χρήστης με βάση το schema.

updatePost: ανακτά τα δεδομένα ενός post με βάση το id του και αποθηκεύει τα νέα τροποποιημένα δεδομένα στο post με το ίδιο id.

`deletePost`: διαγράφει το post με συγκεκριμένο id που έχει επιλέξει ο χρήστης.

`likePost`: ενημερώνει τον αριθμό των likes σε κάθε post διασφαλίζοντας ότι ένας χρήστης θα προσμετράται μόνο μια φορά.

`getPostsByCreator`: Γίνεται αναζήτηση του ονόματος που έχει αιτηθεί ο χρήστης και επιστρέφει τις δημοσιεύσεις που περιέχουν ως όνομα δημιουργού το αιτούμενο.

`getProfilePosts`: πραγματοποιεί την ίδια λειτουργικότητα με την συνάρτηση `getPostsByCreator`.

Εικόνα 3.42 Αρχείο controllers/user.js

```
import bcrypt from 'bcryptjs';
import jwt from 'jsonwebtoken';
import pkg from 'minimatch';
const { match } = pkg;

import UserModel from '../models/user.js';

const secret = 'test';

export const signin = async (req, res) => {
  const { email, password } = req.body;

  try {
    const oldUser = await UserModel.findOne({ email });

    if (!oldUser) return res.status(404).json({ message: "User doesn't exist" });

    const isPasswordCorrect = await bcrypt.compare(password, oldUser.password);

    if (!isPasswordCorrect) return res.status(400).json({ message: "Invalid credentials" });

    const token = jwt.sign({ email: oldUser.email, id: oldUser._id }, secret, { expiresIn: "1h" });
    res.status(200).json({ result: oldUser, token });
  } catch (err) {
    res.status(500).json({ message: "Something went wrong" });
  }
};

export const signup = async (req, res) => {
  const { email, password, firstName, lastName } = req.body;

  try {
    const oldUser = await UserModel.findOne({ email });

    if (oldUser) return res.status(400).json({ message: "User already exists" });

    const hashedPassword = await bcrypt.hash(password, 12);

    const result = await UserModel.create({ email, password: hashedPassword, name: `${firstName} ${lastName}` });

    const token = jwt.sign({ email: result.email, id: result._id }, secret, { expiresIn: "1h" });

    res.status(201).json({ result, token });
  } catch (error) {
    res.status(500).json({ message: "Something went wrong" });
    console.log(error);
  }
};
```

Στο συγκεκριμένο αρχείο υλοποιείται η λογική του sign in (σύνδεση) και sign up (εγγραφή) της εφαρμογής. Στη συνάρτηση sign in ελέγχεται αν ο χρήστης υπάρχει στην εφαρμογή με την αναζήτηση e-mail στη βάση δεδομένων, και αν υπάρχει ελέγχεται ο κωδικός του με τη βοήθεια της βιβλιοθήκης bcrypt. Στη συνάρτηση signup ελέγχεται αν ο χρήστης έχει ήδη λογαριασμό με αυτό το e-mail και αν δεν υπάρχει λογαριασμός δημιουργείται με βάση το user schema. Ο κωδικός πρόσβασης κατακερματίζεται με τη συνδρομή της bcrypt και αποθηκεύεται μαζί με όλα τα στοιχεία στη βάση δεδομένων. Σε περίπτωση που δεν πραγματοποιηθεί κάτι σωστά και στις δύο συναρτήσεις εμφανίζεται αντίστοιχο μήνυμα.

Index.js

Εικόνα 3.43 Αρχείο server/index.js

```
import express from 'express';
import bodyParser from 'body-parser';
import mongoose from 'mongoose';
import cors from 'cors';
import dotenv from 'dotenv';
import postRoutes from './routes/posts.js';
import userRoutes from './routes/users.js';

const app = express();
dotenv.config();

app.use(bodyParser.json({ limit: "30mb", extended: true }));
app.use(bodyParser.urlencoded({ limit: "30mb", extended: true }));
app.use(cors());

app.use('/posts', postRoutes);
app.use("/user", userRoutes);

app.get('/', (req, res) => {
  res.send('message to API');
});

const PORT = process.env.PORT || 5000;

mongoose.connect(process.env.CONNECTION_URL, { useNewUrlParser: true, useUnifiedTopology: true })
  .then(() => app.listen(PORT, () => console.log(`Server running on port: ${PORT}`)))
  .catch((error) => console.log(error.message));

mongoose.set('useFindAndModify', false);
```

Σε αυτό το αρχείο γίνονται όλα τα imports από βιβλιοθήκες που θα χρειαστούν για τη δημιουργία του backend καθώς και δηλώνονται τα routes που θα υπάρχουν και η σύνδεση με τη βάση δεδομένων.

3.3.3 Frontend Εφαρμογής

Το frontend της εφαρμογής είναι το σημείο διεπαφής με τον χρήστη. Μέσω του frontend, ο χρήστης περιηγείται στην εφαρμογή, στέλνοντας αιτήματα στο backend της, με διάφορα actions που πραγματοποιεί. Παρακάτω παρουσιάζονται τα dependencies που χρησιμοποιήθηκαν για την ανάπτυξη του frontend.

Εικόνα 3.44 Frontend Dependencies

```
"dependencies": {
  "@fortawesome/free-solid-svg-icons": "^5.15.4",
  "@fortawesome/react-fontawesome": "^0.1.15",
  "@material-ui/core": "^4.12.1",
  "@material-ui/icons": "^4.11.2",
  "@material-ui/lab": "^4.0.0-alpha.60",
  "@testing-library/jest-dom": "^5.14.1",
  "@testing-library/react": "^11.2.7",
  "@testing-library/user-event": "^12.8.3",
  "axios": "^0.21.1",
  "json-stringify-safe": "^5.0.1",
  "jwt-decode": "^3.1.2",
  "material-ui-chip-input": "^1.1.0",
  "moment": "^2.29.1",
  "react": "^17.0.2",
  "react-dom": "^17.0.2",
  "react-file-base64": "^1.0.3",
  "react-google-login": "^5.2.2",
  "react-redux": "^7.2.4",
  "react-router-dom": "^5.2.1",
  "react-scripts": "4.0.3",
  "react-tinder-card": "^1.4.0",
  "redux": "^4.1.0",
  "redux-thunk": "^2.3.0",
  "web-vitals": "^1.1.2"
```

Το `fortawesome/free-solid-svg-icons` και `fortawesome/react-fontawesome` αφορούν βιβλιοθήκες του Font Awesome που περιέχει διάφορα icons.

Οι βιβλιοθήκες Material-UI(material-ui/core, material-ui/icons, material-ui/lab, material-ui-chip-input) παρέχουν έτοιμα προς χρήση components και icons.

Οι βιβλιοθήκες testing-library της React χρησιμοποιούνται για το testing των react components.

Η βιβλιοθήκη Axios χρησιμοποιείται για τα asynchronous HTTP Requests σε REST endpoints και πραγματοποιεί CRUD (Create, Read, Update, Delete) λειτουργίες.

Η βιβλιοθήκη moment της react παρέχει jsx tags που βοηθούν στη συγγραφή του κώδικα.

Η βιβλιοθήκη jwt-decode χρησιμοποιείται για την αποκωδικοποίηση JWTs token.

Η βιβλιοθήκη json-stringify-safe χρησιμοποιείται για parse JSON αρχείων.

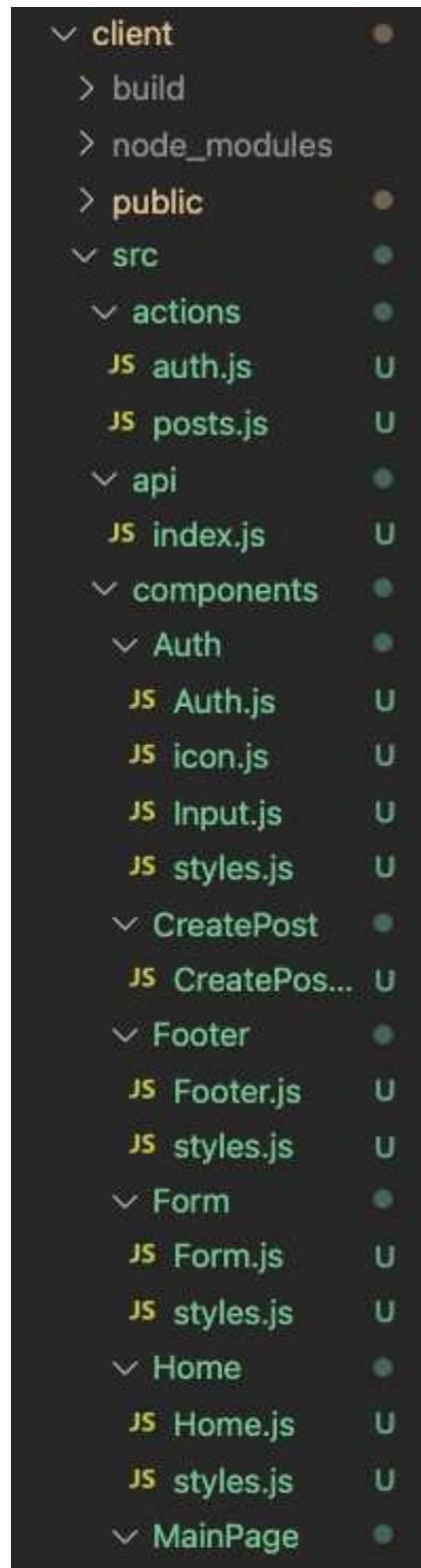
Η βιβλιοθήκη react-google-login χρησιμοποιείται για το Google Login στην εφαρμογή.

Η redux είναι μια βιβλιοθήκη που χρησιμοποιείται για την διαχείριση και τη συγκέντρωση της κατάστασης της εφαρμογής. Μαζί με την react αποτελούν ιδανική λύση για την δημιουργία διεπαφών χρήστη.

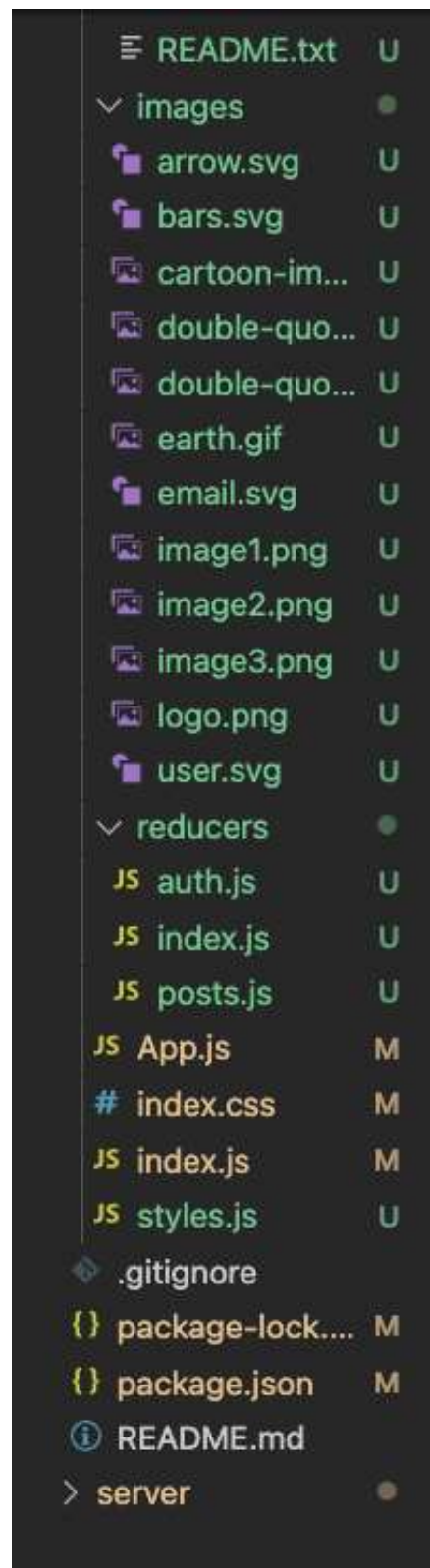
Τα web-vitals είναι ένα σετ από μετρικές του user experience (UX) και χρήσιμο εργαλείο για το SEO της εφαρμογής.

Η δομή των φακέλων είναι η ακόλουθη και θα παρουσιαστούν όλα τα αρχεία λεπτομερώς.

Εικόνα 3.45 Δομή φακέλων frontend



▼	MainPage	●
	JS MainPage.js	U
	JS styles.js	U
▼	MyPosts	●
	⚙ MyPosts.jsx	U
	JS styles.js	U
▼	Navbar	●
	JS Navbar.js	U
	JS styles.js	U
▼	PostCreator	●
	JS PostCreat...	U
	JS styles.js	U
▼	PostDetails	●
	⚙ PostDetail...	U
	JS styles.js	U
▼	Posts	●
	▼ Post	●
	JS Post.js	U
	JS styles.js	U
	JS Posts.js	U
	JS styles.js	U
	⚙ Pagination.jsx	U
	JS styles.js	U
▼	constants	●
	JS actionTypes...	U
▼	fonts / Glory	●
	> static	●
	🔴 Glory-Italic-...	U



Index.js

Εικόνα 3.46 Αρχείο src/index.js

```
import React from 'react';
import ReactDOM from 'react-dom';
import { Provider } from 'react-redux';
import { createStore, applyMiddleware, compose } from 'redux';
import thunk from 'redux-thunk';
import reducers from './reducers';
import './fonts/Glory/static/Glory-SemiBoldItalic.ttf';
import './fonts/Glory/static/Glory-Light.ttf';
import './fonts/Glory/static/Glory-Regular.ttf';

import App from './App';
import './index.css';

const store = createStore(reducers, compose(applyMiddleware(thunk)));

ReactDOM.render(
  <Provider store={store}>
    <App />
  </Provider>,
  document.getElementById('root')
);
```

Στο αρχείο index.js γίνονται import όλες οι βιβλιοθήκες που θα χρειαστούν για την ανάπτυξη του frontend καθώς και το κύριο component στο οποίο θα δηλώνονται όλα τα children components. Το App.js είναι το component που καλείται και φορτώνεται στο html element 'root'. Το 'root' είναι το html element που υπάρχει στο index.html στον φάκελο public όπου και φορτώνει όλα τα components. Επίσης, γίνεται αρχικοποίηση του redux.

Index.css

Στο αρχείο index.css δηλώθηκαν όλες οι γραμματοσειρές που χρησιμοποιήθηκαν στην εφαρμογή. Ακολουθεί παράδειγμα.

Εικόνα 3.47 Αρχείο src/index.css

```
@font-face {  
  font-family: "GloryRegular";  
  src: local("GloryRegular"),  
       url("./fonts/Glory/static/Glory-Regular.ttf") format("truetype");  
}
```

App.js

Εικόνα 3.48 Αρχείο src/App.js

```
import React from 'react';
import { Container } from '@material-ui/core';

import Navbar from './components/Navbar/Navbar';
import Home from './components/Home/Home';
import { BrowserRouter, Switch, Route, Redirect } from 'react-router-dom';
import Auth from './components/Auth/Auth';
import PostDetails from './components/PostDetails/PostDetails';
import CreatePost from './components/CreatePost/CreatePost';
import MainPage from './components/MainPage/MainPage';
import Footer from './components/Footer/Footer';
import MyPosts from './components/MyPosts/MyPosts';
import PostCreator from './components/PostCreator/PostCreator';

const App = () => {
  const user = JSON.parse(localStorage.getItem('profile'));

  return(
    <BrowserRouter>
      <Container maxWidth="xl">
        <Navbar />
        <Switch>
          <Route path="/" exact component={MainPage} />
          <Route path="/posts" exact component={Home} />
          <Route path="/posts/search" exact component={Home} />
          <Route path="/posts/:id" exact component={PostDetails} />
          <Route path="/auth" exact component={() => (!user ? <Auth /> : <Redirect to="/" />)} />
          <Route path="/create" exact component={CreatePost} />
          <Route path="/creators/:name" exact component={PostCreator} />
          <Route path="/myprofile/:name" exact component={MyPosts} />
        </Switch>
        <Footer />
      </Container>
    </BrowserRouter>
  );
};

export default App;
```

Στο root component App.js γίνονται import όλα τα children components και δομούνται με τέτοιο τρόπο ώστε να αναπαρασταθεί η αρχική σελίδα. Το κάθε route αντιστοιχείται και με το component που θα εμφανίζει η σελίδα.

Actions

Ο φάκελος actions περιλαμβάνει όλα τα redux actions που υπάρχουν στην εφαρμογή και χωρίζονται σε δύο αρχεία τα posts.js και auth.js.

Εικόνα 3.49 Αρχείο actions/posts.js

```
import * as api from '../api/index.js';
import { FETCH_ALL, CREATE, UPDATE, DELETE, LIKE, FETCH_BY_SEARCH, START_LOADING,
  END_LOADING, FETCH_POST, FETCH_BY_CREATOR, FETCH_PROFILE_POSTS } from '../constants/actionTypes';

export const getPosts = (page) => async (dispatch) => {

  try {
    dispatch({ type: START_LOADING });
    const { data } = await api.fetchPosts(page);

    dispatch({ type: FETCH_ALL, payload: data });
    dispatch({ type: END_LOADING });
  } catch (error) {
    console.log(error);
  }
};

export const createPost = (post, history) => async (dispatch) => {
  try {
    dispatch({ type: START_LOADING });
    const { data } = await api.createPost(post);

    dispatch({ type: CREATE, payload: data });
    history.push(`/posts/${data._id}`);

  } catch (error) {
    console.log(error);
  }
};

export const updatePost = (id, post) => async (dispatch) => {
  try {
    const { data } = await api.updatePost(id, post);

    dispatch({ type: UPDATE, payload: data });
  } catch (error) {
    console.log(error);
  }
};
```

```

export const getPost= (id) => async (dispatch) => {
  try {
    dispatch({ type: START_LOADING });
    const { data } = await api.fetchPost(id);

    dispatch({ type: FETCH_POST, payload: data });
  } catch (error) {
    console.log(error);
  }
};

export const getPostsByCreator = (name) => async (dispatch) => {
  try {
    dispatch({ type: START_LOADING });
    const { data: { data } } = await api.fetchPostsByCreator(name);

    dispatch({ type: FETCH_BY_CREATOR, payload: { data } });
    dispatch({ type: END_LOADING });
  } catch (error) {
    console.log(error);
  }
};

export const getProfilePosts = (name) => async (dispatch) => {
  try {
    dispatch({ type: START_LOADING });
    const { data: { data } } = await api.fetchPostsByCreator(name);

    dispatch({ type: FETCH_PROFILE_POSTS, payload: { data } });
    dispatch({ type: END_LOADING });
  } catch (error) {
    console.log(error);
  }
};

```



```

export const deletePost = (id) => async (dispatch) => {
  try {
    await api.deletePost(id);

    dispatch({ type: DELETE, payload: id });
  } catch (error) {
    console.log(error);
  }
};

export const likePost = (id) => async (dispatch) => {
  const user = JSON.parse(localStorage.getItem('profile'));

  try {
    const { data } = await api.likePost(id, user?.token);

    dispatch({ type: LIKE, payload: data });
  } catch (error) {
    console.log(error);
  }
};

export const getPostsBySearch = (searchQuery) => async (dispatch) => {
  try {
    dispatch({ type: START_LOADING });
    const { data: { data } } = await api.fetchPostsBySearch(searchQuery);

    dispatch({ type: FETCH_BY_SEARCH, payload: data });
    dispatch({ type: END_LOADING });
  } catch (error) {
    console.log(error);
  }
};

```

Στο αρχείο posts.js υπάρχουν όλα τα redux actions της εφαρμογής που αφορούν τις δημοσιεύσεις. Με τη βοήθεια του redux thunk δηλώνεται μια επιπλέον συνάρτηση που διαχειρίζεται τα asynchronous data.

Εικόνα 3.50 Αρχείο actions/auth.js

```
import * as api from '../api/index.js';
import { AUTH } from '../constants/actionTypes';

export const signin = (formData, router) => async (dispatch) => {
  try {
    const { data } = await api.signIn(formData);

    dispatch({ type: AUTH, data });

    router.push('/');
  } catch (error) {
    console.log(error);
  }
};

export const signup = (formData, router) => async (dispatch) => {
  try {
    const { data } = await api.signUp(formData);

    dispatch({ type: AUTH, data });

    router.push('/');
  } catch (error) {
    console.log(error);
  }
};
```

Σε αυτό το αρχείο δηλώνονται όλα τα redux actions που αφορούν την αυθεντικοποίηση του χρήστη.

API

Στον φάκελο υπάρχει το αρχείο index.js.

Εικόνα 3.51 Αρχείο api/index.js

```
import axios from 'axios';

const API = axios.create({ baseURL: 'https://travel-light-app.herokuapp.com/' });

API.interceptors.request.use((req) => {
  if(localStorage.getItem('profile')) {
    req.headers.Authorization = `Bearer ${JSON.parse(localStorage.getItem('profile')).token}`;
  }
  return req;
});

export const fetchPosts = (page) => API.get(`/posts?page=${page}`);
export const fetchPostsBySearch = (searchQuery) =>
  API.get(`/posts/search?searchQuery=${searchQuery.search || 'none'}&tags=${searchQuery.tags}`);
export const createPost = (newPost) => API.post('/posts', newPost);
export const updatePost = (id, updatedPost) => API.patch(`/posts/${id}`, updatedPost);
export const deletePost = (id) => API.delete(`/posts/${id}`);
export const likePost = (id) => API.patch(`/posts/${id}/likePost`);
export const fetchPost = (id) => API.get(`/posts/${id}`);
export const fetchPostsByCreator = (name) => API.get(`/posts/creator?name=${name}`);
export const getProfilePosts = (name) => API.get(`/myprofile/creator?name=${name}`);

export const signIn = (formData) => API.post('/user/signin', formData);
export const signUp = (formData) => API.post('/user/signup', formData);
```

Σε αυτό το αρχείο δηλώνεται το baseURL της εφαρμογής. Ανάλογα τα actions που πραγματοποιεί ο χρήστης υπάρχουν και τα αντίστοιχα URLs, τα οποία γίνονται export από κάθε συνάρτηση. Η κάθε συνάρτηση δέχεται συγκεκριμένα ορίσματα προκειμένου να κάνει export το σωστό URL.

Constants

Αυτός ο φάκελος περιέχει το αρχείο actionTypes.js.

Εικόνα 3.52 Αρχείο constants/actionTypes.js

```
export const CREATE = 'CREATE';
export const UPDATE = 'UPDATE';
export const DELETE = 'DELETE';

export const FETCH_ALL = 'FETCH_ALL';
export const LIKE = 'LIKE';
export const AUTH = 'AUTH';

export const LOGOUT = 'LOGOUT';
export const FETCH_BY_SEARCH = 'FETCH_BY_SEARCH';

export const START_LOADING = 'START_LOADING';
export const END_LOADING = 'END_LOADING';

export const FETCH_POST = 'FETCH_POST';
export const FETCH_BY_CREATOR = 'FETCH_BY_CREATOR';

export const FETCH_PROFILE_POSTS = 'FETCH_PROFILE_POSTS';
```

Αυτό το αρχείο δημιουργήθηκε για την ευκολία του προγραμματισμού δηλώνοντας το κάθε string ως variable.

Reducers

Στον φάκελο reducers υπάρχουν τα redux reducers σε ξεχωριστά αρχεία όπου το κάθε reducer έχει τη δική του λειτουργικότητα. Το redux store που υπάρχει στο αρχείο index.js στον φάκελο src παίρνει μόνο έναν reducer γι' αυτό γίνεται combine όλων των reducers ώστε να περαστούν. Τα αρχεία που περιέχονται στον φάκελο είναι: index.js, posts.js, auth.js.

Εικόνα 3.53 Αρχείο reducers/index.js

```
import { combineReducers } from 'redux';

import posts from './posts';
import auth from './auth';
const reducers = combineReducers({ posts, auth })

export default reducers;
```

Στο αρχείο index.js γίνεται το combine των reducers ώστε να γίνουν export ως ένα reducer και να χρησιμοποιηθεί στο redux store του αρχείου index.js του φάκελου src.

Εικόνα 3.54 Αρχείο reducers/auth.js

```
import * as actionTypes from '../constants/actionTypes';

const authReducer = (state = { authData: null }, action) => {
  switch (action.type) {
    case actionTypes.AUTH:
      localStorage.setItem('profile', JSON.stringify({ ...action?.data }));

      return { ...state, authData: action.data, loading: false, errors: null };
    case actionTypes.LOGOUT:
      localStorage.clear();

      return { ...state, authData: null, loading: false, errors: null };
    default:
      return state;
  }
};

export default authReducer;
```

Η λειτουργικότητα του auth reducer είναι να δέχεται δύο arguments όπου το ένα είναι το τρέχον state και ένα action και επιστρέφει ένα νέο state. Τα state που μπορεί να επιστρέψει είναι η σύνδεση και η αποσύνδεση στην εφαρμογή.

Εικόνα 3.55 Αρχείο reducers/posts.js

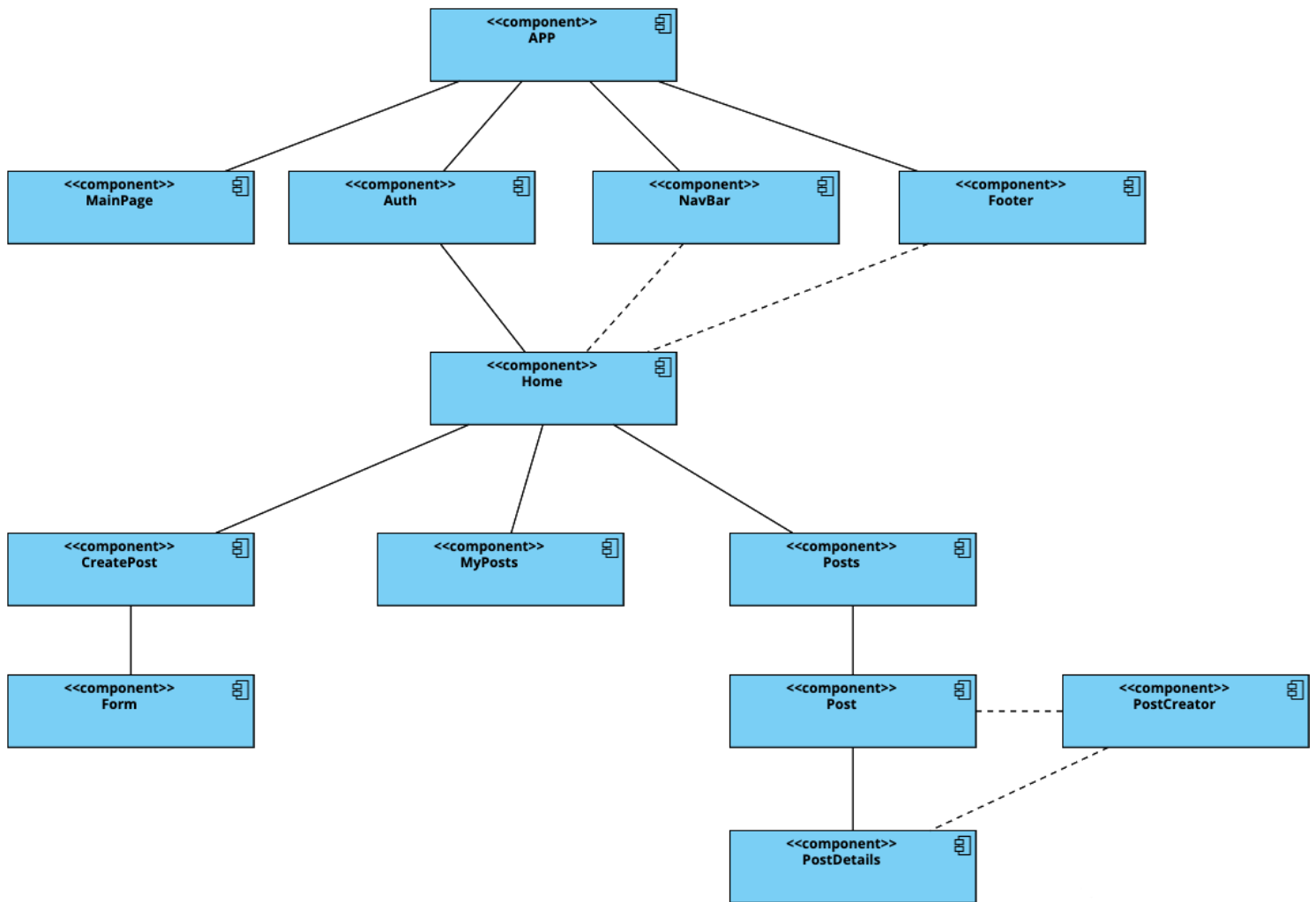
```
import { FETCH_ALL, CREATE, UPDATE, DELETE, LIKE, FETCH_BY_SEARCH, START_LOADING,
  END_LOADING, FETCH_POST, FETCH_BY_CREATOR, FETCH_PROFILE_POSTS } from '../constants/actionTypes';
export default (state = { isLoading: true, posts: [] }, action) => {
  switch (action.type) {
    case START_LOADING:
      return { ...state, isLoading: true };
    case END_LOADING:
      return { ...state, isLoading: false };
    case FETCH_ALL:
      return {
        ...state,
        posts: action.payload.data,
        currentPage: action.payload.currentPage,
        numberOfPages: action.payload.numberOfPages,
      };
    case FETCH_POST:
      return {
        ...state, post: action.payload
      };
    case FETCH_BY_CREATOR:
      return { ...state, posts: action.payload.data };
    case FETCH_PROFILE_POSTS:
      return { ...state, posts: action.payload.data };
    case CREATE:
      return { ...state, posts: action.payload };
    case UPDATE:
      return { ...state, posts: state.posts.map((post) => (post._id === action.payload._id ? action.payload : post)) };
    case LIKE:
      return { ...state, posts: state.posts.map((post) => post._id === action.payload._id ? action.payload : post) };
    case DELETE:
      return { ...state, posts: state.posts.filter((post) => post._id !== action.payload) };
    case FETCH_BY_SEARCH:
      return {
        ...state, posts: action.payload
      };
    default:
      return state;
  }
};
```

Σε αυτό το αρχείο υπάρχουν όλα τα πιθανά actions του χρήστη στην εφαρμογή καθώς και ένα τρέχον state που βρίσκεται ο χρήστης. Με βάση όλα τα πιθανά actions του χρήστη θα επιστρέφει και ανάλογο state.

Components

Τα children components της εφαρμογής είναι κομμάτια διεπαφής χρήστη που συνθέτουν την εφαρμογή. Παρακάτω ακολουθεί διάγραμμα αναπαράστασης των components που χρησιμοποιήθηκαν για την υλοποίηση της εφαρμογής, το οποίο είναι ο σκελετός του UI.

Εικόνα 3.56 Διάγραμμα αναπαράστασης components της εφαρμογής



Κάθε φάκελος είναι ένα component και περιέχει το js αρχείο με την υλοποίηση του καθώς και ένα άλλο js αρχείο που περιέχει το css του. Κάθε component της βιβλιοθήκης material-UI στο αρχείο js έχει μια κλάση, η οποία χρησιμεύει για το css του κάθε component. Ακολουθούν τα components που δημιουργήθηκαν και μια σύντομη περιγραφή τους.

AUTH

Εικόνα 3.57 Αρχείο components/auth/auth.js

```
return (
  <Container component="main" maxWidth="xs">
    <Paper className={classes.paper} elevation={3}>
      <Avatar className={classes.avatar}>
        <LockOutlinedIcon />
      </Avatar>
      <Typography variant="h5">{isSignup ? 'Sign Up' : 'Sign In'}</Typography>
      <form className={classes.form} onSubmit={handleSubmit}>
        <Grid container spacing={2}>
          {
            isSignup && {
              <Input name="firstName" label="First Name" handleChange={handleChange} autoFocus half />
              <Input name="lastName" label="Last Name" handleChange={handleChange} half />
            }
          }
          <Input name="email" label="Email Address" handleChange={handleChange} type="email" />
          <Input name="password" label="Password" handleChange={handleChange} type={showPassword ? "text" : "password"}
            handleShowPassword={handleShowPassword}/>
          { isSignup && <Input name="confirmPassword" label="Repeat Password" handleChange={handleChange} type="password"/> }
        </Grid>

        <Button type="submit" style={{ width: '100%' }} variant="contained" color="primary" className={classes.submit}>
          { isSignup ? 'Sign Up' : 'Sign In' }
        </Button>

        <GoogleLogin
          clientId="783802952863-7vlf5va0namq85enn8upupccvfie2acl.apps.googleusercontent.com"
          render={({renderProps}) => (
            <Button className={classes.googleButton}
              color="primary"
              style={{ width: '100%' }}
              onClick={renderProps.onClick}
              disabled={renderProps.disabled}
              startIcon={<Icon />}
              variant="contained"
            />
          )}
        />
      </form>
    </Paper>
  </Container>
)
```



```

<GoogleLogin
  clientId="783802952863-7vlf5va0namq85enn8upupccvfie2acl.apps.googleusercontent.com"
  render={({renderProps}) => (
    <Button className={classes.googleButton}
      color="primary"
      style={{ width: '100%' }}
      onClick={renderProps.onClick}
      disabled={renderProps.disabled}
      startIcon={<Icon />}
      variant="contained"
    >
      Google Sign In
    </Button>
  )}
  onSuccess={googleSuccess}
  onFailure={googleError}
  cookiePolicy="single_host_origin"
/>

<Grid container justifyContent="flex-end">
  <Grid item>
    <Button onClick={switchMode}>
      { isSignup ? "Already have an account? Sign In" : "Don't have an account? Sign Up" }
    </Button>
  </Grid>
</Grid>
</form>
</Paper>
</Container>
);

export default SignUp;

```

Σε αυτό το component υλοποιήθηκε η σύνδεση με Google Login, και με λογαριασμό της εφαρμογής. Οπότε, όταν ένας χρήστης θελήσει να συνδεθεί στην εφαρμογή, το component που περιέχει την υλοποίηση είναι το Auth.

CREATEPOST

Εικόνα 3.58 Αρχείο components/createpost/createpost.js

```
import React, { useState } from 'react';
import Form from '../Form/Form';
import { Container } from '@material-ui/core';

const CreatePost = () => {
  const [currentId, setCurrentId] = useState(0);

  return (
    <Container>
      <Form currentId={currentId} setCurrentId={setCurrentId} />
    </Container>
  );
};

export default CreatePost;
```

Αυτό το component εμφανίζεται κατά την δημιουργία ενός νέου post, και προγραμματιστικά καλεί ένα άλλο component που περιέχει την φόρμα συμπλήρωσης πεδίων.

FOOTER

Εικόνα 3.59 Αρχείο components/footer/footer.js

```
import React from 'react';
import useStyles from './styles';
import { Typography } from '@material-ui/core';

const Footer = () => {

  const classes = useStyles();

  return (
    <Typography component="h6" className={classes.footer} style={{ width: '100%' }}>
      This app is for educational purposes. &copy; Copyrights Ioanna Tziouvaka, University Of Macedonia.
    </Typography>
  );
};

export default Footer;
```

Το component Footer εμφανίζεται στο τέλος της κάθε σελίδας και εμφανίζει ένα μήνυμα.

FORM

Εικόνα 3.60 Αρχείο components/form/form.js

```
return {
  <Container>
    <div className={classes.menuLink}>
      <img className={classes.arrow} src={arrow} alt="left arrow" />
      <Typography className={classes.postsLink} component={Link} to="/posts" variant="h3">Back to Posts</Typography>
    </div>

    <div className={classes.section}>
      <img className={classes.cartoonImage} src={cartoon} alt="sharing" />
      <Typography variant="body2" color="textSecondary" component="p" className={classes.paragraph}>
        Tell people a few things about your upcoming post!<br /><br />
        You can detail a beautiful city, or describe your unique experience when skydiving.
        Tell people about your Sunday walk by your favorite river.
        Show them your favorite spots that are hidden gems.
        You have the chance to use hashtags to assist users who search posts like yours.<br /><br />
        Make sure to share special tips to help other users!
      </Typography>
    </div>

    <Paper className={classes.paper} elevation={6}>
      <form autoComplete="off" noValidate className={` ${classes.root} ${classes.form}`} onSubmit={handleSubmit}>
        <Typography variant="h6">{currentId ? 'Editing "${post?.title}"' : 'Creating an Experience'}:</Typography>
        <TextField name="title" variant="outlined" label="Title" style={{ width: '100%' }}
          value={postData.title} onChange={(e) => setPostData({ ...postData, title: e.target.value })} />
        <TextField name="message" variant="outlined" label="Your Experience" style={{ width: '100%' }}
          multiline rows={4} value={postData.message}
          onChange={(e) => setPostData({ ...postData, message: e.target.value })} />
        <div style={{ padding: '5px 0', width: '99%' }}>
          <ChipInput
            name="tags"
            variant="outlined"
            label="Tags"
            style={{ width: '100%' }}
            value={postData.tags}
            onAdd={(chip) => handleAddChip(chip)}
            onDelete={(chip) => handleDeleteChip(chip)}
          />
        </div>
      </form>
    </Paper>
  </Container>
}
```

```

<Paper className={classes.paper} elevation={6}>
  <form autoComplete="off" noValidate className={` ${classes.root} ${classes.form}`} onSubmit={handleSubmit}>
    <Typography variant="h6">{currentId ? `Editing "${post?.title}"` : 'Creating an Experience'}</Typography>
    <TextField name="title" variant="outlined" label="Title" style={{ width: '100%' }}
      value={postData.title} onChange={(e) => setPostData({ ...postData, title: e.target.value })} />
    <TextField name="message" variant="outlined" label="Your Experience" style={{ width: '100%' }}
      multiline rows={4} value={postData.message}
      onChange={(e) => setPostData({ ...postData, message: e.target.value })} />
    <div style={{ padding: '5px 0', width: '99%' }}>
      <ChipInput
        name="tags"
        variant="outlined"
        label="Tags"
        style={{ width: '100%' }}
        value={postData.tags}
        onAdd={(chip) => handleAddChip(chip)}
        onDelete={(chip) => handleDeleteChip(chip)}
      />
    </div>
    <div className={classes.fileInput}><FileBase type="file" multiple={false}
      onDone={({ base64 }) => setPostData({ ...postData, selectedFile: base64 })} /></div>
    <div className={classes.buttons}>
      <Button className={classes.buttonSubmit} variant="contained" color="primary" size="large" type="submit"
        style={{ width: '100%' }}>Submit</Button>
      <Button className={classes.buttonClear} variant="contained" color="secondary" size="small" onClick={clear}
        style={{ width: '100%' }}>Clear</Button>
    </div>
  </form>
</Paper>
</Container>
);

export default Form;

```

Αυτό το component υλοποιεί την φόρμα συμπλήρωσης πεδίων και χρησιμοποιείται για την σελίδα Add a Post.

HOME

Εικόνα 3.61 Αρχείο components/home/home.js

```
return (  
  <Grow in>  
    <Container maxWidth="xl">  
      <Grid container justifyContent="space-between" alignItems="stretch" spacing={3} className={classes.gridContainer}>  
        <Grid item xs={12} sm={6} md={3}>  
          <AppBar className={classes.appBarSearch} position="static" color="primary">  
            <TextField  
              name="search"  
              variant="outlined"  
              label="Search Experiences..  
              onKeyDown = {handleKeyPress}  
              style={{ width: '100%' }}  
              value={search}  
              onChange={(e) => setSearch(e.target.value)}  
              className={classes.field}  
            />  
            <ChipInput  
              style={{ margin: '10px 0'}}  
              value={tags}  
              onAdd={(chip) => handleAdd(chip)}  
              onDelete={(chip) => handleDelete(chip)}  
              label="Search Tags..  
              variant="outlined"  
            />  
            <Button onClick={searchPost} className={classes.searchButton} variant="contained" color="primary">Search</Button>  
          </AppBar>  
          <Paper className={classes.paper}>  
            <div className={classes.menu}>  
              <img className={classes.bars} src={bars} alt="menu bars" />  
              <Typography className={classes.menu2} variant="body2" color="textSecondary" component="h2">Menu</Typography>  
            </div>  
            <div className={classes.menuOptions}>  
              <Typography className={classes.add} variant="body2" color="textSecondary" component="h2"  
                onClick={openLink3} style={{cursor: 'pointer'}}> Posts</Typography>  
              <Typography className={classes.add} variant="body2" color="textSecondary" component="h2"  
                onClick={openLink} style={{cursor: 'pointer'}}> Add a Post</Typography>  
              <Typography className={classes.add} variant="body2" color="textSecondary" component="h2"  
                onClick={openLink2} style={{cursor: 'pointer'}}> My Posts</Typography>  
            </div>  
          </Paper>  
        </Grid item>  
      </Grid>  
    </Container>  
  </Grow in>  
)
```



```

    <Button onClick={searchPost} className={classes.searchButton} variant="contained" color="primary">Search</Button>
  </AppBar>
  <Paper className={classes.paper}>
    <div className={classes.menu}>
      <img className={classes.bars} src={bars} alt="menu bars" />
      <Typography className={classes.menu2} variant="body2" color="textSecondary" component="h2">Menu</Typography>
    </div>
    <div className={classes.menuOptions}>
      <Typography className={classes.add} variant="body2" color="textSecondary" component="h2"
        onClick={openLink3} style={{cursor: 'pointer'}}> Posts</Typography>
      <Typography className={classes.add} variant="body2" color="textSecondary" component="h2"
        onClick={openLink} style={{cursor: 'pointer'}}> Add a Post</Typography>
      <Typography className={classes.add} variant="body2" color="textSecondary" component="h2"
        onClick={openLink2} style={{cursor: 'pointer'}}> My Posts</Typography>
    </div>
  </Paper>
</Grid>
<Grid item xs={12} sm={6} md={9}>
  <Posts setCurrentId={setCurrentId} code={1}/>
  {{!searchQuery || !tags.length}} {{ {
    <Paper elevation={6} className={classes.pagination}>
      <Pagination page={page} />
    </Paper>
  }}
</Grid>
</Grid>
</Container>
</Grow>
);
};

export default Home;

```

Αυτό το component χρησιμοποιείται για την σελίδα των δημοσιεύσεων. Σε αυτό το αρχείο καλούνται άλλα components όπως το Pagination για την σελιδοποίηση των δημοσιεύσεων και το Posts για την δομή των δημοσιεύσεων.

MAINPAGE

Εικόνα 3.62 Αρχείο components/mainpage/mainpage.js

```
import { Typography, Container, Divider } from '@material-ui/core';
import useStyles from './styles';
import earth from '../../../images/earth.gif';
import image1 from '../../../images/image1.png';
import image2 from '../../../images/image2.png';
import image3 from '../../../images/image3.png';
import quotes from '../../../images/double-quotes.png';
import quotes1 from '../../../images/double-quotes1.png';

const MainPage = () => {
  const classes = useStyles();

  return (
    <Container className={classes.body}>
      <Typography variant="body2" color="textSecondary" component="h1" className={classes.title}>Welcome</Typography>
      <Typography variant="body2" color="textSecondary" component="h1" className={classes.title2}>Please Sign in to explore new adventures</Typography>
      <div className={classes.earth}></div>
      <img className={classes.image} src={earth} alt="planet earth" />
      <Divider className={classes.divide} />
      <Typography variant="body2" color="textSecondary" component="h1" className={classes.question}>What is Travel Light ?</Typography>
      <div className={classes.section}>
        <img className={classes.imageCircle} src={image1} alt="sharing" />
        <img className={classes.imageCircle} src={image2} alt="exploring" />
        <img className={classes.imageCircle} src={image3} alt="planning" />
      </div>
      <Divider className={classes.divide} />
      <div className={classes.quote}>
        <img className={classes.quotes} src={quotes} alt="double-quotes" />
        <Typography variant="body2" color="textSecondary" component="h1" className={classes.quote1}>Experience, Travel - These are as education in themselves</Typography>
        <Typography variant="body2" color="textSecondary" component="h2" className={classes.quote2}>EURIPIDES</Typography>
        <img className={classes.quotes1} src={quotes1} alt="double-quotes" />
      </div>
    </Container>
  );
};

export default MainPage;
```

Το component Main Page αναπαριστά την αρχική σελίδα της εφαρμογής όπου γίνεται η περιγραφή εκείνης, πριν γίνει η σύνδεση ή η εγγραφή του χρήστη στην εφαρμογή.

MYPOSTS

Εικόνα 3.63 Αρχείο components/myposts/myposts.jsx

```
return(  
  <Grow in>  
    {user?.result ? (  
      <Container maxWidth="xl">  
        <Grid container justifyContent="space-between" style={{alignItems: 'flex-start', flexWrap: 'unset'}} spacing={3}>  
          <div className={classes.sectionMenu}>  
            <Paper className={classes.paperProfile}>  
              <Typography className={classes.profile} component="h2">MY PROFILE</Typography>  
              <Divider />  
              <div className={classes.Myprofile}>  
                <Avatar className={classes.purple} alt={user?.result.name} src={user?.result.imageUrl}>  
                  {user?.result.name.charAt(0)} </Avatar>  
                <div className={classes.name}>  
                  <img className={classes.userIcon} src={userName} alt="User Name" />  
                  <Typography className={classes.userName} variant="h6"> {user?.result.name} </Typography>  
                </div>  
                <div className={classes.mail}>  
                  <img className={classes.mailIcon} src={email} alt="email" />  
                  <Typography className={classes.email} variant="h6"> {user?.result.email} </Typography>  
                </div>  
              </div>  
            </Paper>  
            <Paper className={classes.paper}>  
              <div className={classes.menu}>  
                <img className={classes.bars} src={bars} alt="menu bars" />  
                <Typography className={classes.menu2} variant="body2" color="textSecondary" component="h2">Menu</Typography>  
              </div>  
              <div className={classes.menuOptions}>  
                <Typography className={classes.add} variant="body2" color="textSecondary" component="h2" onClick={openLink3} style={{cursor: 'pointer'}}> Posts</Typography>  
                <Typography className={classes.add} variant="body2" color="textSecondary" component="h2" onClick={openLink} style={{cursor: 'pointer'}}> Add a Post</Typography>  
                <Typography className={classes.add} variant="body2" color="textSecondary" component="h2" onClick={openLink2} style={{cursor: 'pointer'}}> My Posts</Typography>  
              </div>  
            </Paper>  
          </div>  
        </Grid>  
      ) : (  
        <div>  
          <h1>No user found</h1>  
        </div>  
      )  
    )  
  )  
)
```

```

<Paper className={classes.edit} elevation={6}>
  <form autoComplete="off" noValidate className={` ${classes.root} ${classes.form}`} onSubmit={handleSubmit}>
    <Typography variant="h6">{currentId ? 'Editing "${post?.title}"' : 'Edit A Post'}</Typography>
    <TextField name="title" variant="outlined" label="Title" style={{ width: '100%', marginTop: '10px' }}
      value={postData.title} onChange={(e) => setPostData({ ...postData, title: e.target.value })} />
    <TextField name="message" variant="outlined" label="Your Experience" style={{ width: '100%', marginTop: '10px' }}
      multiline rows={4} value={postData.message} onChange={(e) => setPostData({ ...postData, message: e.target.value })} />
    <div style={{ padding: '5px 0', width: '100%', marginBottom: '10px' }}>
      <ChipInput
        name="tags"
        variant="outlined"
        label="Tags"
        style={{ width: '100%', marginTop: '10px' }}
        value={postData.tags}
        onAdd={(chip) => handleAddChip(chip)}
        onDelete={(chip) => handleDeleteChip(chip)}
      />
    </div>
    <div className={classes.fileInput}><FileBase type="file" multiple={false}
      onDone={({ base64 }) => setPostData({ ...postData, selectedFile: base64 })} /></div>
    <div className={classes.buttons}>
      <Button className={classes.buttonSubmit} variant="contained" color="primary"
        size="large" type="submit" style={{ width: '100%' }}>Submit</Button>
      <Button className={classes.buttonClear} variant="contained" color="secondary"
        size="large" onClick={clear} style={{ width: '100%' }}>Clear</Button>
    </div>
  </form>
</Paper>
</Container>
) : (
  <Typography className={classes.message} variant="h6">Please Sign In.</Typography>
)
</Grow>
};
};
export default MyPosts;

```

Το components My Posts αναπαριστά τη σελίδα του προφίλ συνδεδεμένου χρήστη. Σε αυτό το αρχείο καλούνται κι άλλα components όπως το Post που ορίζει τη δομή και την εμφάνιση της κάθε δημοσίευσης της εφαρμογής.

NAVBAR

Εικόνα 3.64 Αρχείο components/navbar/navbar.js

```
const classes = useStyles();
const [user, setUser] = useState(JSON.parse(localStorage.getItem('profile')));
const dispatch = useDispatch();
const history = useHistory();
const location = useLocation();

const logout = () => {
  dispatch({ type: LOGOUT });

  history.push('/auth');

  setUser(null);
};

useEffect(() => {
  const token = user?.token;

  if (token) {
    const decodedToken = decode(token);

    if (decodedToken.exp < 1000 < new Date().getTime()) logout();
  }

  setUser(JSON.parse(localStorage.getItem('profile')));
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [location]);

return (
  <AppBar className={classes.appBar} position="static" color="inherit">
    <div className={classes.brandContainer}>
      <img className={classes.image} src={logo} alt="travel light" height="60" />
      <Typography component={Link} to="/posts" className={classes.heading} variant="h2" align="center">Travel Light</Typography>
    </div>
    <Toolbar className={classes.toolbar}>
      {user?.result ? (
        <div className={classes.profile}>
          <Avatar className={classes.purple} alt={user?.result.name} src={user?.result.imageUrl} /> {user?.result.name.charAt(0)} </Avatar>
          <Typography className={classes.userName} variant="h6">{user?.result.name}</Typography>
          <Button variant="contained" className={classes.logout} color="secondary" onClick={logout}>Logout</Button>
        </div>
      ) : (
        <Button component={Link} to="/auth" variant="contained" color="primary" className={classes.button}>Sign In</Button>
      )}
    </Toolbar>
  </AppBar>
);
export default Navbar;
```

Αυτό το component αναπαριστά το header της εφαρμογής που εμφανίζεται το logo και το κουμπί σύνδεσης/αποσύνδεσης.

POSTCREATOR

Εικόνα 3.65 Αρχείο components/postcreator/postcreator.js

```
if (!posts.length && !isLoading) return 'No posts';

const openLink = (e) => {
  history.push('/create');
}

const openLink3 = (e) => {
  history.push('/posts');
}

const openLink2 = (e) => {
  history.push(`/myprofile/${user?.result.name}`);
}

return (
  <div>
    <Typography variant="h2" className={classes.creator}>{name}</Typography>
    <div style={{ margin: '10px 0 50px 0' }} />
    <Grid container justifyContent="space-between" style={{ alignItems: 'flex-start' }} spacing={3} className={classes.gridContainer}>
      <Paper className={classes.paper}>
        <div className={classes.menu}>
          <img className={classes.bars} src={bars} alt="menu bars" />
          <Typography className={classes.menu2} variant="body2" color="textSecondary" component="h2">Menu</Typography>
        </div>
        <div className={classes.menuOptions}>
          <Typography className={classes.add} variant="body2" color="textSecondary" component="h2" onClick={openLink3} style={{ cursor: 'pointer' }}> Posts</Typography>
          <Typography className={classes.add} variant="body2" color="textSecondary" component="h2" onClick={openLink} style={{ cursor: 'pointer' }}> Add a Post</Typography>
          <Typography className={classes.add} variant="body2" color="textSecondary" component="h2" onClick={openLink2} style={{ cursor: 'pointer' }}> My Posts</Typography>
        </div>
      </Paper>

      {isLoading ? <CircularProgress style={{ color: '#a5a5af', width: '100px', height: '100px', position: 'relative', right: '30%' }} /> : (
        <Grid container alignItems="stretch" spacing={3}>
          {posts?.map((post) => (
            <Grid key={post._id} item xs={12} sm={12} md={6} lg={3}>
              <Post post={post} />
            </Grid>
          ))}
        </Grid>
      )}
    </div>
  );
}

export default PostCreator;
```

Σε αυτό το αρχείο γίνεται η υλοποίηση του προφίλ ενός χρήστη που έχει δημοσιεύσει στην εφαρμογή.

POSTDETAILS

Εικόνα 3.66 Αρχείο components/postdetails/postdetails.js

```
return (
  <Paper className={classes.paper} elevation={6}>
    <div className={classes.card}>
      <div className={classes.section}>
        <div className={classes.menuLink}>
          <img className={classes.arrow} src={arrow} alt="left arrow" />
          <Typography className={classes.postsLink} component={Link} to="/posts" variant="h3">Back to Posts</Typography>
        </div>
        <Typography className={classes.title} variant="h3" component="h2">{post.title}</Typography>
        <Typography className={classes.message} gutterBottom variant="body1" component="p">{post.message}</Typography>
        <Typography className={classes.tags} gutterBottom variant="h6" color="textSecondary" component="h2">{post.tags.map((tag) => `#${tag}`)}</Typography>
        <Typography className={classes.creator} variant="h6" style={{ fontFamily: 'GloryRegular' }}>Post by:
        <Link to={`/${creators}/${post.name}`} className={classes.creatorName}>
          {`${post.name}`}
        </Link>
        </Typography>
        <Typography variant="body1" style={{ fontFamily: 'GloryRegular' }}>{moment(post.createdAt).fromNow()}</Typography>
        <Divider />
      </div>
      <div className={classes.imageSection}>
        <img className={classes.media} src={post.selectedFile} alt={post.title} />
      </div>
    </div>

    {recommendedPosts.length > 0 ? (
      <div className={classes.recommendedSection}>
        <Typography gutterBottom variant="h5" style={{ fontFamily: 'GloryRegular' }}>Other people posted:</Typography>
        <Divider />
        <div className={classes.recommendedPosts}>
          {recommendedPosts.map(({ title, name, message, likes, selectedFile, _id }) => (
            <Card className={classes.recommendedCard}>
              <div style={{ margin: '20px', cursor: 'pointer' }} onClick={() => openPost(_id)} key={_id}>
                <Typography className={classes.othersTitle} gutterBottom variant="h6">{title}</Typography>
                <Typography className={classes.othersName} gutterBottom variant="subtitle2">by: {name}</Typography>
                <Typography className={classes.briefMessage} gutterBottom variant="subtitle2">{message.split(' ').splice(0, 80).join(' ')}...</Typography>
                <img src={selectedFile} width="200px" alt="" />
              </div>
            </Card>
          ))}
        </div>
      </div>
    ) : null}
  </Paper>
)
export default PostDetails;
```

Αυτό το component υλοποιεί την σελίδα της ανάρτησης δομώντας τις πληροφορίες που έχει καταχωρίσει ο χρήστης.

POSTS

Εικόνα 3.67 Αρχείο components/posts/posts.js

```
import React from 'react';
import Post from '../Post/Post';
import useStyles from '../styles';
import { useSelector } from 'react-redux';
import { Grid, CircularProgress } from '@material-ui/core';

const Posts = ({ setCurrentId }) => {
  const classes = useStyles();
  const { posts, isLoading } = useSelector((state) => state.posts);

  if(!posts.length && !isLoading) return <h1 className={classes.noPosts}>We didn't find what you're looking for.</h1>;

  return (
    <div>
      {isLoading ? <CircularProgress style={{'color': '#a5a5af', 'width': '100px', 'height': '100px', 'position': 'relative', 'left': '50%'}} /> : (
        <Grid className={classes.container} container alignItems="stretch" spacing={3}>
          {posts.map((post) => (
            <Grid key={post._id} item xs={12} sm={12} md={6} lg={6}>
              <Post post={post} setCurrentId={setCurrentId} />
            </Grid>
          ))}
        </Grid>
      )}
    </div>
  );
};

export default Posts;
```

Το component Posts καλεί το component Post και ορίζει πως θα εμφανίζεται όλα τα post, στην σελίδα που θα κληθεί το component Posts.

POST

Εικόνα 3.68 Αρχείο components/post/post.js

```
return {
  <Card className={classes.card} raised elevation={6}>
    <ButtonBase className={classes.cardAction} onClick={openPost}>
      <CardMedia className={classes.media} image={post.selectedFile} title={post.title} />
      <div className={classes.overlay}>
        <Typography variant="h6">{post.name}</Typography>
        <Typography variant="body2">{moment(post.createdAt).fromNow()}</Typography>
      </div>

      {(user?.result?.googleId === post?.creator || user?.result?.id === post?.creator) && code === 0} && {
        <div className={classes.overlay2} name="edit">
          <Button
            onClick={(e) => {
              e.stopPropagation();
              setCurrentId(post._id);
            }}
            style={{ color: 'white' }}
            size="small"
          >
            <MoreHorizIcon fontSize="medium" />
          </Button>
        </div>
      )}
    <div className={classes.details}>
      <Typography variant="body2" color="textSecondary" component="h2">{post.tags.map((tag) => `#${tag} `)}</Typography>
    </div>
    <Typography className={classes.title} gutterBottom variant="h5" component="h2">{post.title}</Typography>
    <CardContent>
      <Typography variant="body2" color="textSecondary" component="p">{post.message.split(' ').splice(0, 20).join(' ')}...</Typography>
    </CardContent>
    <ButtonBase>
      <CardActions className={classes.cardActions}>
        <Button size="small" color="primary" disabled={!user?.result} onClick={() => dispatch(likePost(post._id))}>
          <Likes />
        </Button>
        {(user?.result?.googleId === post?.creator || user?.result?.id === post?.creator) && {
          <Button size="small" className={classes.delete} onClick={() => dispatch(deletePost(post._id))}>
            <DeleteIcon className={classes.delete} fontSize="small" /> &nbsp;Delete
          </Button>
        )}
      </CardActions>
    </Card>
  </div>
};
export default Post;
```

Το component Post καθορίζει πως θα γίνει η διάταξη των πληροφοριών της δημοσίευσης στην προεπισκόπηση της.

PAGINATION

Εικόνα 3.69 Αρχείο components/pagination.jsx

```
import React, { useEffect } from 'react';
import { Pagination, PaginationItem } from '@material-ui/lab';
import { Link } from 'react-router-dom';
import { useDispatch, useSelector } from 'react-redux';

import { getPosts } from '../actions/posts';
import useStyles from './styles';

const Paginate = ({ page }) => {
  const { numberOfPages } = useSelector((state) => state.posts);
  const classes = useStyles();
  const dispatch = useDispatch();

  useEffect(() => {
    if (page) {
      dispatch(getPosts(page));
    }
  }, [dispatch, page]);

  return (
    <Pagination
      className={classes.paginationBar}
      classes={{ ul: classes.ul }}
      count={numberOfPages}
      page={Number(page) || 1}
      variant="outlined"
      color="primary"
      renderItem={({ item }) => (
        <PaginationItem {...item} component={Link} to={`/${posts?page=${item.page}`} />
      )}
    />
  );
};

export default Paginate;
```

Σε αυτό το αρχείο γίνεται η υλοποίηση της σελιδοποίησης των δημοσιεύσεων.

ΚΕΦΑΛΑΙΟ 4

Συμπεράσματα και Μελλοντικές Βελτιώσεις

4.1 Συμπεράσματα

Κατά την ενασχόληση με την δημιουργία της διαδικτυακής εφαρμογής Travel Light, οι περισσότερες δυσκολίες που αντιμετωπίστηκαν αφορούσαν το προγραμματιστικό κομμάτι σε σχέση με το σχεδιαστικό κομμάτι της εφαρμογής.

Ωστόσο, οι στόχοι για την ανάπτυξη της εφαρμογής επιτεύχθηκαν ώστε η εφαρμογή να εξυπηρετεί τις ανάγκες των χρηστών της. Ουσιαστικά οι χρήστες δημιουργούν το περιεχόμενο της πλατφόρμας. Ο κάθε χρήστης μπορεί να δημιουργήσει λογαριασμό στην εφαρμογή και να προσθέσει περιεχόμενο κοινοποιώντας εμπειρίες, συμβουλές, ταξίδια, φωτογραφίες. Μπορεί να εξερευνήσει δημοσιεύσεις συγκεκριμένων χρηστών οι οποίες μπορούν να βοηθήσουν στην διοργάνωση κάποιου ταξιδιού ή στην παρότρυνση μιας εμπειρίας.

Η εύκολη λύση, λόγω περιορισμένων κόστους, για το deployment της εφαρμογής ήταν οι δωρεάν λύσεις που προσφέρουν το heroku για το backend και το netlify για το frontend.

Συμπερασματικά, προκύπτει ότι υπάρχει περιθώριο ανάπτυξης διαδικτυακών εφαρμογών κοινωνικών δικτύων με σύγχρονα εργαλεία και μεθόδους που μπορούν να προσφέρουν αξιόλογη βοήθεια στους χρήστες οι οποίοι δημιουργούν εικονικές κοινότητες. Τα κοινωνικά δίκτυα εξάλλου βασίζονται στη δικτύωση των χρηστών μέσω μορφών αλληλεπίδρασης, που υποστηρίζουν όλες τις διαστάσεις της κοινωνικοποίησης τους γι αυτό και είναι εξαιρετικά δημοφιλή. Η δημοφιλία του κάθε κοινωνικού δικτύου οφείλεται στον όγκο των αναρτήσεων των χρηστών.

Το link για την εφαρμογή είναι το εξής <https://travel-light-app.netlify.app/> και ο κώδικας της βρίσκεται στο GitHub <https://github.com/iotzio24/travellight>.

Στην επόμενη ενότητα θα αναλυθούν οι μελλοντικές αναπτύξεις της εφαρμογής.

4.2 Μελλοντικές Βελτιώσεις

Η εφαρμογή θα μπορούσε να γίνει πιο λειτουργική προσθέτοντας επιπλέον χαρακτηριστικά. Η επιπλέον ανάπτυξη της εφαρμογής θα ενισχύσει το user engagement σε αυτή κάνοντάς την πιο χρηστική. Επιπλέον προσθήκες στις δυνατότητες της εφαρμογής με νέες τεχνολογίες είναι απαραίτητες, καθώς οι απαιτήσεις, οι συνήθειες και οι προσδοκίες των χρηστών εξελίσσονται διαρκώς οπότε επιβάλλεται συνεχής παρακολούθηση για να μην επέλθει κορεσμός. Η εφαρμογή δεν έχει bugs προς επίλυση ωστόσο μερικές προσθήκες που θα μπορούσαν να γίνουν είναι οι εξής:

- Περισσότερα φίλτρα στην σελίδα των δημοσιεύσεων.
- Βαθμολογία σε κάθε μέρος ή εμπειρία.
- Πιο εξειδικευμένο ερωτηματολόγιο για συμπλήρωση στη σελίδα Add a Post ανάλογα αν είναι καταχώρηση για έναν προορισμό ή καταχώρηση εμπειρίας.
- Σε κάθε καταχώρηση δημοσίευσης να υπάρχουν εξειδικευμένα tags που θα επιλέγει ο χρήστης για να αξιοποιούνται ως φίλτρα στη σελίδα δημοσιεύσεων. Για παράδειγμα, για έναν προορισμό ο χρήστης θα επιλέγει αν ο προορισμός είναι κατάλληλος για παιδιά.
- Για την ενίσχυση του κοινωνικού δικτύου θα μπορούσε να προστεθεί η λειτουργικότητα των σχολίων κάτω από κάθε δημοσίευση ώστε οι χρήστες να είναι σε επικοινωνία αν χρειάζονται διευκρινίσεις.
- Πιο αναλυτικό προφίλ χρήστη το οποίο θα είναι εμφανές σε όλους τους χρήστες. Θα μπορούν να υπάρχουν προσωπικές ερωτήσεις για τον κάθε χρήστη ώστε στο προφίλ του να υπάρχει μια μπάρα που θα εμφανίζει σε ποσοστό τις απαντήσεις του. Για παράδειγμα, οι ερωτήσεις θα αφορούν το travel frequency του χρήστη, το sightseeing intensity που έχει κλπ.
- Ενσωμάτωση χάρτη με δυνατότητα κοινοποίησης παρουσίας.
- Να μεταφερθεί σε πληρωμένο domain χωρίς να βασίζεται στις δωρεάν λύσεις του heroku και netlify.

ΒΙΒΛΙΟΓΡΑΦΙΑ

[1] Σωτηριάδου, Α. & Παπαδάκης, Σ. (2012). Αξιοποίηση των Κοινωνικών Δικτύων για τη Διδακτική της Πληροφορικής σε Ενηλίκους – Βιβλιογραφική ανασκόπηση. 6ο Πανελλήνιο Συνέδριο Διδακτική της Πληροφορικής, 417-426, Φλώρινα

[2] Παπάνης, Γιαβρίμης, Βίκη & Παπάνης, 2011

[3]<http://apothetirio.teiep.gr/xmlui/bitstream/handle/123456789/4858/866.pdf?sequence=1> , Πτυχιακή Εργασία

[4] <https://onlinelibrary.wiley.com/doi/full/10.1111/j.1083-6101.2007.00393.x>

[5] https://repository.kallipos.gr/bitstream/11419/728/3/02_chapter_06.pdf

[6] A. Mislove, “Online Social Networks: Measurement, Analysis and Applications to Distributed Information Systems”, PhD Thesis υποβλήθηκε στο Rice University, Houston, 2009.

[7] A. Mislove, M. Marcon, K. P. Gummadi, P. Druschel, and B. Bhattacharjee, “Measurement and Analysis of Online Social Networks”, in Proceeding of the 5th ACM/USENIX Internet Measurement Conference (IMC’07), 2007.

[8] N. Papailiou, D. Apostolou and G. Mentzas, “Social Networks for Knowledge Management in Management Consulting Firms”, in Proceeding of the 4th Conference Professional Knowledge Management Experiences and Vicious, GITO-Verlag, 2007.

[9]https://commons.wikimedia.org/wiki/File:Social_Network_Analysis_Visualization.png

[10]http://dione.lib.unipi.gr/xmlui/bitstream/handle/unipi/12298/Evangelatou_%20E_MBA1519.pdf?sequence=1&isAllowed=y Πτυχιακή Εργασία

[11]<https://dspace.lib.uom.gr/bitstream/2159/22904/4/ChopteridisIoannisMsc2019.pdf> f Μεταπτυχιακή Εργασία

[12] <https://www.intechopen.com/books/social-media-and-journalism-trends-connections-implications/evolving-networks-and-social-network-analysis-methods-and-techniques> Άρθρο σε επιστημονικό περιοδικό

[13] <https://www.annalsofrscb.ro/index.php/journal/article/view/7719/5721> Article

[14]

https://books.google.gr/books?hl=el&lr=&id=m86PDwAAQBAJ&oi=fnd&pg=PP1&dq=material+ui+library&ots=gnReazZl14&sig=i6r4ck3qjm68FNREwYtqgbYiYag&redir_esc=y#v=twopage&q&f=true Ηλεκτρονικό Βιβλίο, React Material-UI Cookbook, Adam Boduch

- [15] MUI Documentation
- [16] https://books.google.gr/books?hl=el&lr=&id=4QKACwAAQBAJ&oi=fnd&pg=PP1&dq=nodejs+&ots=SdUCTKsfvF&sig=n-eMRb8jwz0oFVK0K-d8RrCW0xo&redir_esc=y#v=onepage&q&f=false ebook page 5
- [17] https://en.wikipedia.org/wiki/Google_Maps
- [18] https://www.researchgate.net/figure/A-screen-capture-from-a-smartphone-of-the-Google-Maps-Android-app-showing-possible_fig4_341937967
- [19] <https://www.advicelocal.com/blog/google-maps-respond-google-my-business-reviews/>
- [20] <https://www.wired.com/story/how-to-google-maps-social-tips/>
- [21] <https://jonvadillo.com/nested-react-components/>
- [22] <https://amplifywebpro.com/>
- [23] <https://www.educative.io/edpresso/what-is-express-body-parser>
- [24] https://books.google.gr/books?hl=el&lr=&id=097dDwAAQBAJ&oi=fnd&pg=PP1&dq=mern+stack+applications&ots=CMG4fhh7v2&sig=eZlwj1U_kfSjFgK6wnjKxEnT9C8&redir_esc=y#v=onepage&q=mern%20stack%20applications&f=false
- [25] https://www.theseus.fi/bitstream/handle/10024/495427/FINAL_VERSION.pdf?sequence=2&isAllowed=y
- [26] <https://www.theseus.fi/bitstream/handle/10024/498420/Bang%20Nguyen%20-%20Thesis.pdf?sequence=2&isAllowed=y>
- [27] https://www.theseus.fi/bitstream/handle/10024/502110/Cuong_Cao_Nguyen.pdf?sequence=2
- [28] https://www.theseus.fi/bitstream/handle/10024/343968/Thesis_Purna_Baral.pdf?sequence=2&isAllowed=y
- [29] <https://uol.de/f/2/dept/informatik/ag/svs/download/reader/reader-seminar-ws2016.pdf#page=18>
- [30] https://books.google.gr/books?hl=el&lr=&id=uGUKiNkKRJOC&oi=fnd&pg=PP1&dq=mongodb+web+application&ots=halzPfbSsh&sig=QGsq3YpK9s-BxfIKBeXScXsicjY&redir_esc=y#v=onepage&q=mongodb%20web%20application&f=false
- [31] https://link.springer.com/chapter/10.1007/978-1-4842-3039-8_7
- [32] <http://www.ijaconline.com/wp-content/uploads/2021/01/27.pdf>

[33]

https://books.google.gr/books?hl=el&lr=&id=DxTFXO771tIC&oi=fnd&pg=PT12&dq=mongoose+javascript&ots=MK2FFTyZJ8&sig=hlrzGo3rMtstASwwWHJcgNt77go&redir_esc=y#v=onepage&q=mongoose%20javascript&f=false

[34] <http://www.ijeast.com/papers/151-154,Tesma510,IJEAST.pdf>

[35] <https://medium.com/techiepedia/what-exactly-a-mern-stack-is-60c304bffe4>